To do before we begin:

1. Install Docker from https://docs.docker.com/desktop/ then open it
2. In Terminal, download the Workshop library by running

   git clone https://github.com/dinhngockhanh/IICDWorkshopInferringClonality.**git**
3. In Terminal, go to the downloaded folder then run

   docker compose build

For more information, please scan the QR code.

Inferring Clonality from DNA

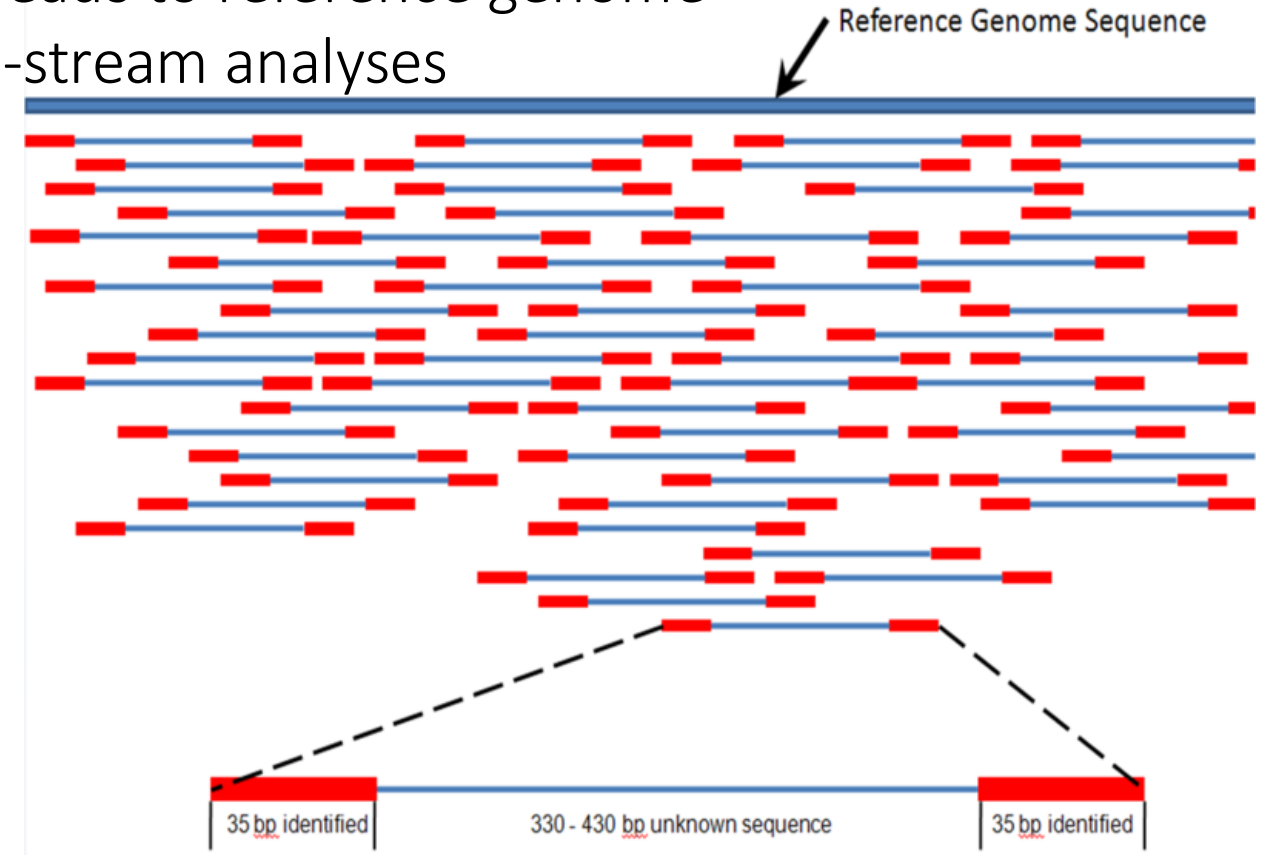
Yanjie Chen Khanh N. Dinh

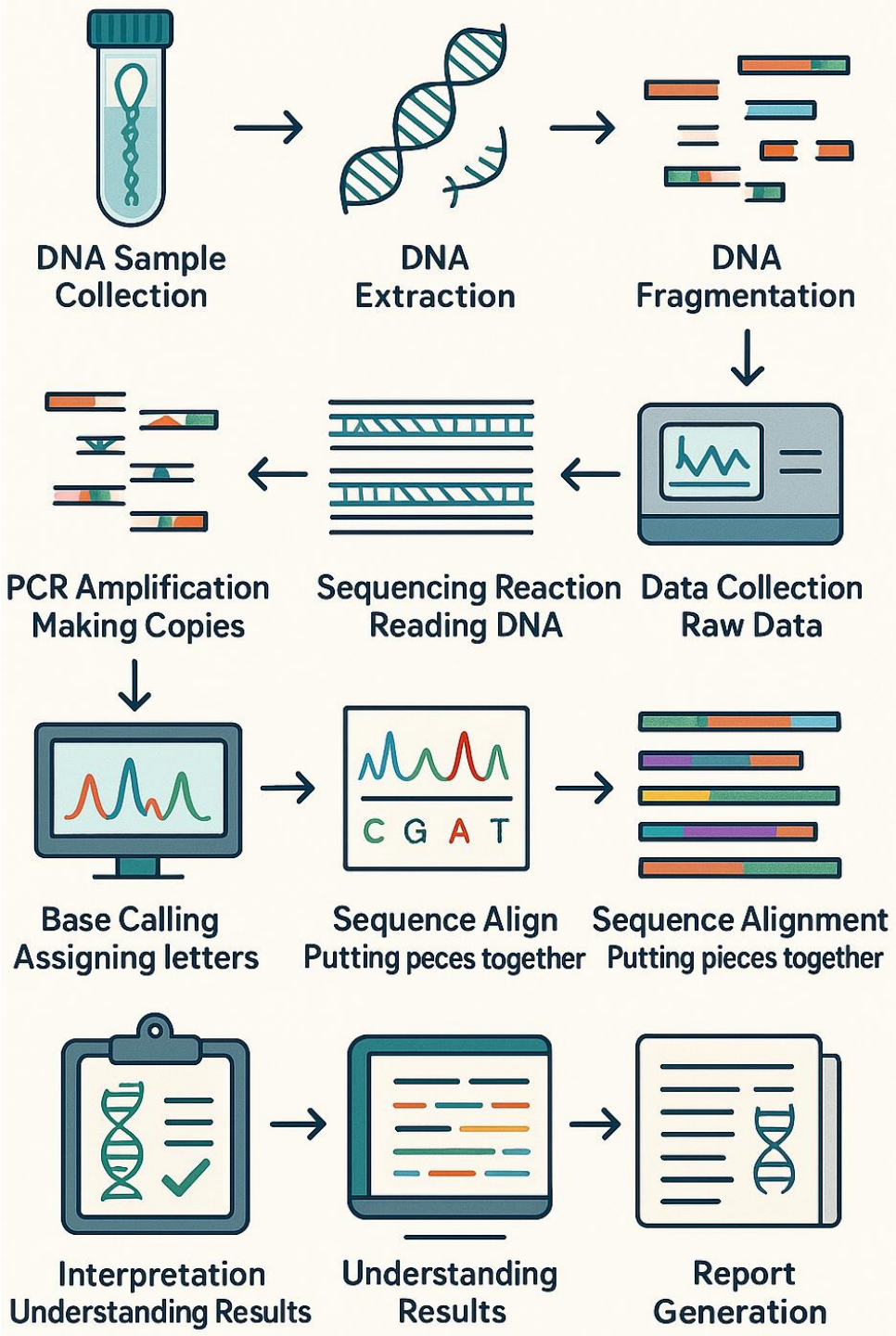
Irving Institute for Cancer Dynamics and Department of Statistics
Columbia University, New York, NY, USA

COLUMBIA | HERBERT AND FLORENCE IRVING INSTITUTE FOR CANCER DYNAMICS

Basics of DNA-sequencing

1. Collect DNA from sample
2. Fragment each chromosome into pieces of < 1000 nucleotides
3. Sequence some fragments
4. Map reads to reference genome
5. Down-stream analyses





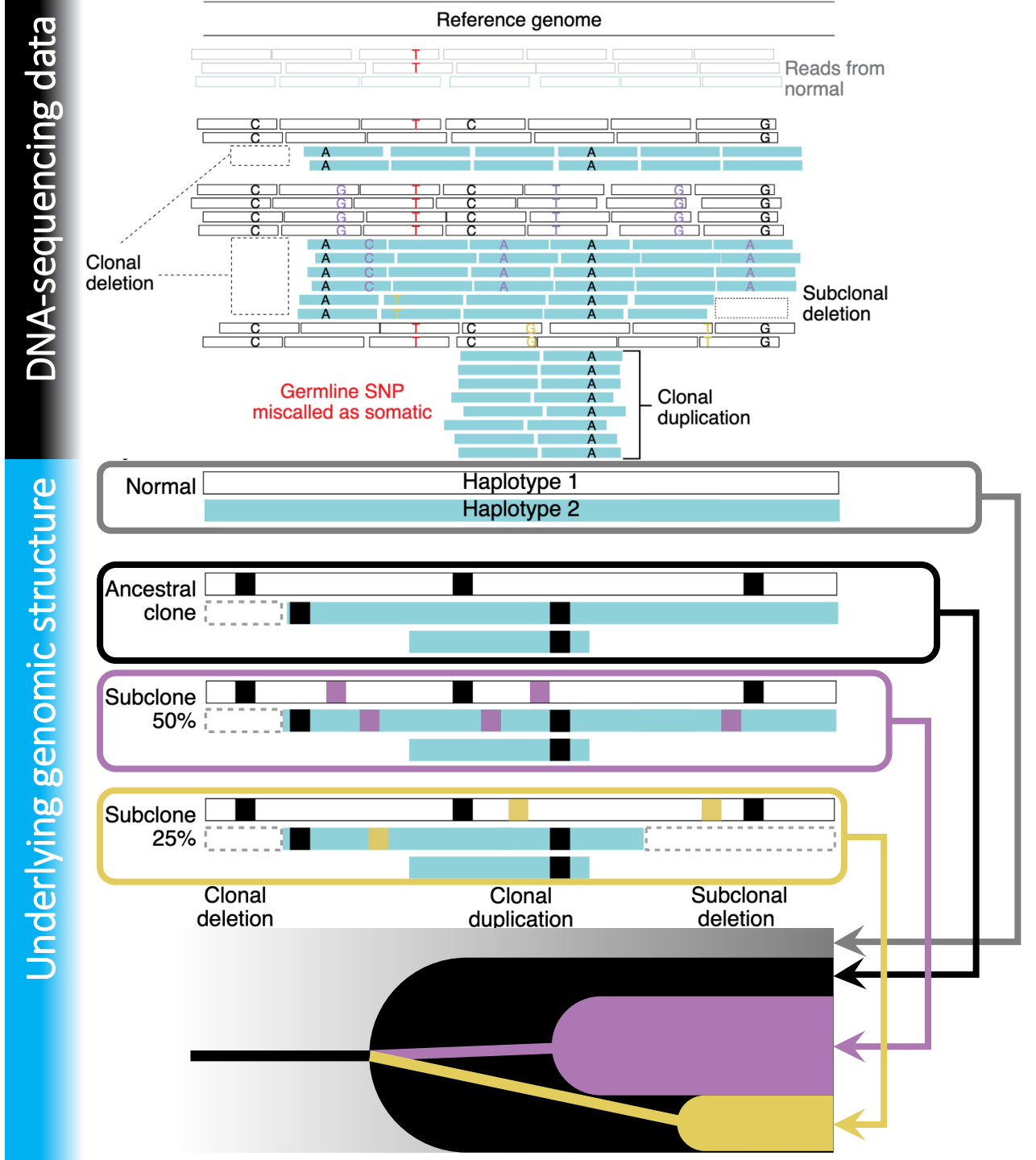
Definitions

- Intra-tumor heterogeneity (ITH) = presence of subclones in a tumor
- In general, an increase in ITH is associated with lower survival and higher risks of relapse and metastasis
- Therefore, inferring ITH from DNA-sequencing data is an important task in DNA-seq data analysis

Tarabichi M, Salcedo A, Deshwar AG et al. Nature methods. 2021.

Definitions

- Sample purity: % tumor cells in a sample
- Subclone: a subgroup of tumor cells characterized by the genotype of its most recent common ancestor (MRCA)
- (Sub)clonal mutations: mutations present in all cells of a (sub)clone
- Subclonal prevalence: % cells belonging to a subclone in the sample

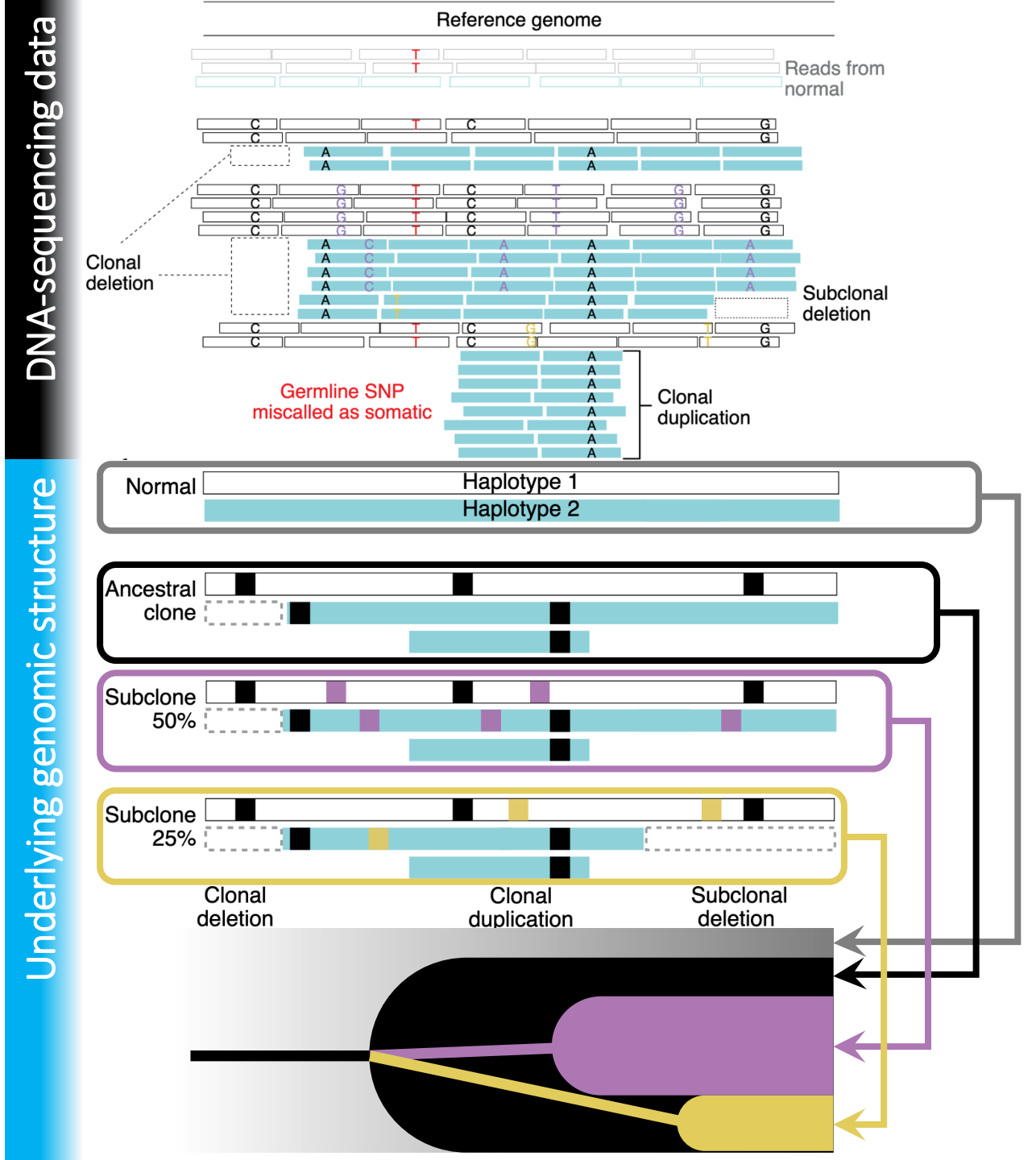


Tarabichi M, Salcedo A, Deshwar AG et al. Nature methods. 2021.

Definitions

Definitions for copy numbers (CNs):

- CN of allele A/B: # copies originating from haplotype A/B
- Major CN: $= \max(\text{CN A}, \text{CN B})$
- Minor CN: $= \min(\text{CN A}, \text{CN B})$
- Total CN: $= \text{CN A} + \text{CN B}$



Definitions

Definitions for copy numbers (CNs):

- logR: log ratio of normalized tumor's total CN against normalized normal total CN at site i; $= \log_2 \left(\frac{T_i/T}{N_i/N} \right)$ where

  T_i = # reads in tumor at site i

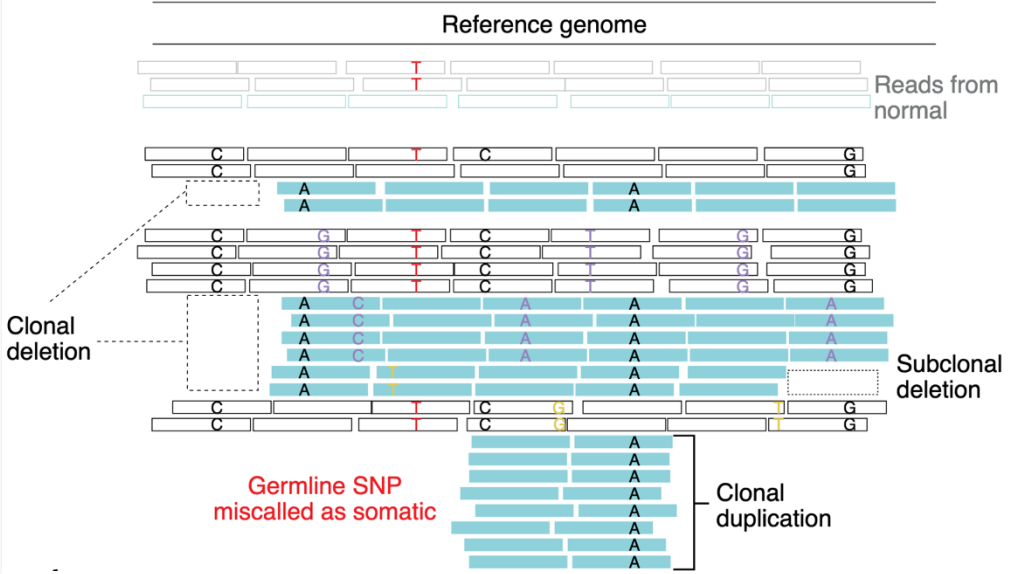
  T = mean # reads in tumor

  N_i = # reads in normal tissue at site i

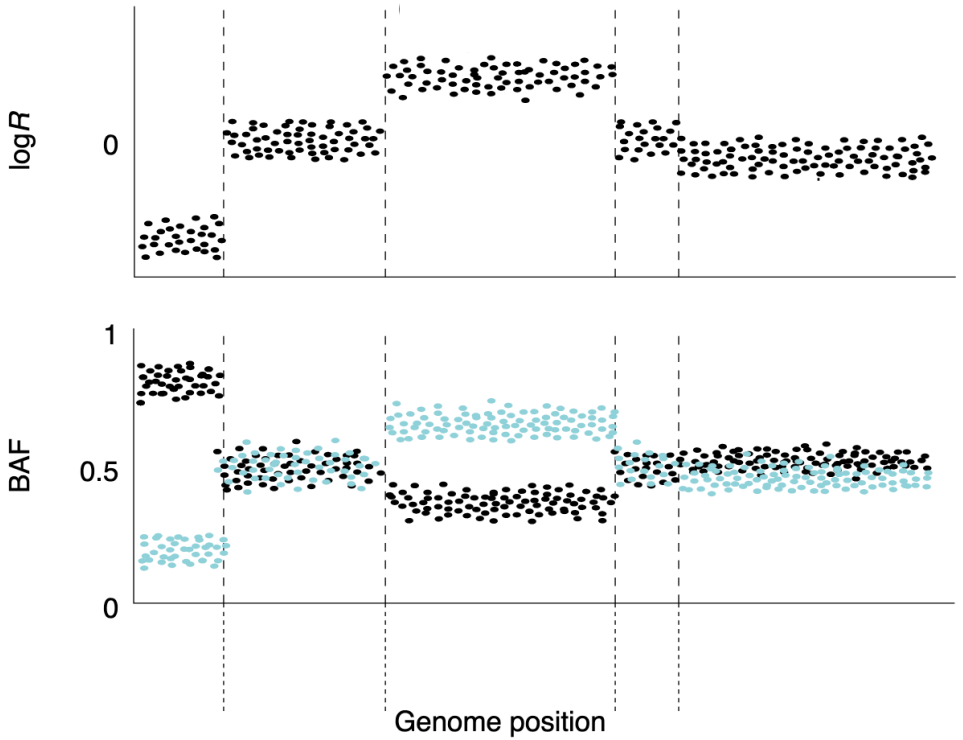
  N = mean # reads in normal tissue

- B allele frequency (BAF): % tumor reads that belong to allele B

DNA-sequencing data



Metrics for CN reconstruction



Definitions

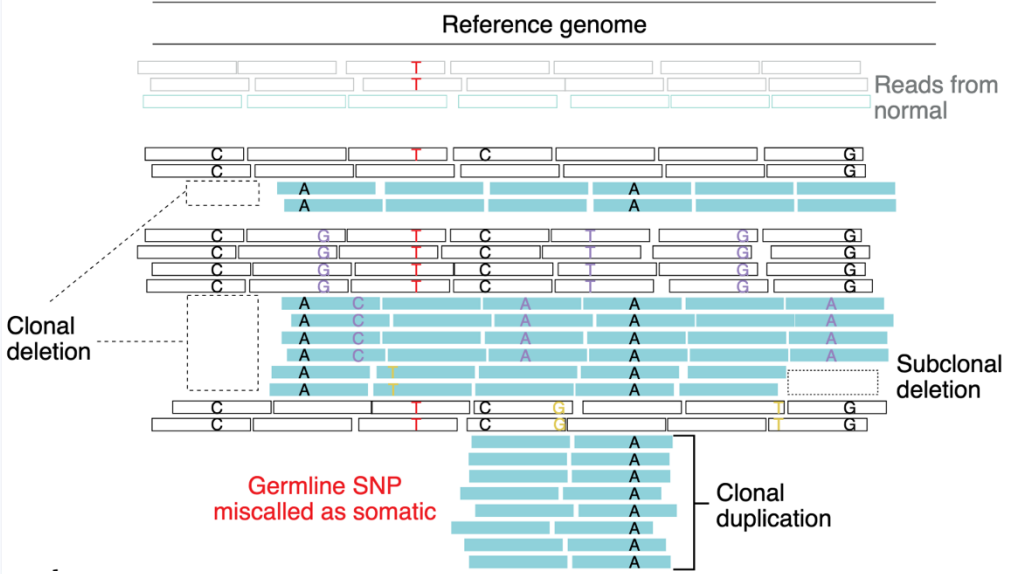
How do we infer CNs from logR/BAF?

- $\log R \approx 0 \Longrightarrow ?$

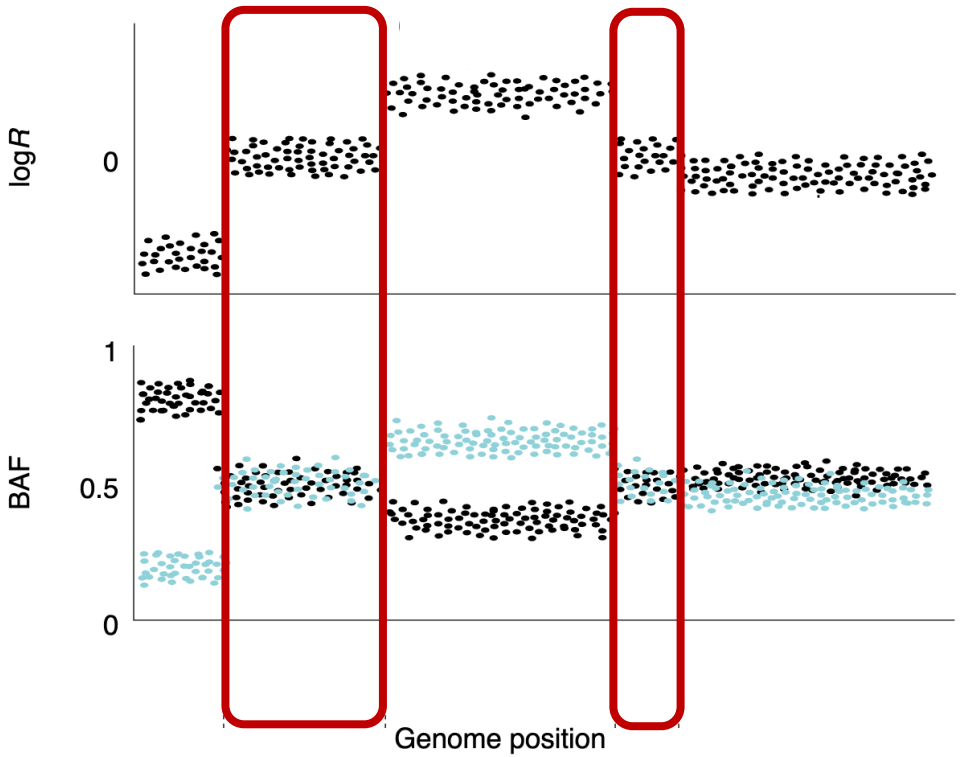
- $\log R = \log_2\left(\frac{T_i/T}{N_i/N}\right)$

- BAF = % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



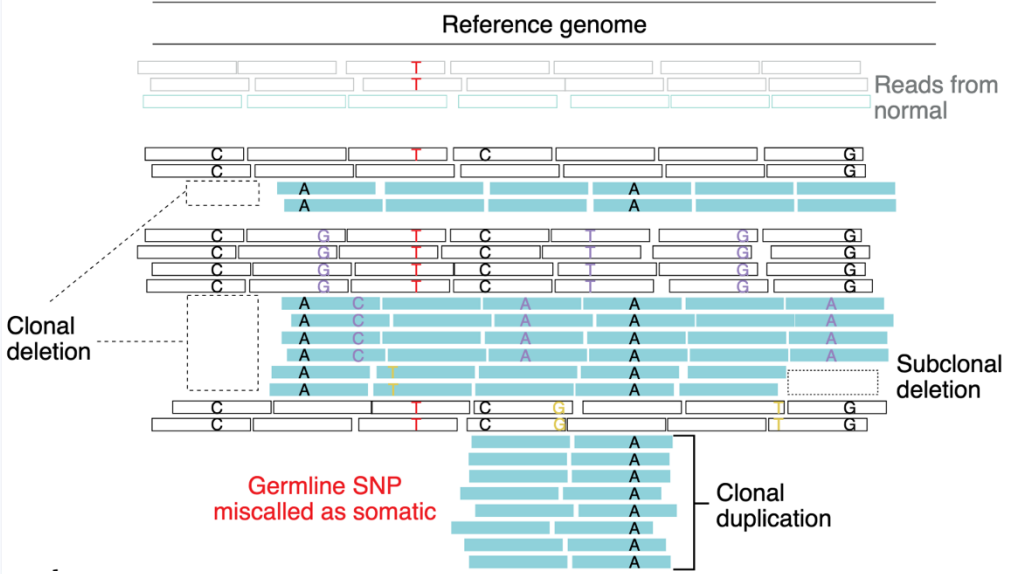
Definitions

How do we infer CNs from logR/BAF?

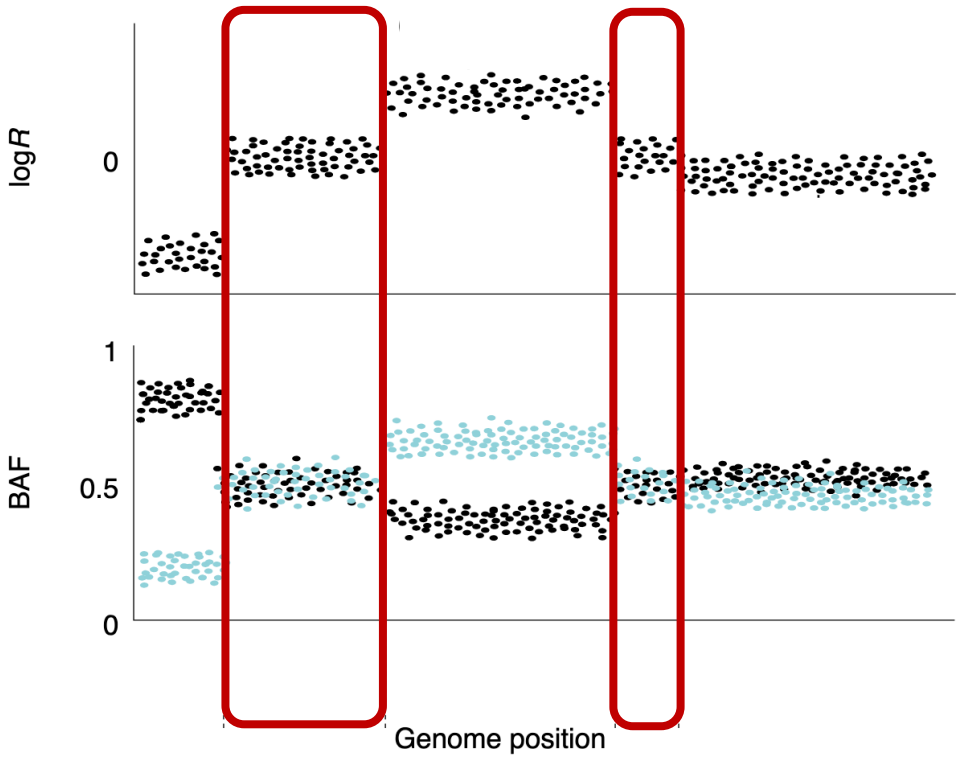
- logR $\approx 0 \Longrightarrow$ Total CN $= 2$
- BAF $\approx 0.5 \Longrightarrow$?

- logR $= \log_2 \left(\frac{T_i/T}{N_i/N} \right)$
- BAF $=$ % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



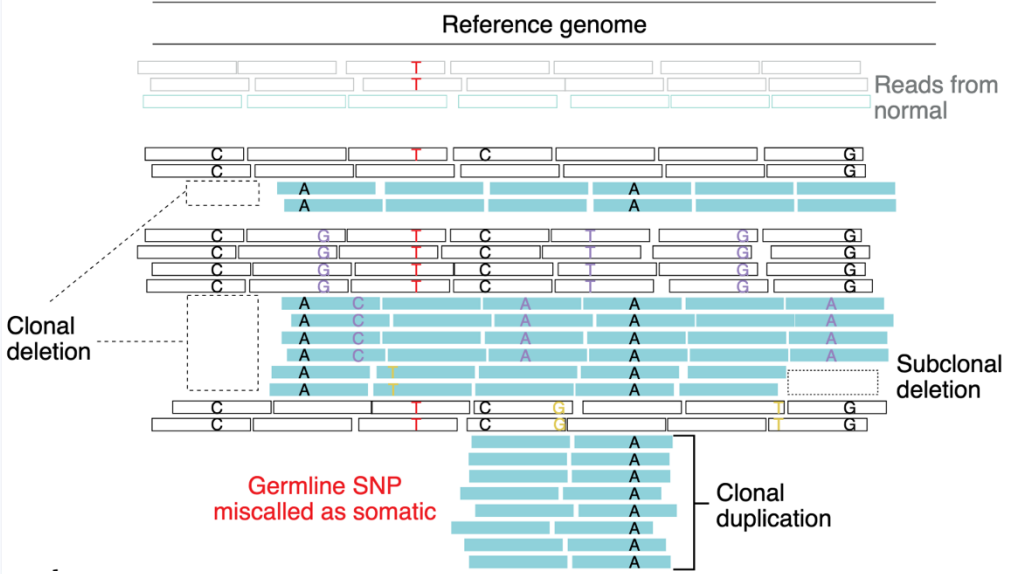
Definitions

How do we infer CNs from logR/BAF?

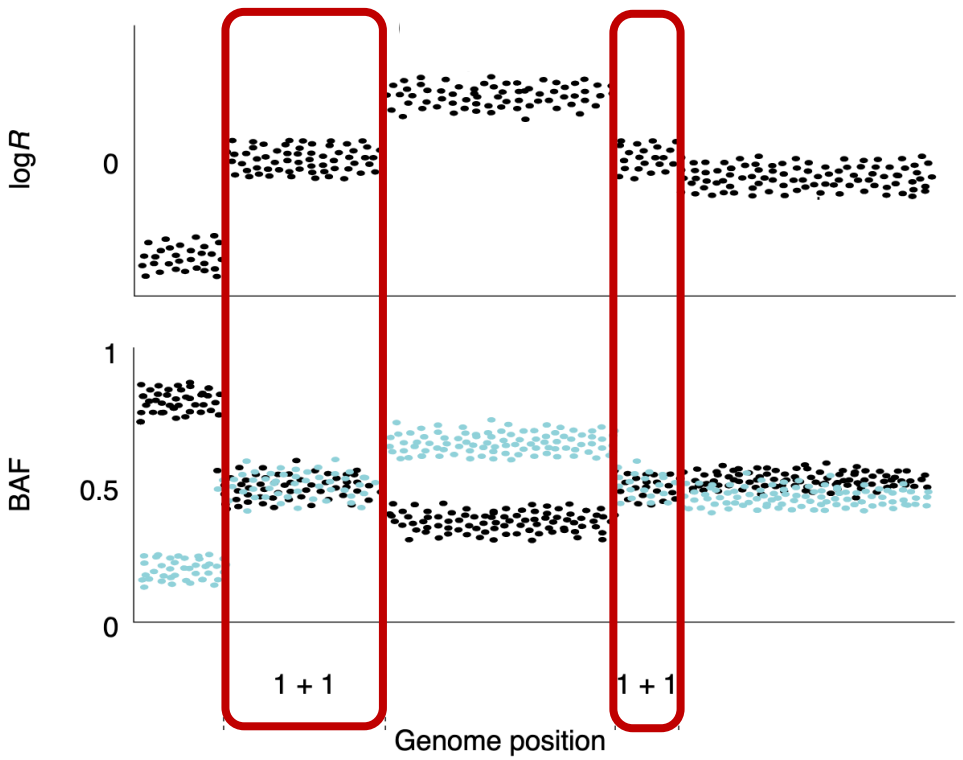
- logR $\approx 0 \Longrightarrow$ Total CN $= 2$
- BAF $\approx 0.5 \Longrightarrow$ CN A $=$ CN B

- logR $= \log_2 \left(\frac{T_i/T}{N_i/N} \right)$
- BAF $=$ % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



Definitions

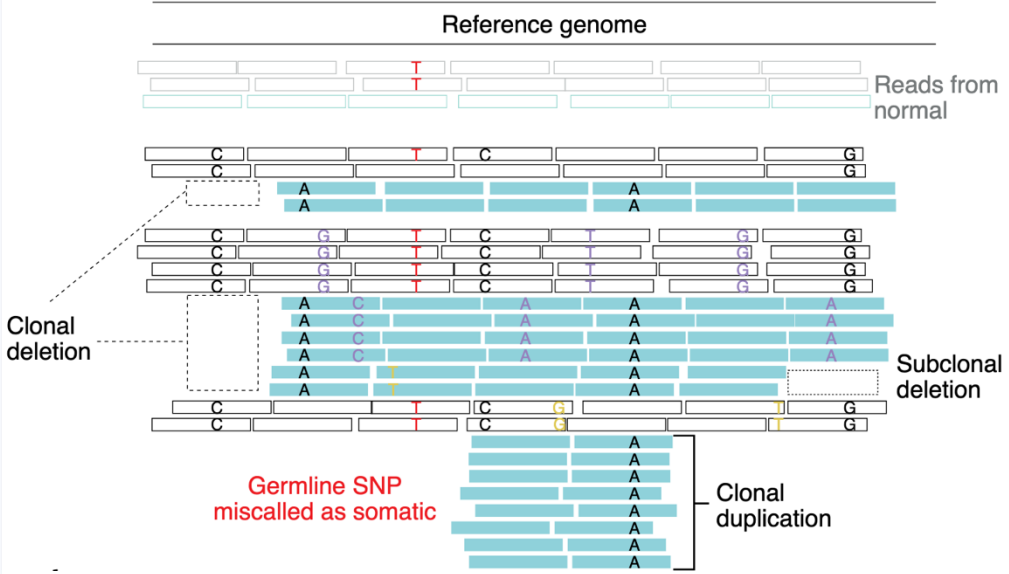
How do we infer CNs from logR/BAF?

- $\log R \approx -1 \Longrightarrow ?$

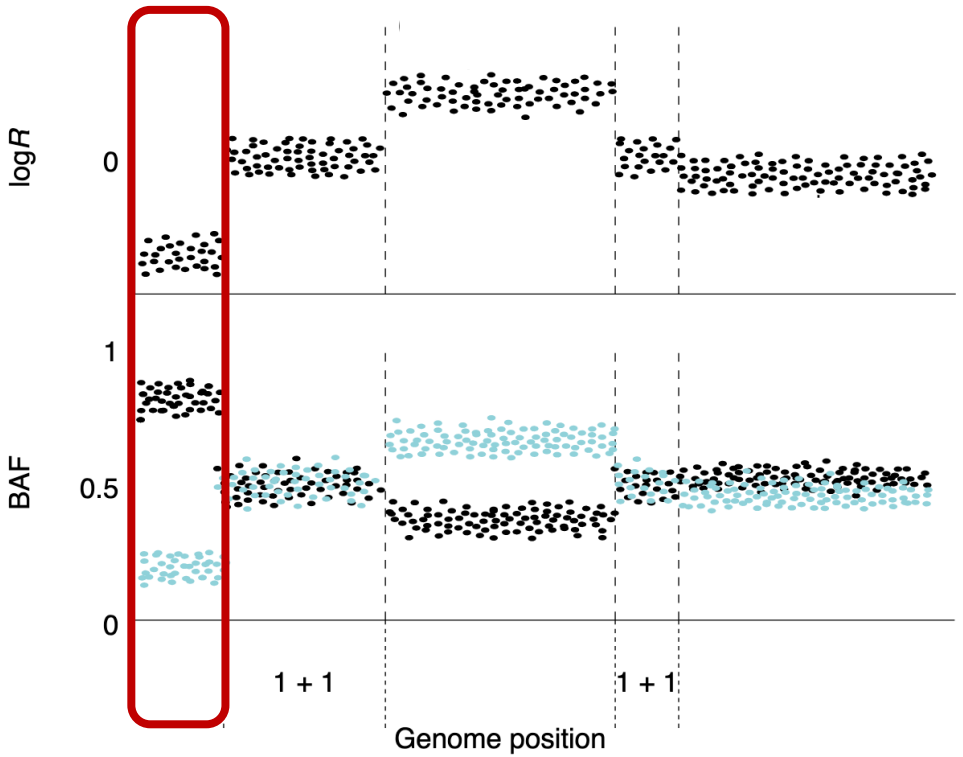
- $\log R = \log_2\left(\frac{T_i/T}{N_i/N}\right)$

- BAF = % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



Definitions

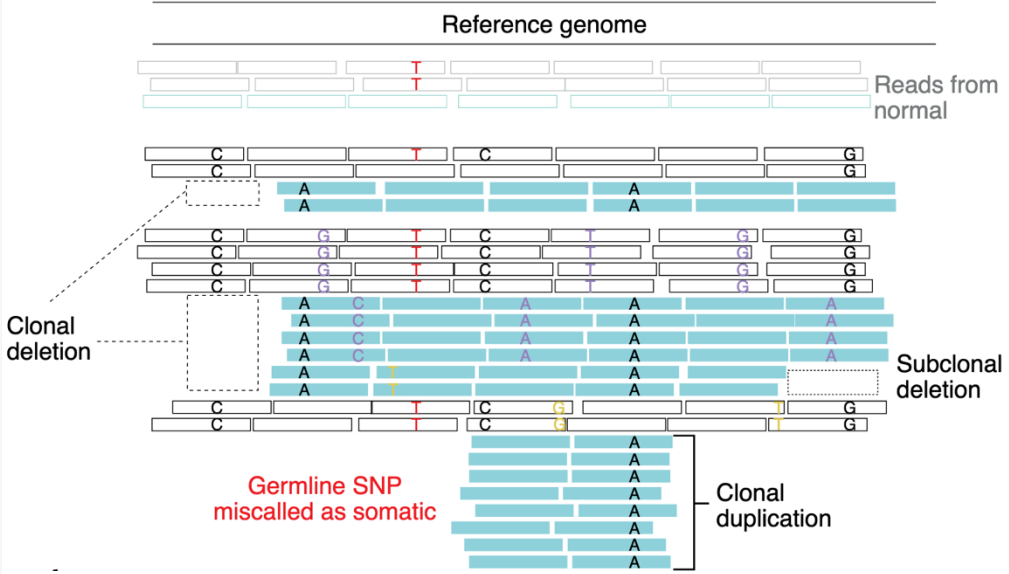
How do we infer CNs from logR/BAF?

- $\log R \approx -1 \Longrightarrow$ Total CN $= 1$

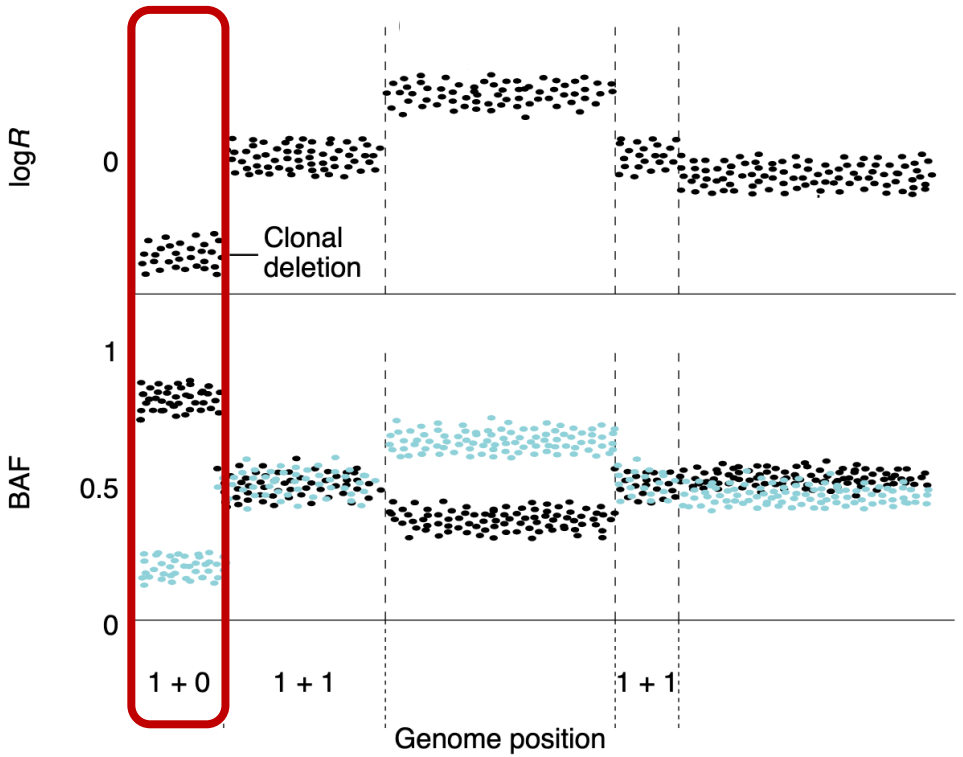
- $\log R = \log_2 \left(\frac{T_i/T}{N_i/N} \right)$

- BAF $=$ % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



Definitions

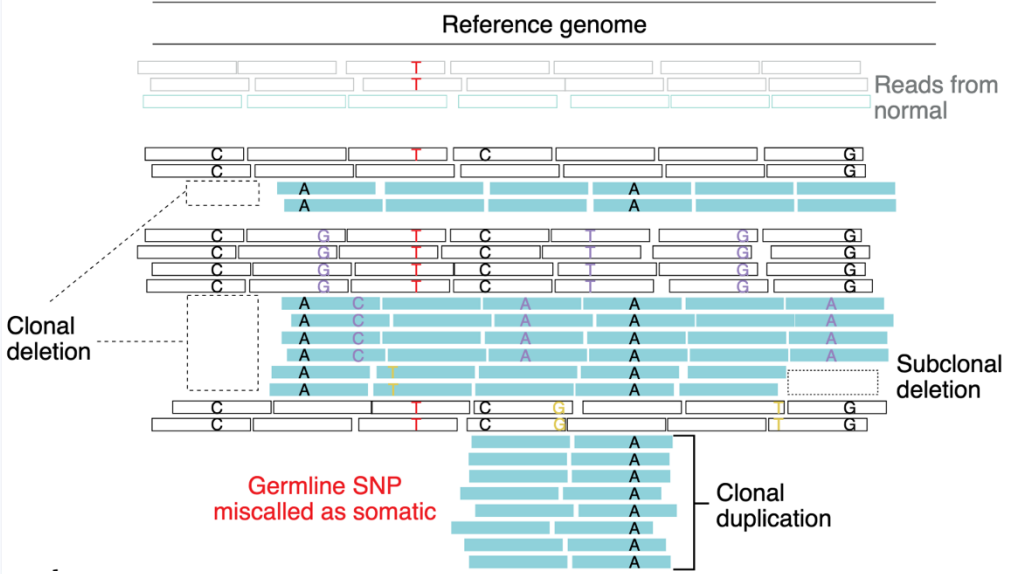
How do we infer CNs from logR/BAF?

- $\log R \approx 0.58 \Longrightarrow ?$

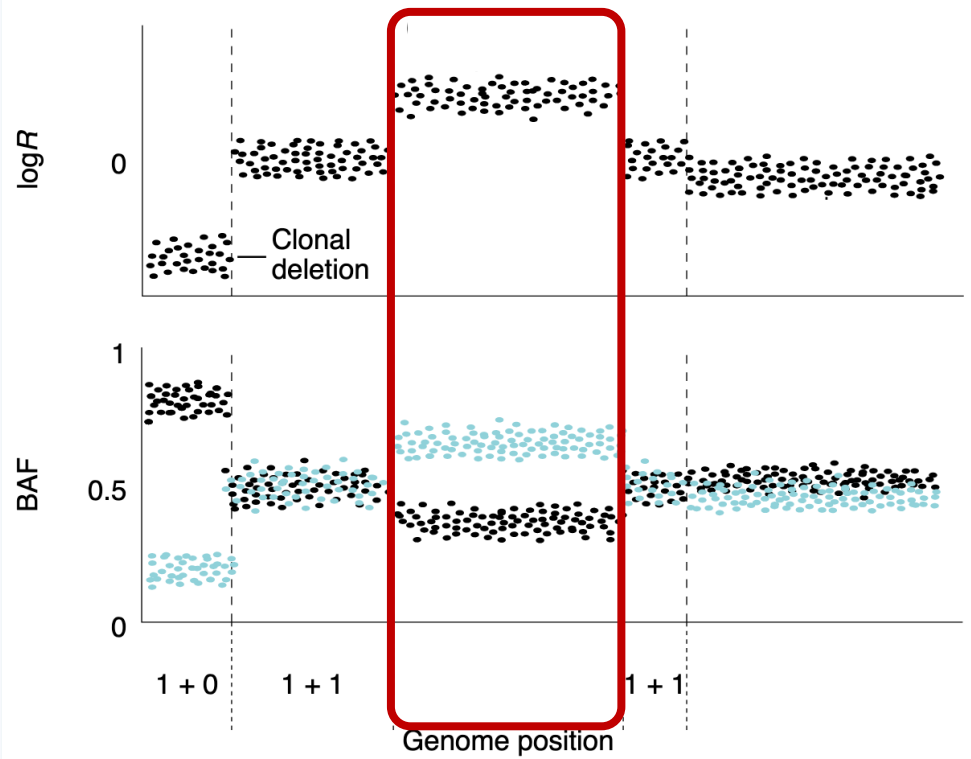
- $\log R = \log_2\left(\frac{T_i/T}{N_i/N}\right)$

- BAF = % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



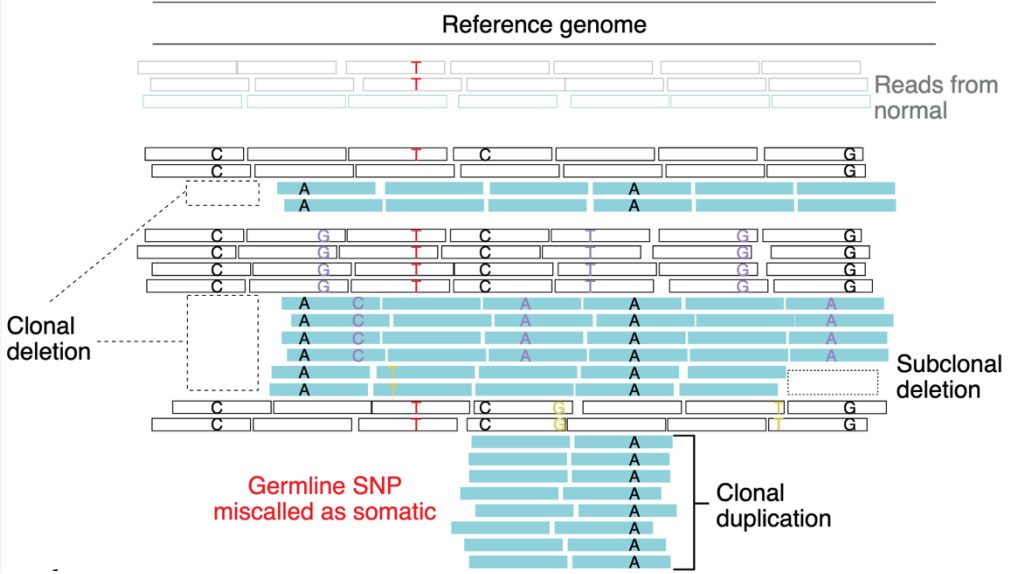
Definitions

How do we infer CNs from logR/BAF?

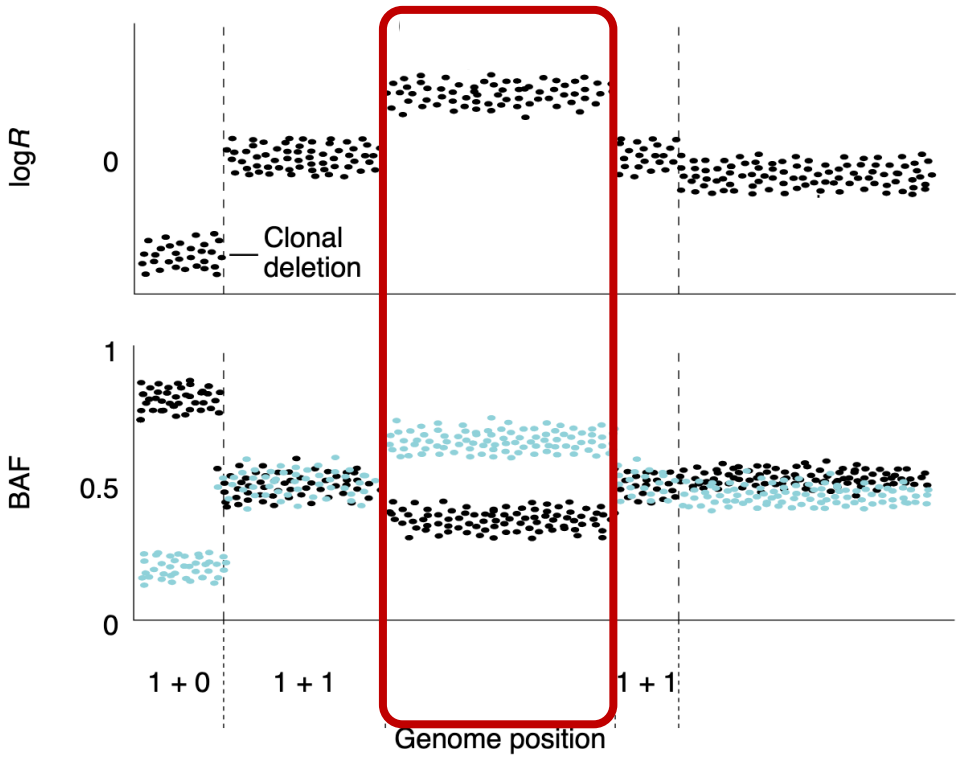
- logR $\approx 0.58 \Longrightarrow$ Total CN $= 3$
- BAF ≈ 0.33 or $0.66 \Longrightarrow$?

- logR $= \log_2 \left(\frac{T_i/T}{N_i/N} \right)$
- BAF $=$ % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



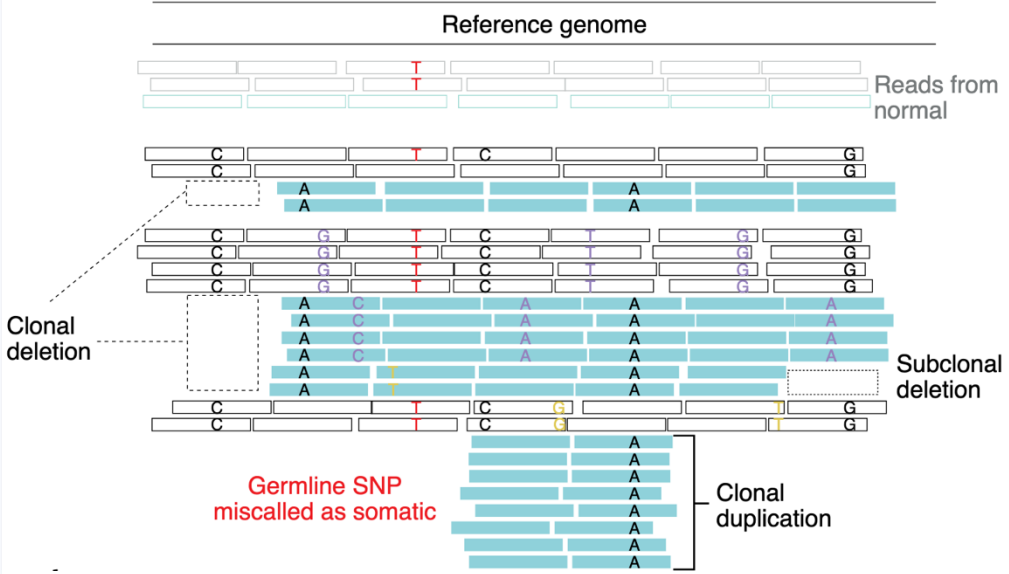
Definitions

How do we infer CNs from logR/BAF?

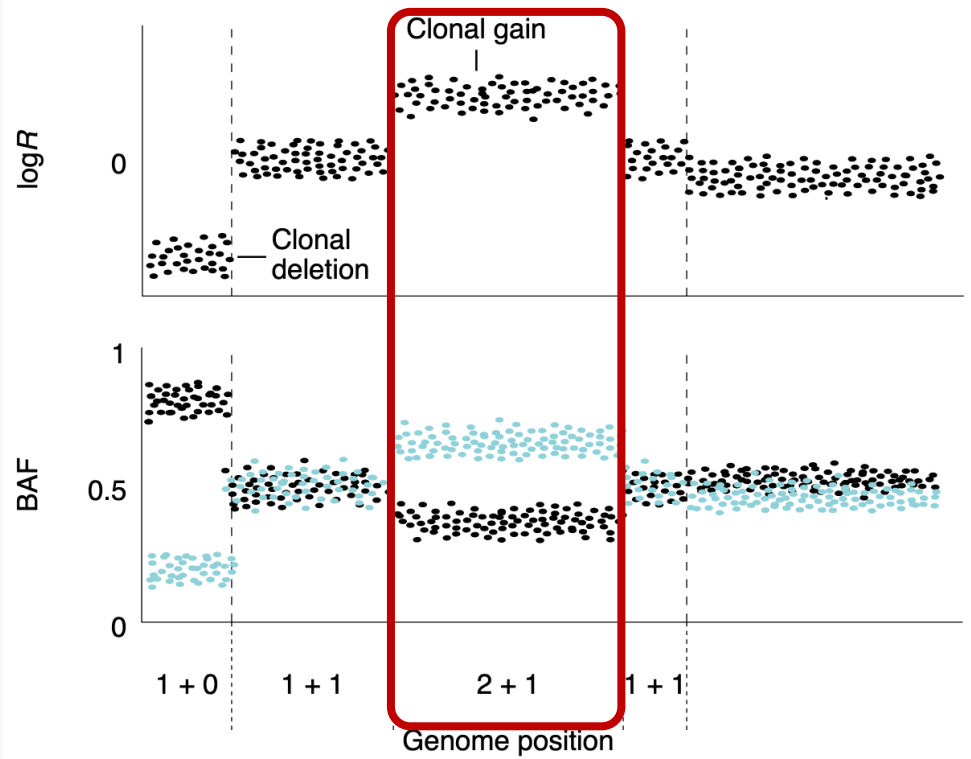
- logR $\approx 0.58 \Longrightarrow$ Total CN $= 3$
- BAF ≈ 0.33 or $0.66 \Longrightarrow$ CN B = 1 or 2

- logR $= \log_2 \left(\frac{T_i/T}{N_i/N} \right)$
- BAF $=$ % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



Definitions

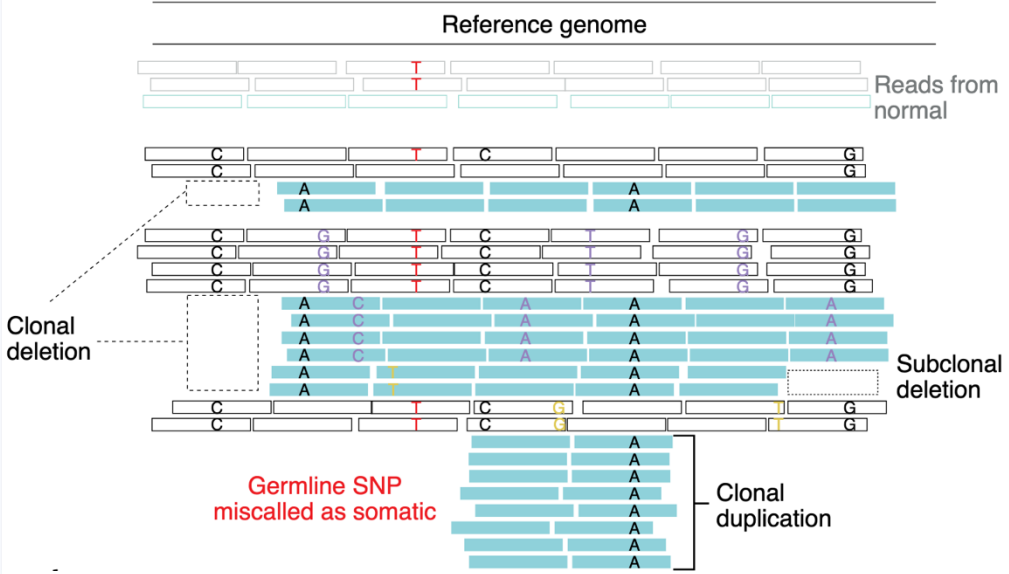
How do we infer CNs from logR/BAF?

- logR ≈ 0.8 and BAF ≈ 0.45 or $0.55 \Longrightarrow$?

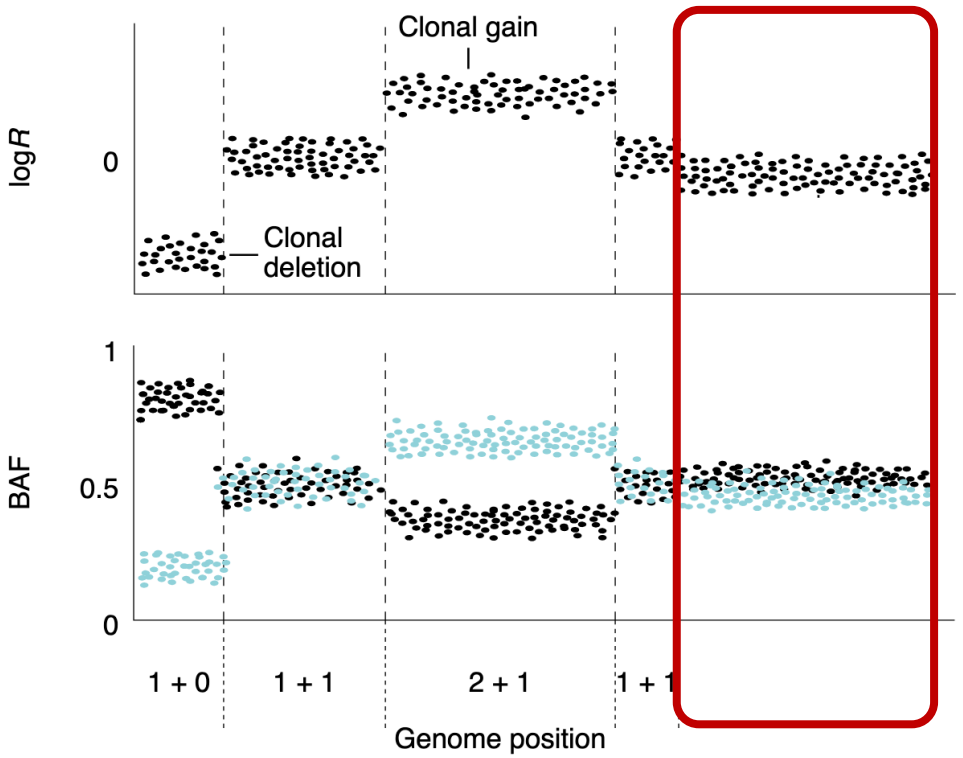
- $\text{logR} = \log_2\left(\frac{T_i/T}{N_i/N}\right)$

- BAF = % reads from allele B

DNA-sequencing data



Metrics for CN reconstruction



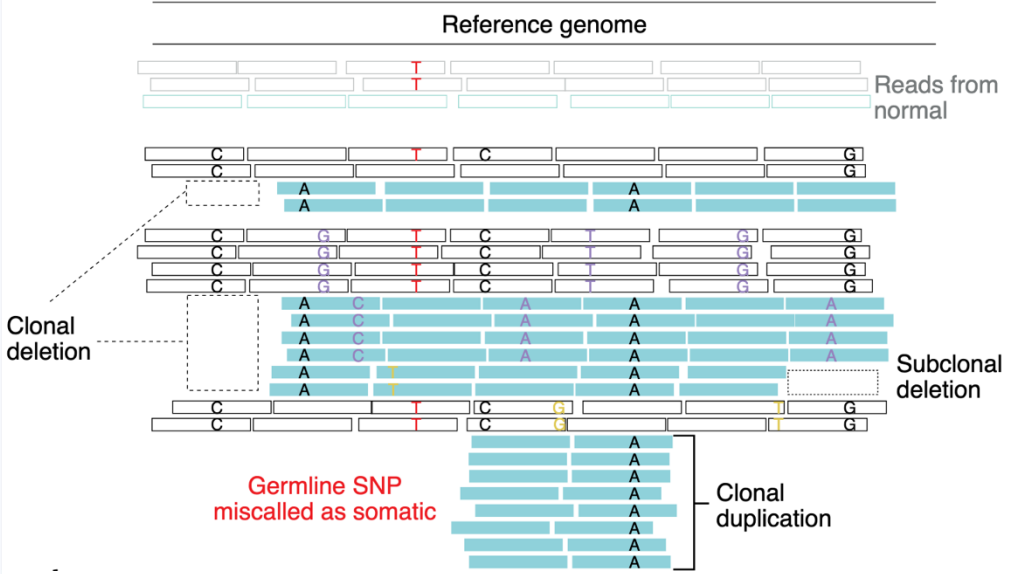
Definitions

How do we infer CNs from logR/BAF?

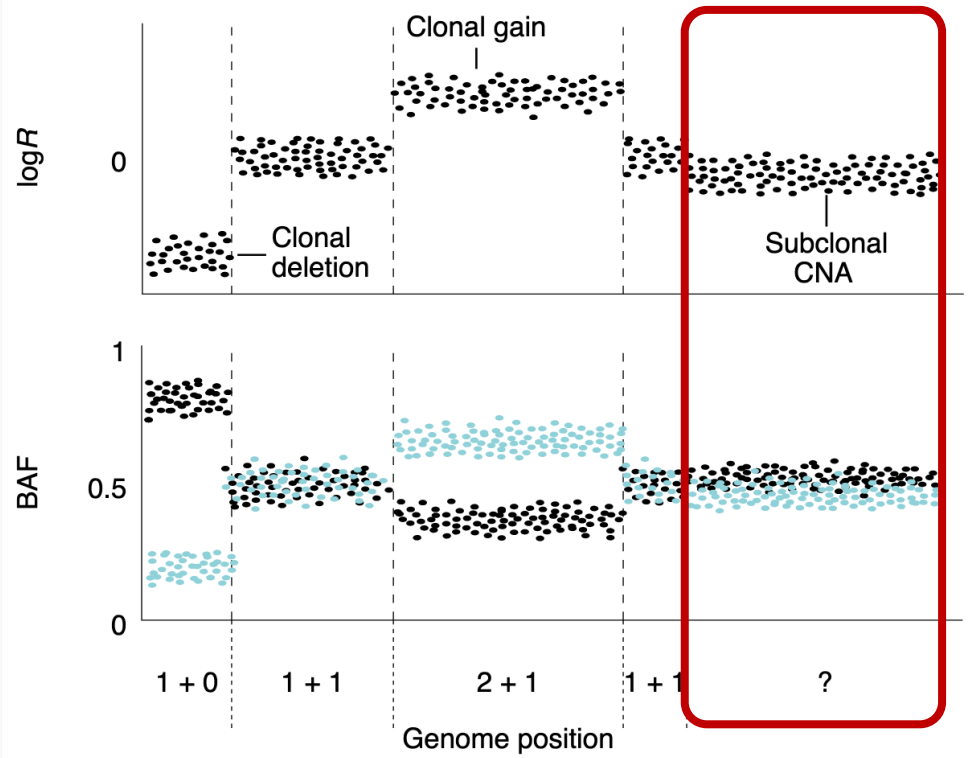
- logR ≈ 0.8 and BAF ≈ 0.45 or $0.55 \Longrightarrow$ subclonal CN events

- $\text{logR} = \log_2\left(\frac{T_i/T}{N_i/N}\right)$
- BAF = % reads from allele B

DNA-sequencing data



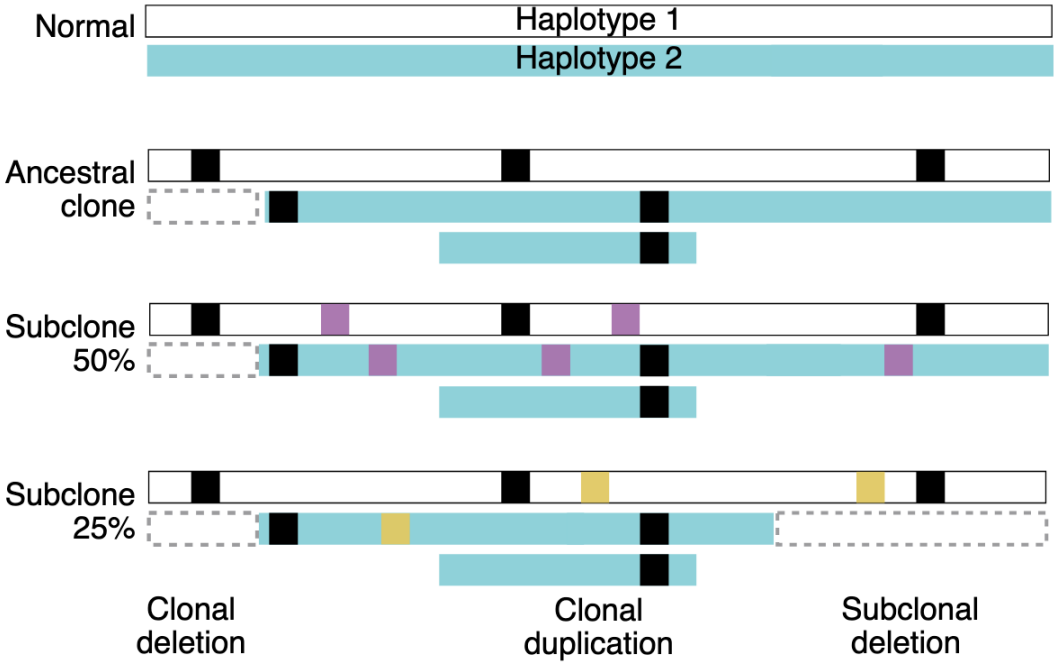
Metrics for CN reconstruction



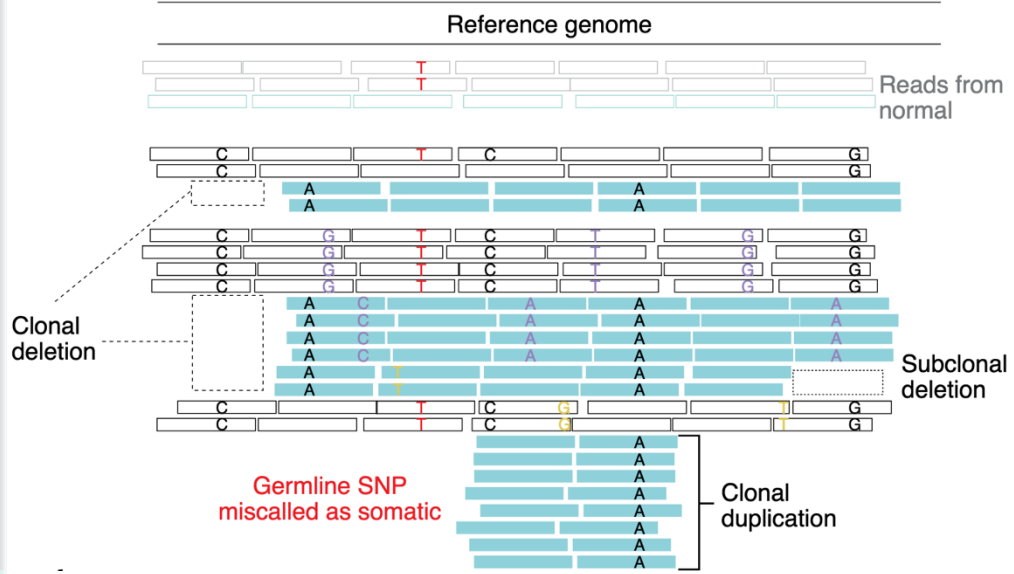
Definitions

This analysis can recover (sub)clonal CN profiles and associated copy number variants (CNVs)

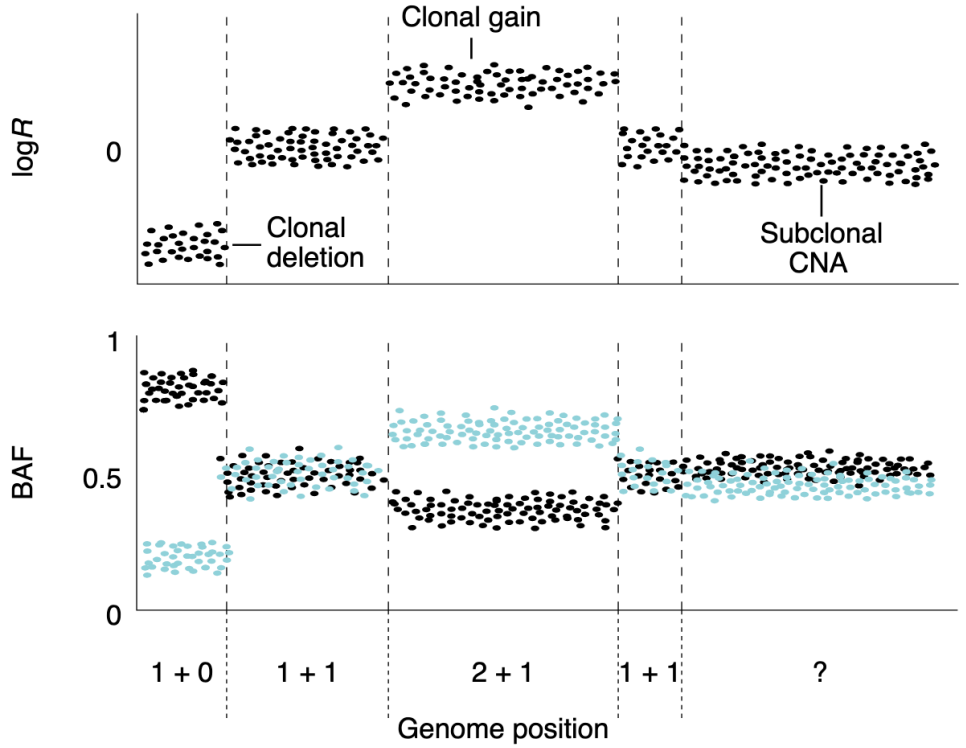
In practice, we also have to account for sample purity $\Longrightarrow$ there are often many equally plausible solutions of purity $\times$ CN profile



DNA-sequencing data



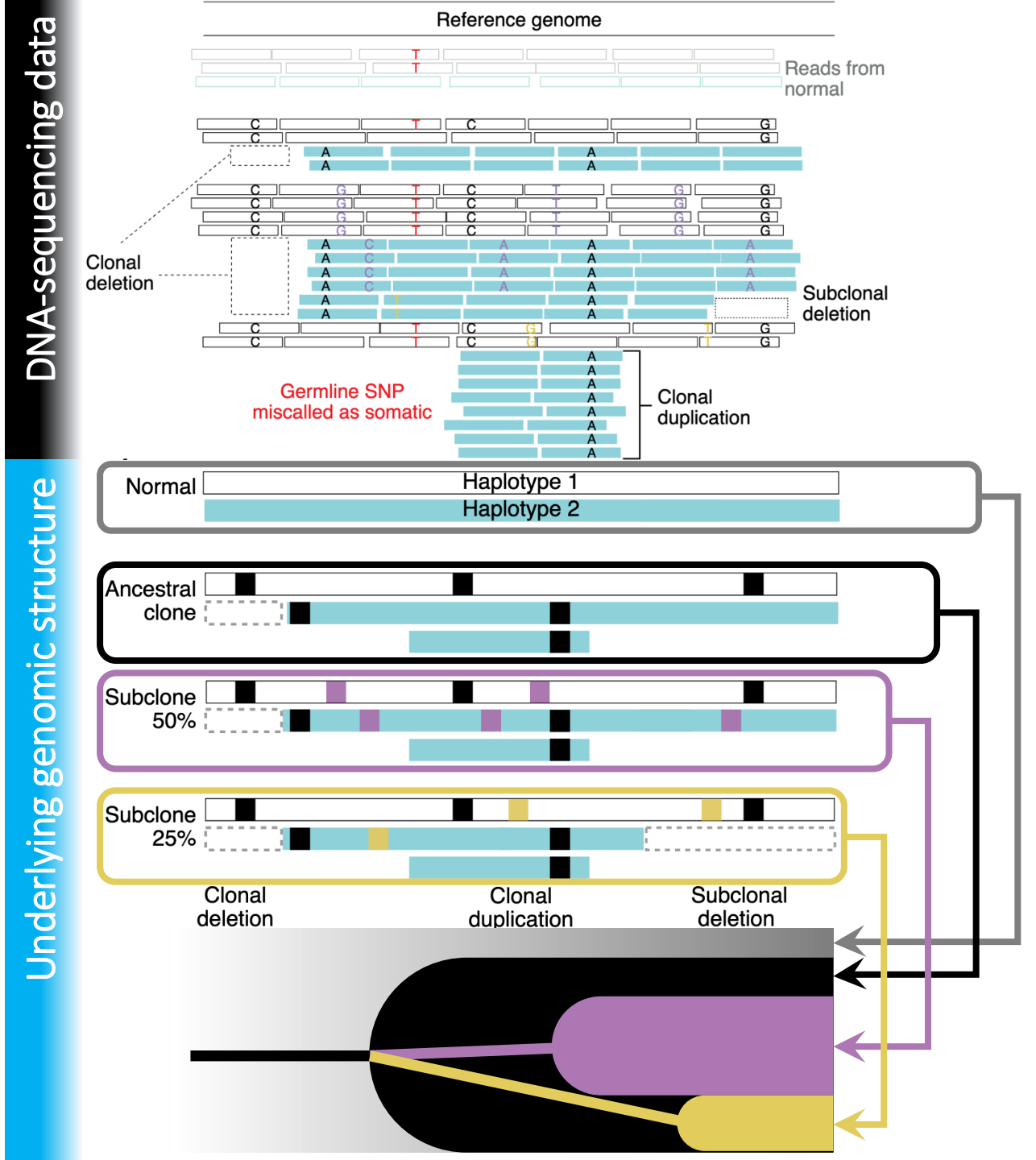
Metrics for CN reconstruction



Definitions

Definitions for mutations:

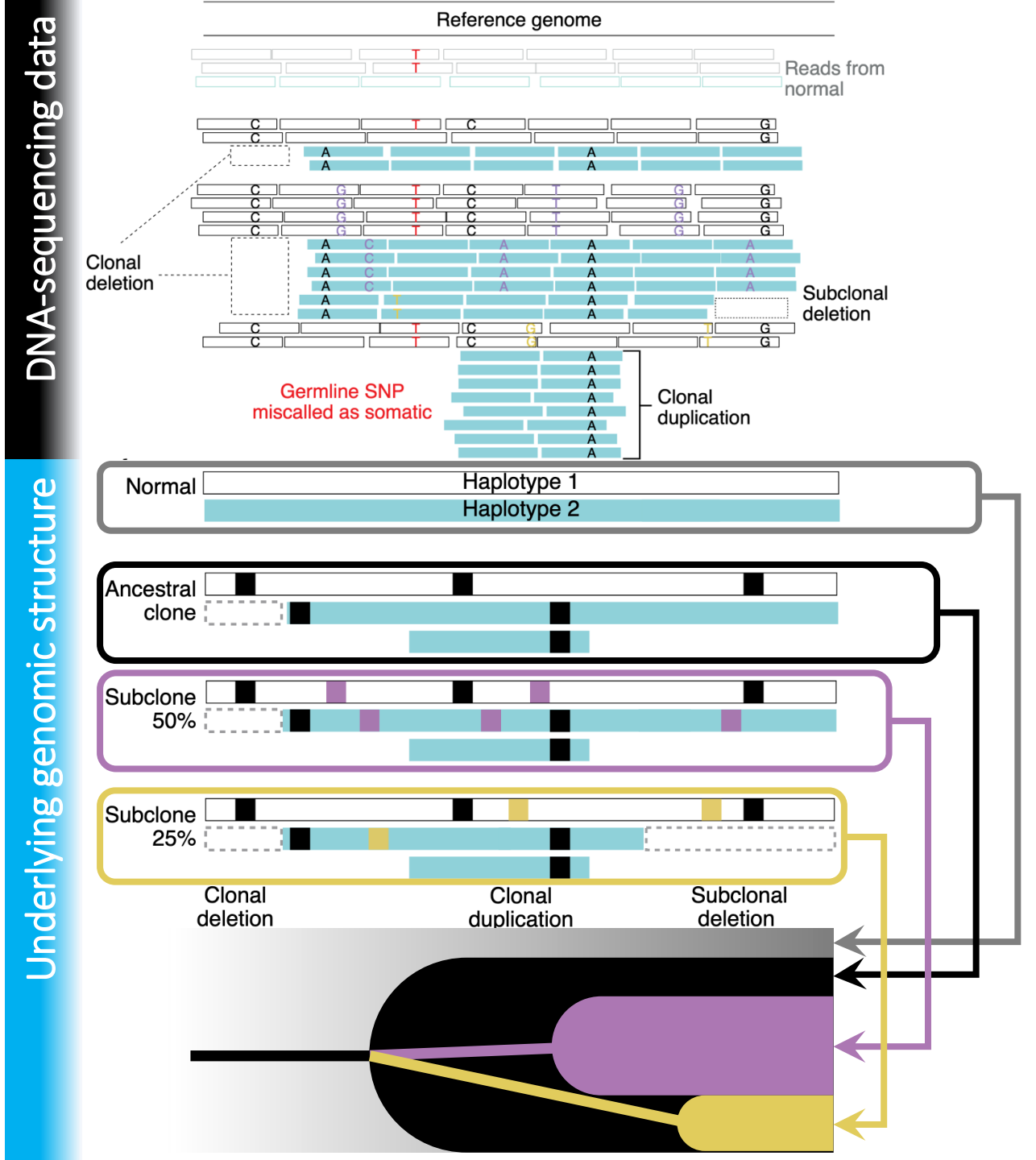
- Total read count: # sequencing reads
- Variant read count: # reads with a mutation
- Reference read count: # reads without a mutation
- Multiplicity: # DNA copies with a mutation
- Variant allele frequency (VAF): % reads carrying a mutation $= \frac{\textbf{Variant read count}}{\textbf{Total read count}}$
- Cancer cell fraction (CCF): % sequenced cells carrying a mutation



Definitions

What happens to a mutation's VAF in each case below, all else being equal?

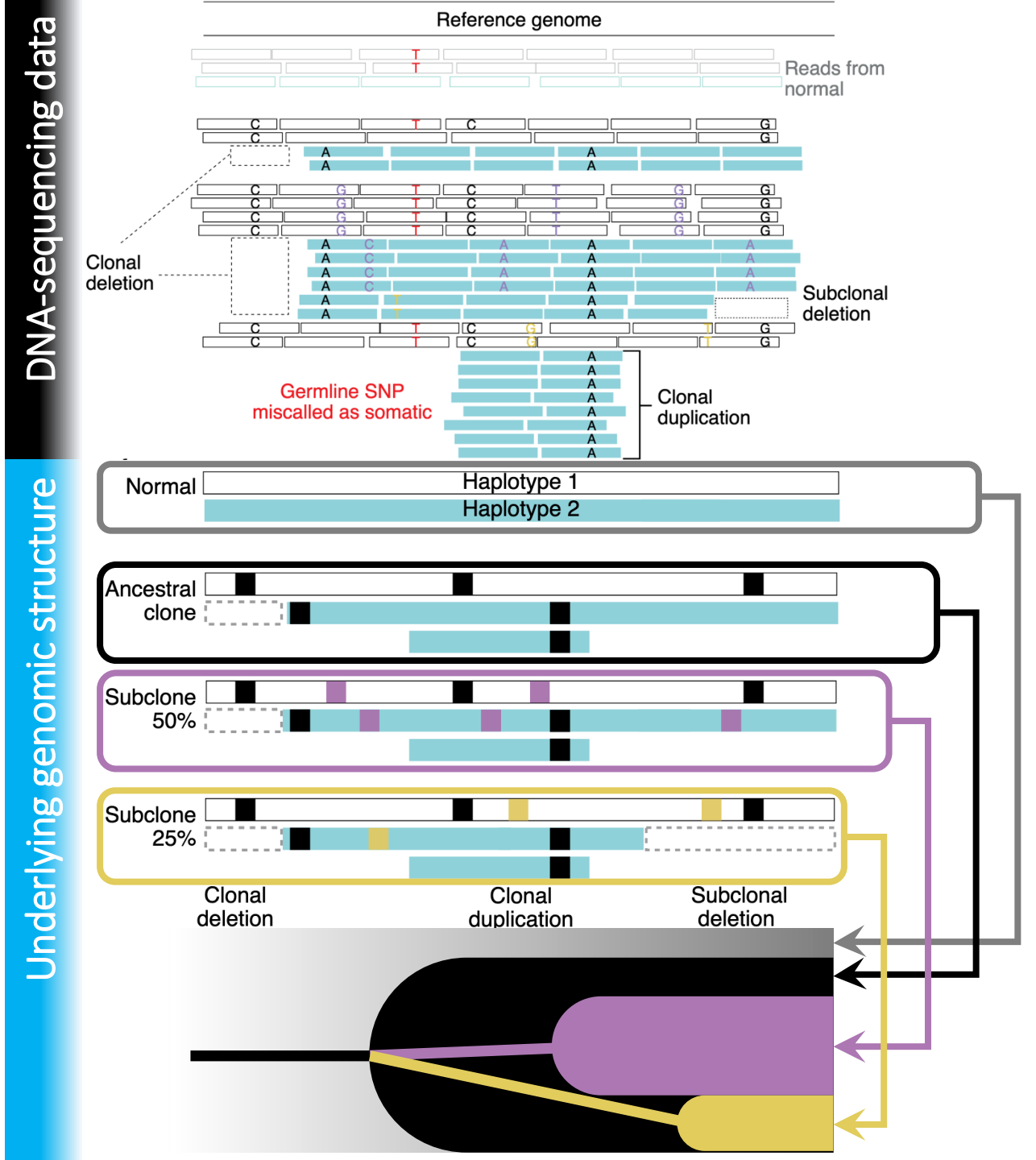
1. Sample purity (ρ) $\nearrow \Longrightarrow$?



Definitions

What happens to a mutation's VAF in each case below, all else being equal?

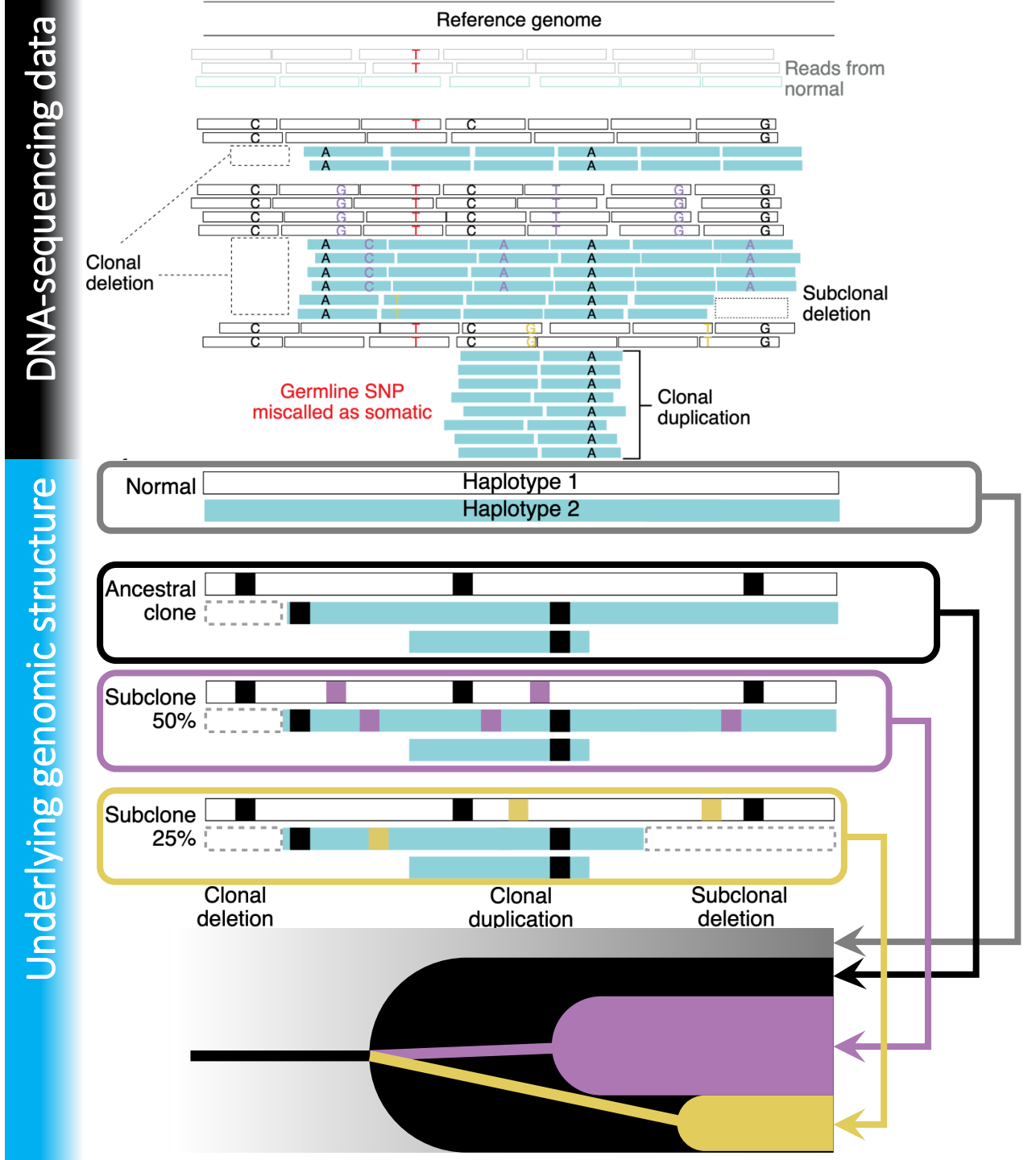
1. Sample purity (ρ) ↗ ⟹ VAF ↗
2. CCF ↗ ⟹ ?



Definitions

What happens to a mutation's VAF in each case below, all else being equal?

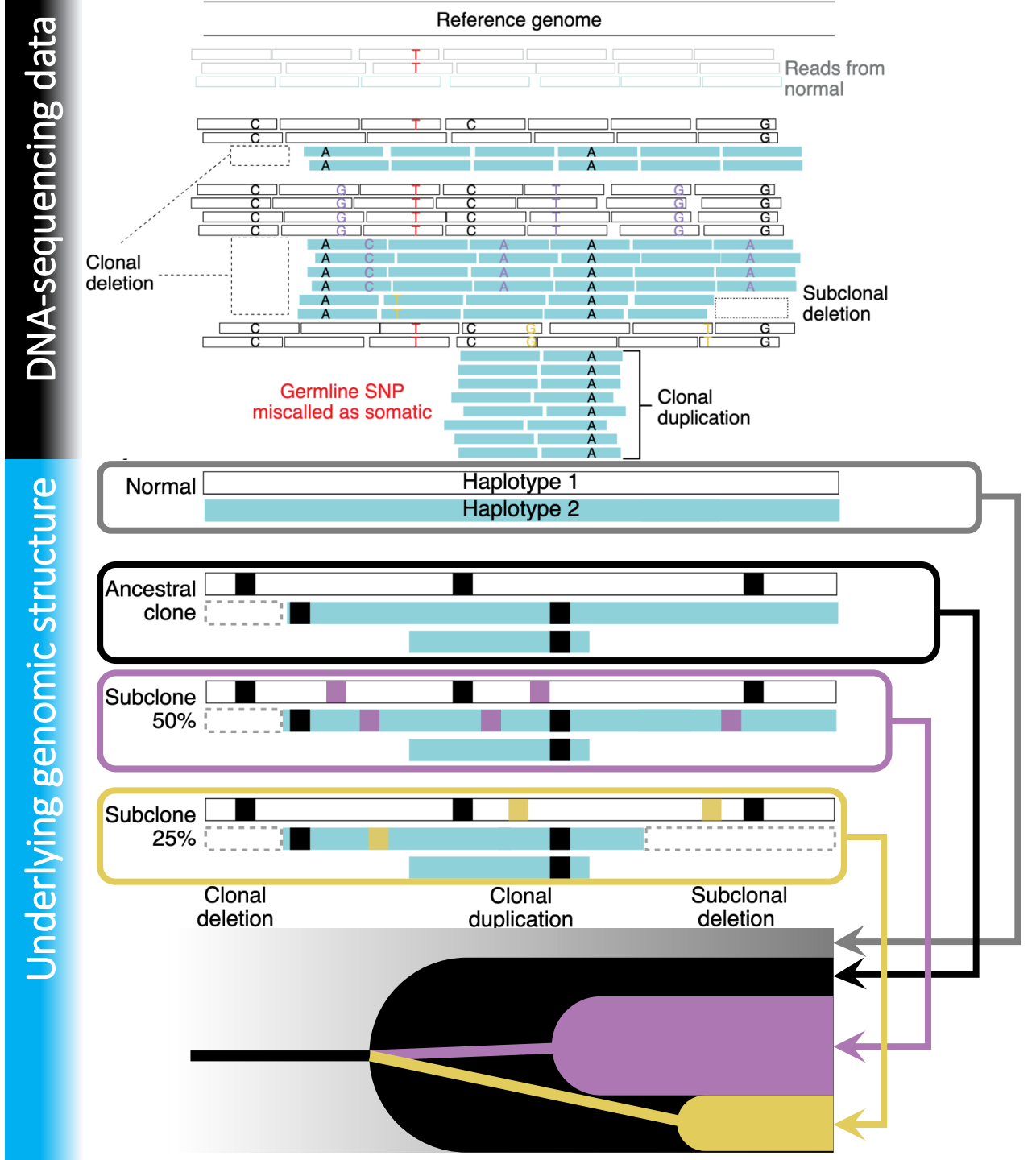
1. Sample purity (ρ) ↗ ⟹ VAF ↗
2. CCF ↗ ⟹ VAF ↗
3. Multiplicity (m) ↗ ⟹ ?



Definitions

What happens to a mutation's VAF in each case below, all else being equal?

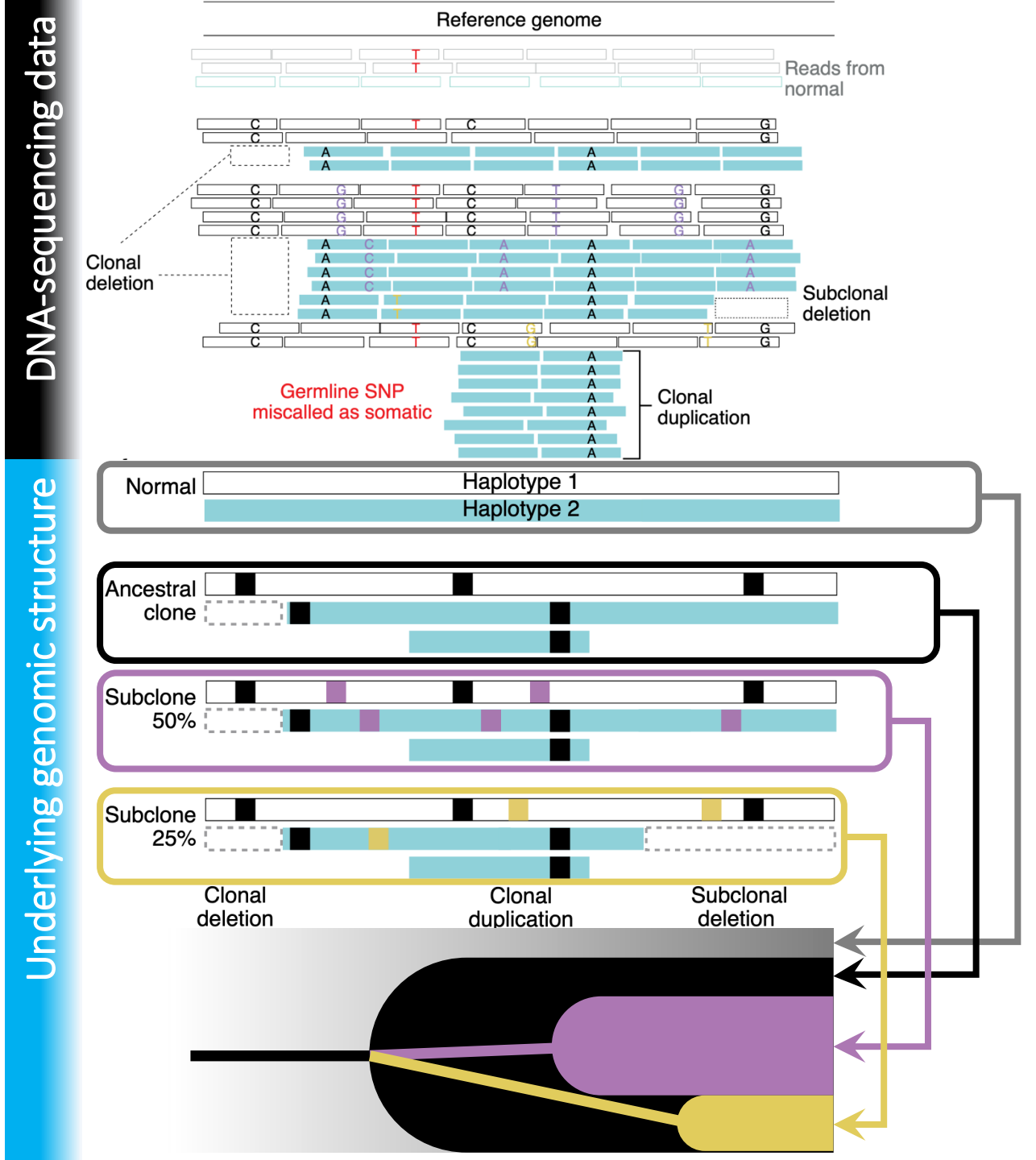
1. Sample purity (ρ) ↗ ⟹ VAF ↗
2. CCF ↗ ⟹ VAF ↗
3. Multiplicity (m) ↗ ⟹ VAF ↗
4. Total CN (N_T) ↗ ⟹ ?



Definitions

What happens to a mutation's VAF in each case below, all else being equal?

1. Sample purity (ρ) ↗ ⟹ VAF ↗
2. CCF ↗ ⟹ VAF ↗
3. Multiplicity (m) ↗ ⟹ VAF ↗
4. Total CN (N_T) ↗ ⟹ VAF ↘



Definitions

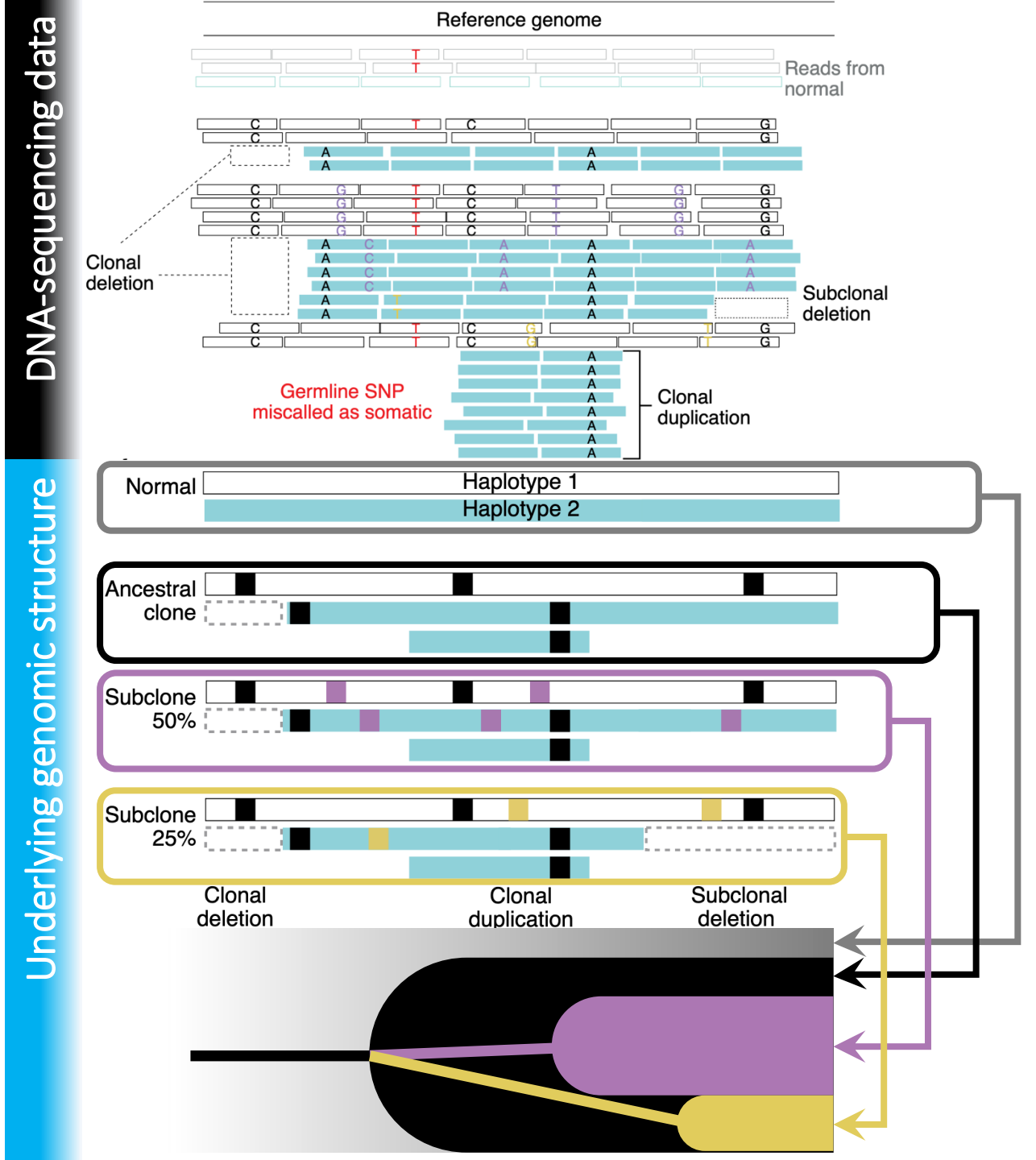
What happens to a mutation's VAF in each case below, all else being equal?

1. Sample purity (ρ) ↗ ⟹ VAF ↗
2. CCF ↗ ⟹ VAF ↗
3. Multiplicity (m) ↗ ⟹ VAF ↗
4. Total CN (N_T) ↗ ⟹ VAF ↘

$$\mathrm{VAF} = \frac{\text{mean \# mutated copies}}{\text{mean \# total copies}} = \frac{\mathrm{CCF}\cdot\rho\cdot m}{N_T\cdot\rho+2\cdot(1-\rho)}$$

or, equivalently:

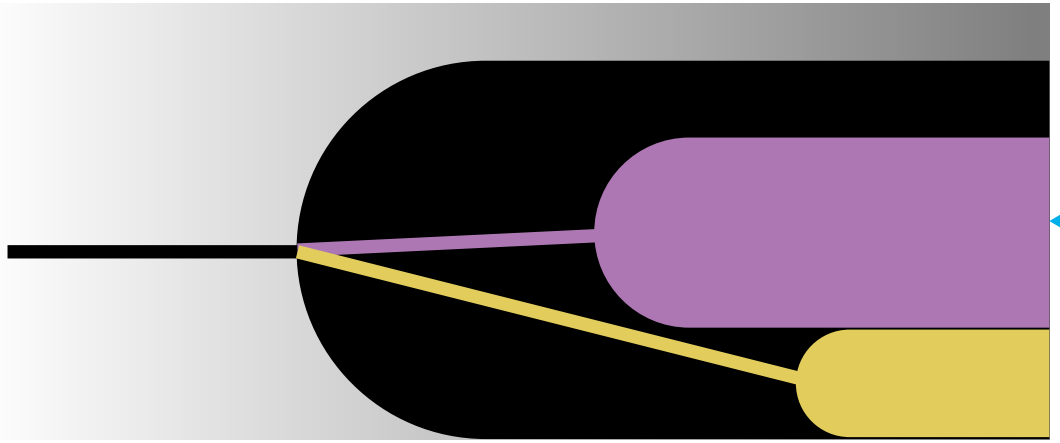
$$\mathrm{CCF} = \frac{\mathrm{VAF}}{\rho\cdot m}\left[N_T\cdot\rho+2\cdot(1-\rho)\right]$$



Definitions

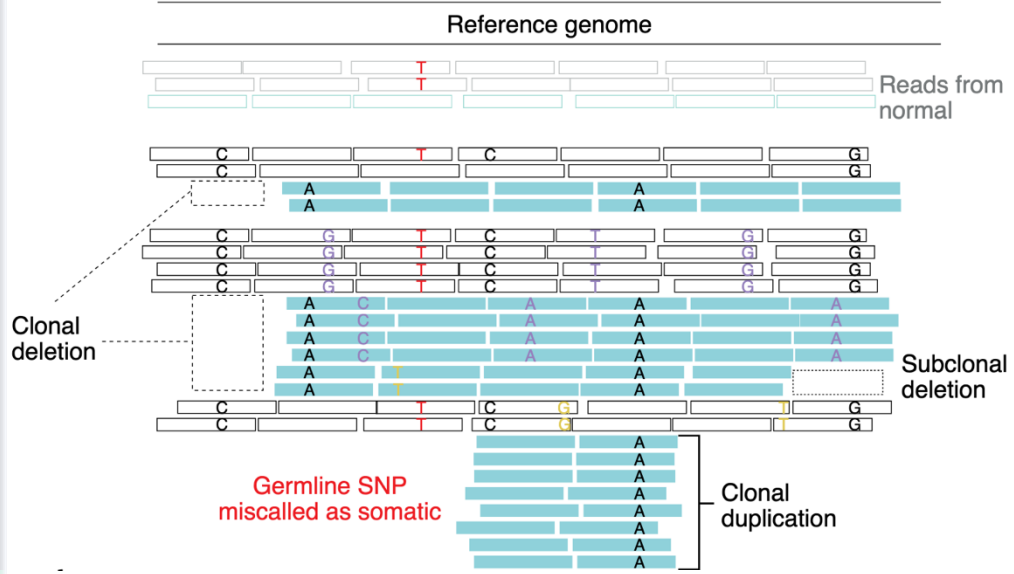
Clustering mutations by CCFs can recover the number of subclones & their relationships

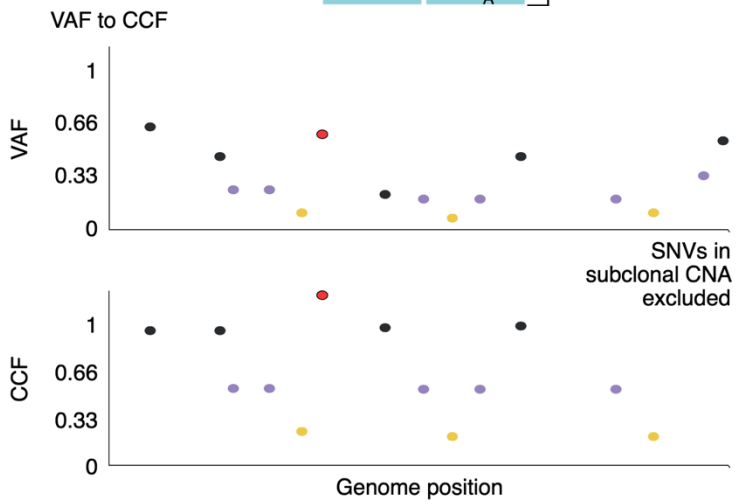
If we filter for mutations with same CCF, multiplicity and total CN, then clustering mutations by VAFs is equivalent to clustering by CCFs

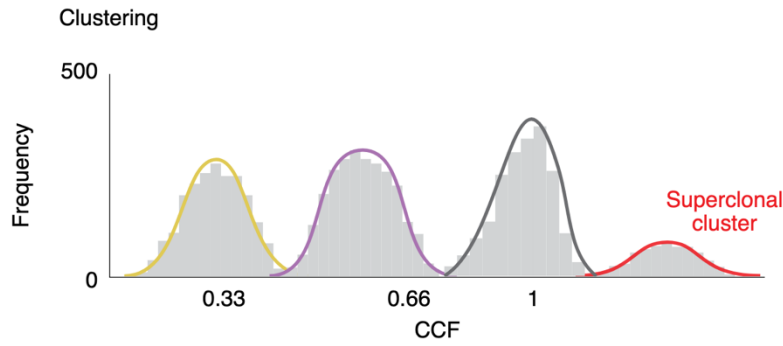


DNA-sequencing data

Inferring mutational CCFs





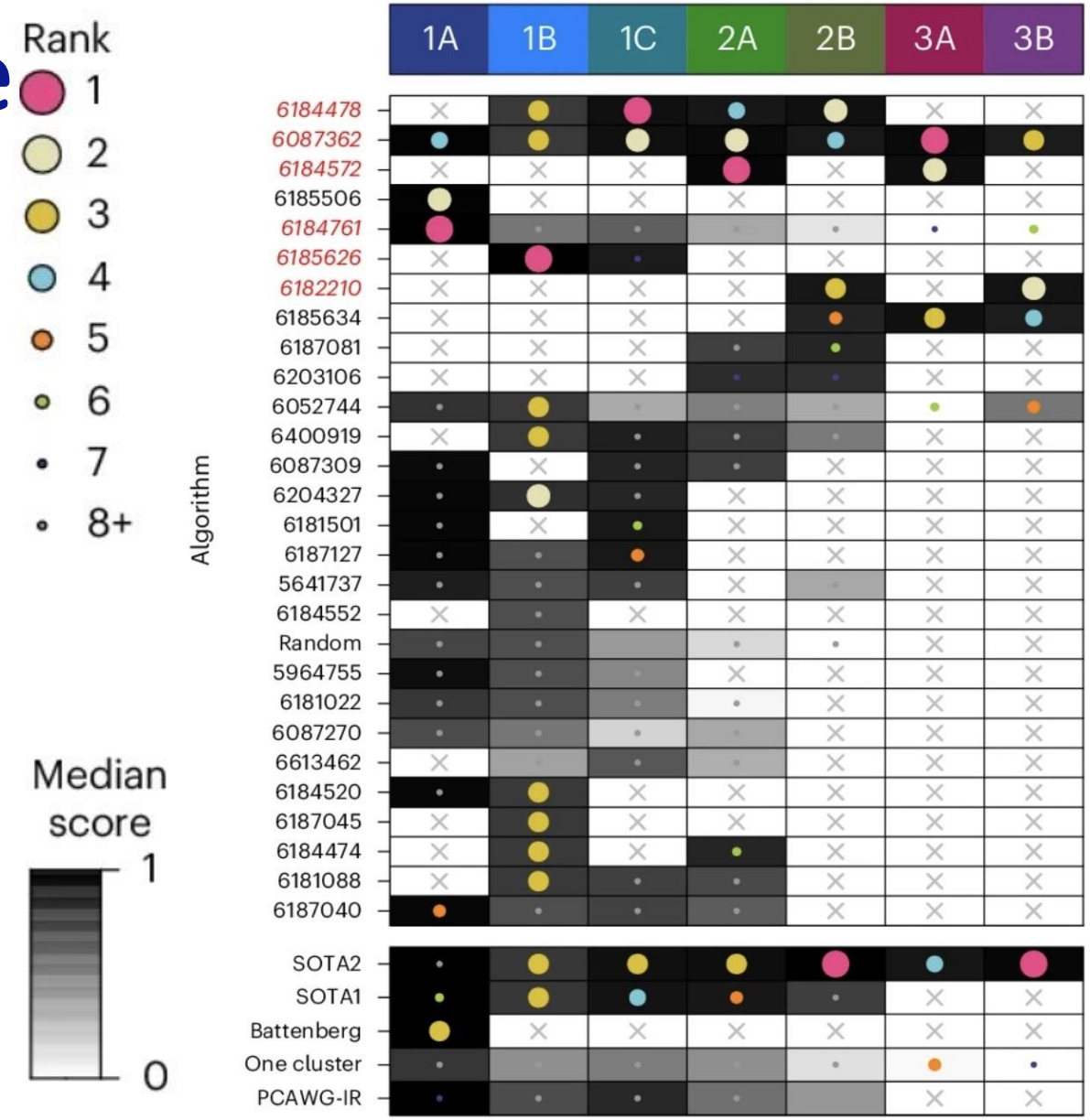


ICGC-TCGA DREAM Challenge

- The ICGC/TCGA DREAM Challenge tested 31 methods with 51 synthetic tumors, simulated to mirror real tumors

- No single method or consensus strategy can reliably infer all aspects of tumor evolution

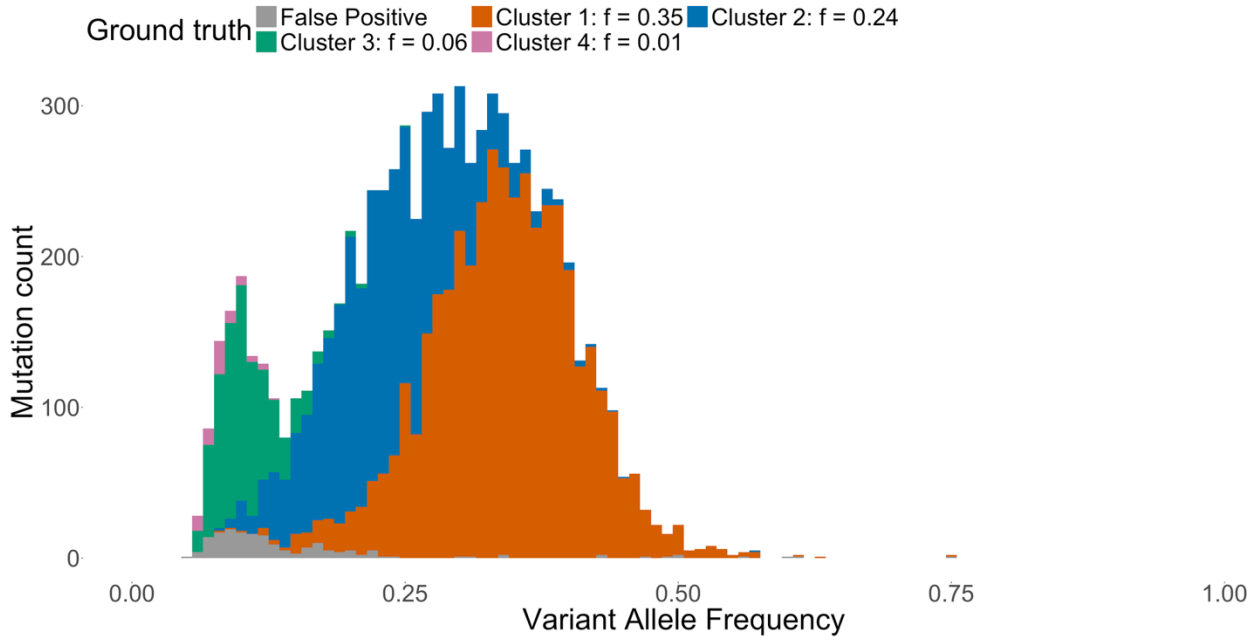
- Worse results for data with lower coverage, diverse copy numbers or poorer read mappability



Salcedo A, Tarabichi M, Espiritu SM, et al. Nature biotechnology. 2020 Jan;38(1):97-107.
Salcedo A, Tarabichi M, Buchanan A, et al. Nature biotechnology. 2024 Jun 11:1-2.

ICGC-TCGA DREAM Challenge

- We use one synthetic ICGC-TCGA DREAM sample (S8) as the example for all methods



Ground truth:

- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: $0.7, 0.48, 0.12, 0.02$
- Subclonal mutation counts: $5981, 3401, 1034, 104$

PhyloWGS

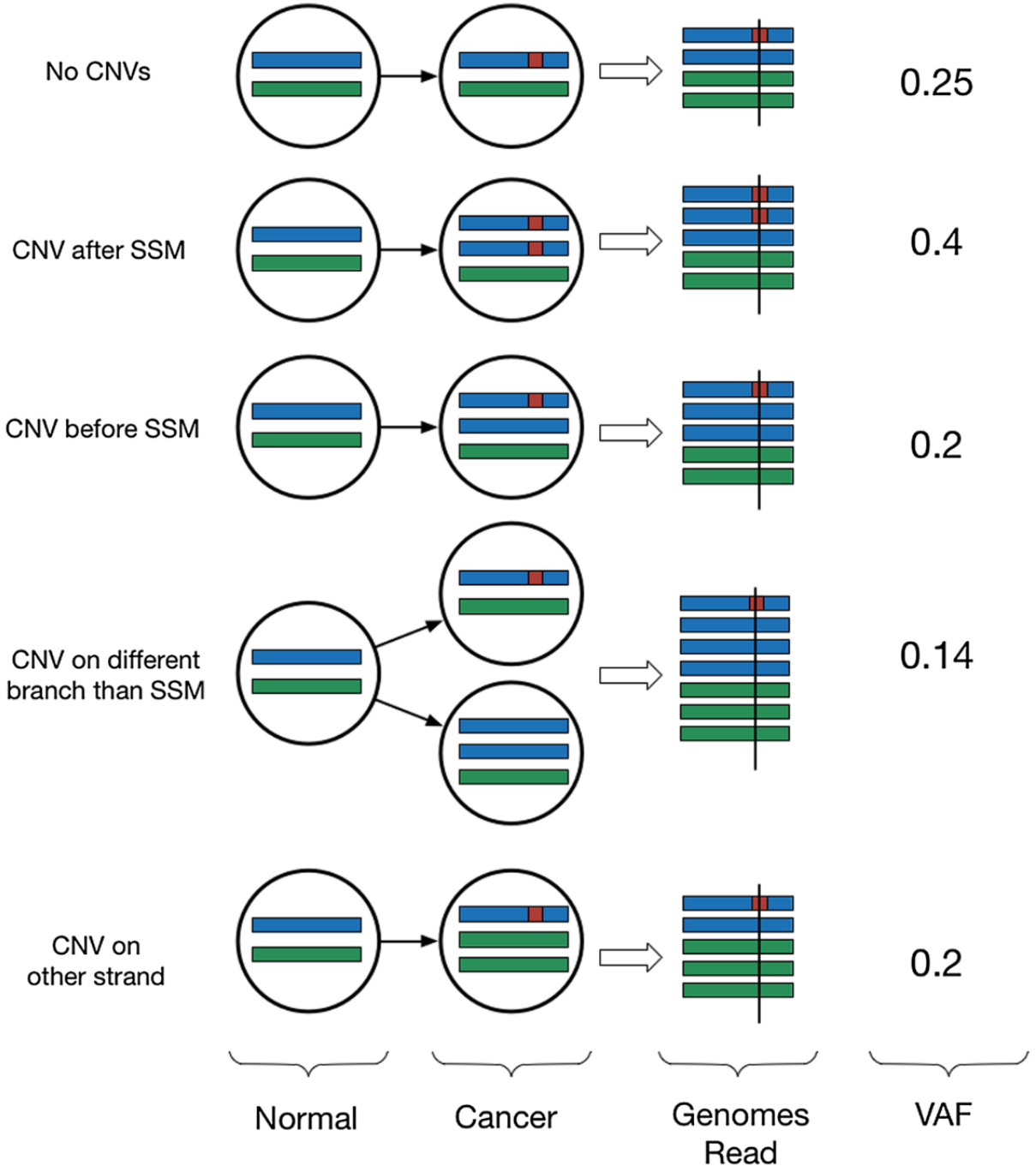
- Assumes (sub)clonal CNVs are known
- Treats each CNV c as a pseudo-mutation with:

  ϕ_c = CCF

  C, C^p, C^m = total, paternal, maternal CNs

  list of actual mutations in the affected region
- For each actual mutation:

  If CCF ϕ_m and relation between the mutation and CNVs affecting the site are known $\Longrightarrow$ expected VAF x_{ssm}



PhyloWGS

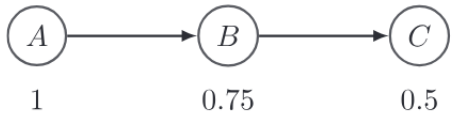
- PhyloWGS seeks the best-fit rooted tree where

  root node = normal cells

  K other nodes = subclones

  clone k: CCF = ϕ_k, with assigned mutations and CNVs

- Constraint: $\phi_k \geq \sum_l \phi_l$ across children nodes
- Likelihood for a given tree: from expected reference/variant read counts across all nodes (accounting for CNVs), assuming **variant read count ~ Binomial(total read count, expected VAF)**



A: –
B: SSM1; SSM2; SSM3; SSM4
C: SSM5; SSM6; SSM7; SSM8; CNV1

(A)



A: –
B: SSM1; SSM2; SSM3; SSM4
C: SSM5; SSM6; SSM7; SSM8; CNV1

(B)



A: –
B: SSM1; SSM2; SSM3
C: SSM4; SSM5; SSM6; SSM7; SSM8

(C)



A: –
C: CNV1

(D)

PhyloWGS

Markov chain Monte Carlo (MCMC) to find best-fitted tree. For each MCMC chain:

- Iteration 1: Sample a tree from the prior distribution
- Iteration $2, 3, \ldots$:
  - Modify previous tree (add, remove, or reorder nodes) and reassign mutations to the nodes
  - Resample node CCFs $\boldsymbol{\phi_k}$
  - Compute likelihood of new tree → move to new tree only if likelihood is higher
  - Go to next iteration

Default: PhyloWGS runs **4** MCMC chains, each with length **2500**, and discards the first **1000** iterations ("burn-in") in each chain before tallying the posterior across remaining iterations

Interactive session 1

PhyloWGS requires two input files, created with parsing functions **parse_cnvs.py** and **create_phylowgs_inputs.py** (input: VCF file + CNV calls from Battenberg/TITAN).

data/PhyloWGS_ssm.txt lists a mutation in each row, with following columns:

- **id**: mutation ID
- **gene**: genomic location (chromosome & nucleotide index)
- **a**: reference read count
- **d**: total read count
- **mu_r**: probability that a read from the reference sample matches the reference base
- **mu_v**: probability that a read from the tumor sample matches the reference base

Interactive session 1

PhyloWGS requires two input files, created with parsing functions **parse_cnvs.py** and **create_phylowgs_inputs.py** (input: VCF file + CNV calls from Battenberg/TITAN).

data/PhyloWGS_cnv.txt lists a CNV group in each row, with following columns:

- **cnv**: CNV group ID
- **a**: sum of reference read counts across all mutations in listed segments
- **d**: sum of total read counts across all mutations in listed segments
- **ssms**: list of all mutations in listed segments in format **<mutation id>, <major allele multiplicity>,<minor allele multiplicity>**, separated by ";"
- **physical_cnvs**: list of CNVs in the group in format **<chrom>,<start>,<end>, <major CN>,<minor CN>,<CCF>**, separated by ";"

Interactive session 1

1. Run a smoke test of PhyloWGS with

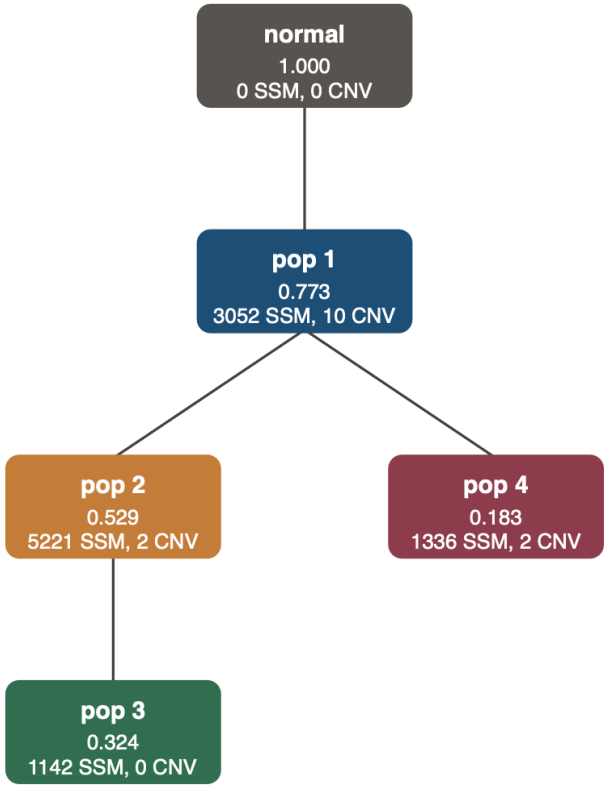
   docker compose run --rm workshop bash src/run_phylowgs.sh

2. Examine the code in **src/run_phylowgs.sh**
3. Examine the results from a PhyloWGS run with default parameters in **S8-noXY.html**

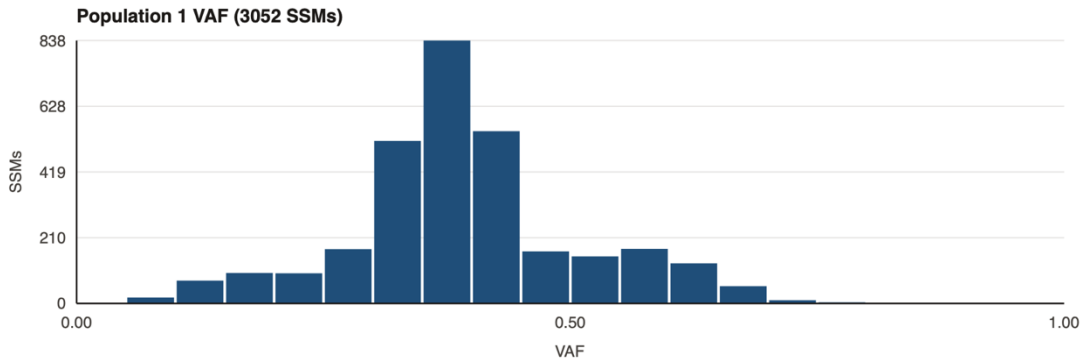
Interactive session 1

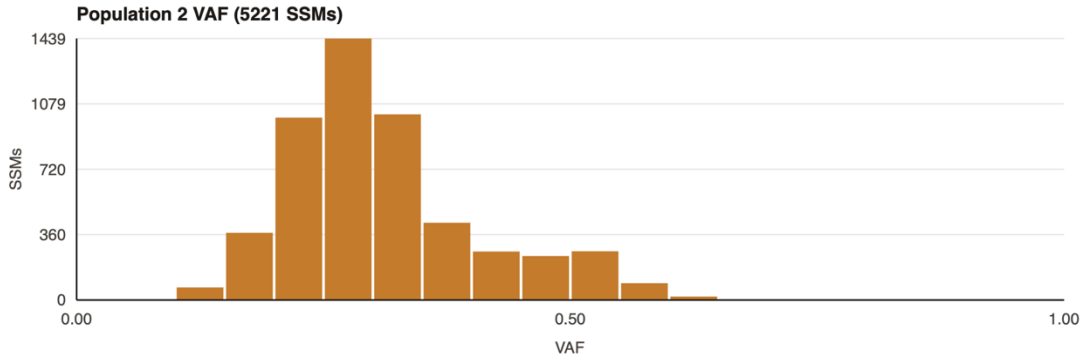
Ground truth:

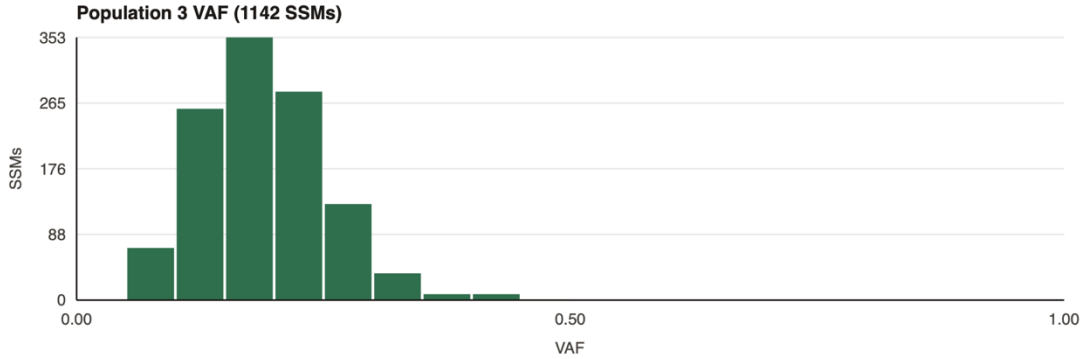
- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: $0.7, 0.48, 0.12, 0.02$
- Subclonal mutation counts: $5981, 3401, 1034, 104$

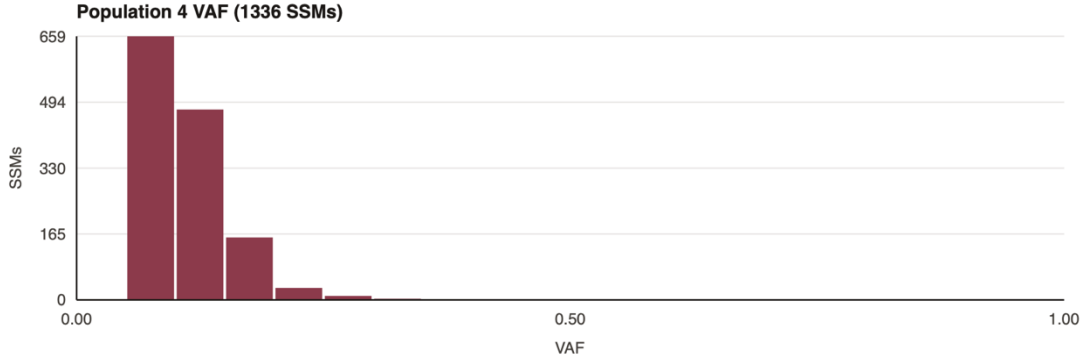


What do we think of the result?









Interactive session 1

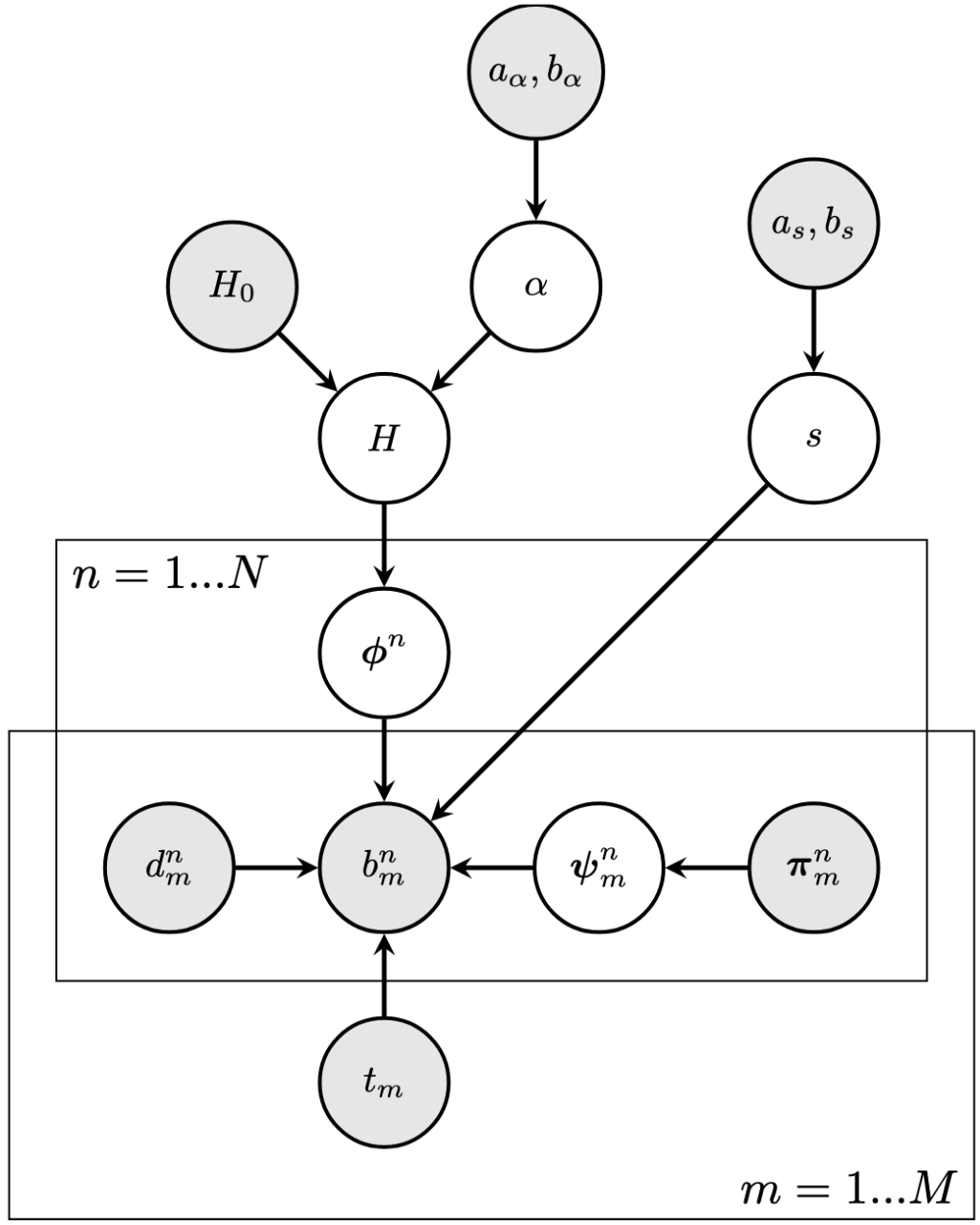
	Incorporate CNVs	Multi-samples	Efficient
PhyloWGS			

PyClone-VI

PyClone employs a hierarchical Bayes statistical model.

Shaded nodes: observed or user-defined

White nodes: latent, to be inferred

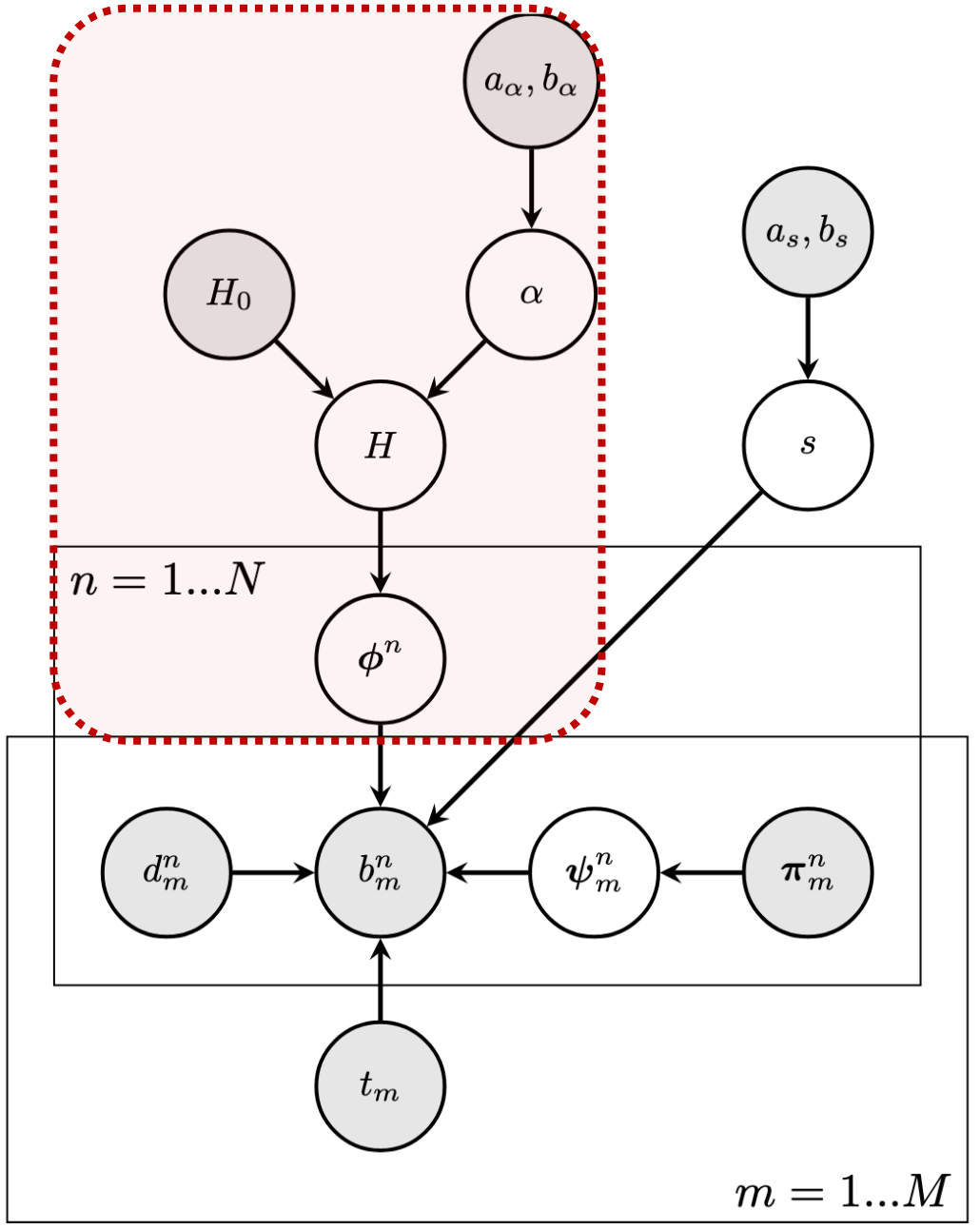


PyClone-VI

PyClone employs a hierarchical Bayes statistical model.

For mutation $n = 1, \ldots, N$ in sample $m = 1, \ldots, M$:

- $H_0 \sim \mathrm{U}([0,1]^M)$ = base cluster CCF distribution
- $\alpha \sim \mathrm{Gamma}(a_\alpha, b_\alpha)$ = Dirichlet Process (DP) concentration parameter
- $H \sim \mathrm{DP}(\alpha, H_0)$ = discrete distribution for cluster CCFs
- $\phi^n = (\phi_1^n, \ldots, \phi_M^n)$ = mutation n's CCF in each sample

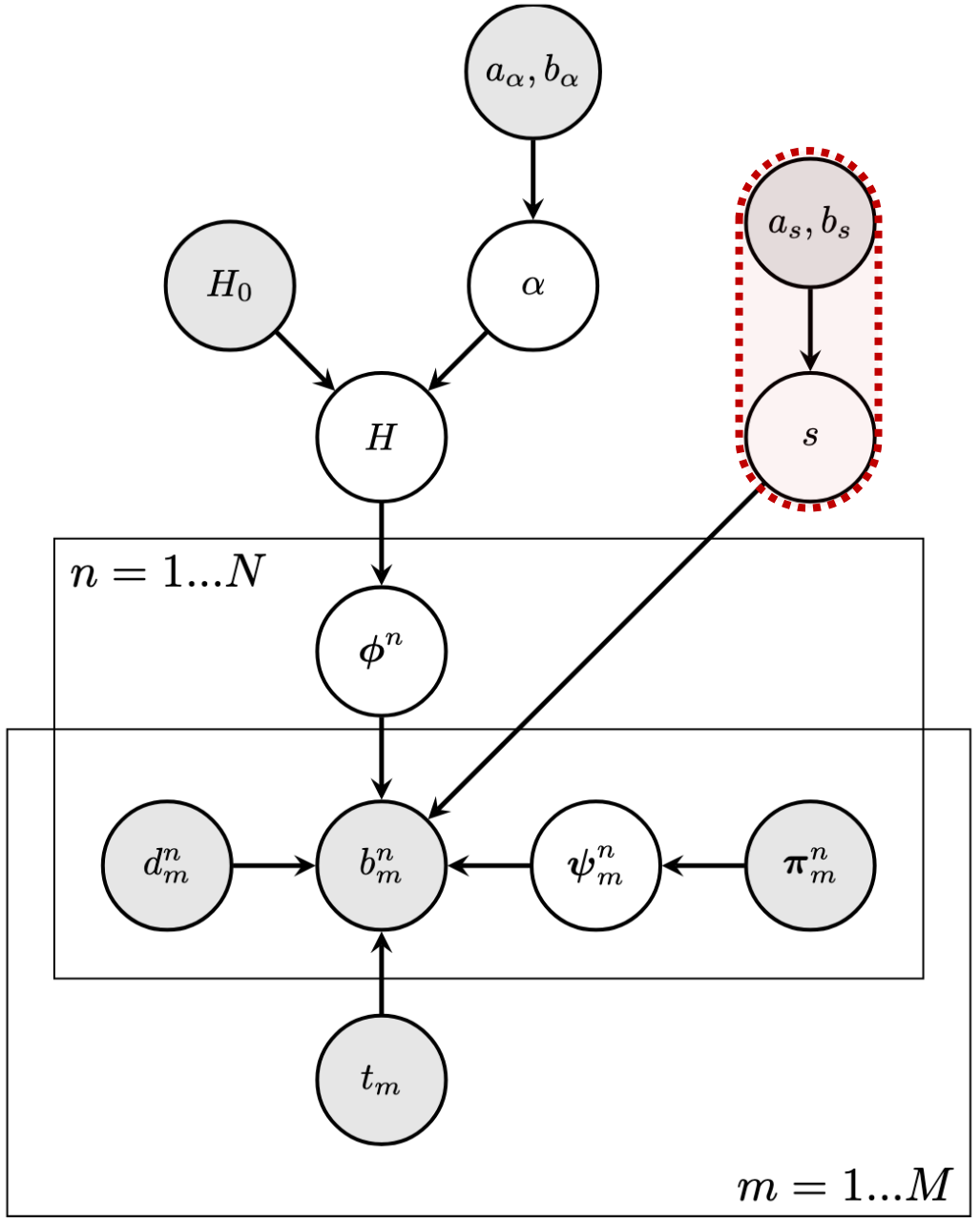


PyClone-VI

PyClone employs a hierarchical Bayes statistical model.

For mutation $\boldsymbol{n} = 1, \ldots, \boldsymbol{N}$ in sample $\boldsymbol{m} = 1, \ldots, \boldsymbol{M}$:

- $\boldsymbol{s} \sim \text{Gamma}(\boldsymbol{a_s}, \boldsymbol{b_s})$ = noise in read count data

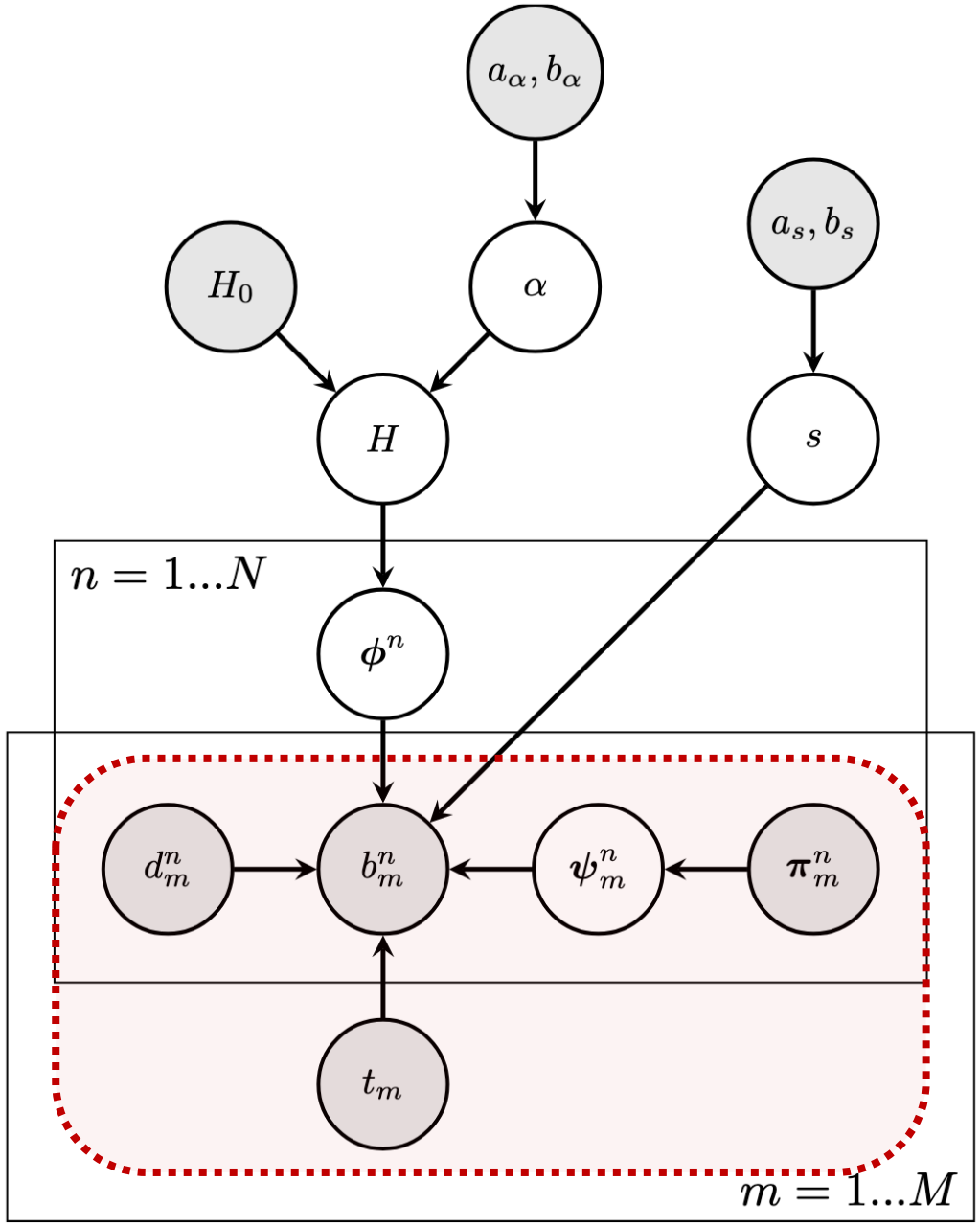


PyClone-VI

PyClone employs a hierarchical Bayes statistical model.

For mutation $n = 1, \ldots, N$ in sample $m = 1, \ldots, M$:

- t_m = purity in sample m
- $\psi_m^n \sim \text{Categorical}(\pi_m^n)$ = genotypes of the normal, reference and mutated cells at mutation m's location (e.g., (AA, AA, AB))
- d_m^n = total read count
- $b_m^n \sim \text{BetaBinomial}(d_m^n, \xi(\psi_m^n, \phi^n, t_m), s)$ = variant read count



PyClone-VI

Original PyClone employs Markov chain Monte Carlo (MCMC) to find best-fitted tree.

PyClone-VI replaces MCMC with Variational Inferences (VI) to speed up $100\times$, with following simplifications:

- Requires maximum cluster count (unneeded clusters are emptied out later), instead of adaptive cluster count
- Mutation CCFs are restricted to a grid, instead of being continuous → likelihood at each grid point can be computed once before hand
- Assigns mutations to most probable cluster (cost $= \mathcal{O}(\boldsymbol{N})$), instead of seeking the co-clustering matrix between each pair of mutations (cost $= \mathcal{O}(\boldsymbol{N^2})$)
- VI seeks the optimal solution, instead of the posterior distribution

Interactive session 2

PyClone-VI requires one input file: **data/PyCloneVI_mutation_table.tsv**, which lists a mutation in each row, with following columns:

- **mutation_id**: mutation ID
- **sample_id**: sample ID
- **ref_counts**: reference read count
- **alt_counts**: variant read count
- **major_cn**: major CN
- **minor_cn**: minor CN
- **normal_cn**: total CN in the normal tissue at the genomic site
- **tumour_content**: sample purity

Interactive session 2

1. Run PyClone-VI with

   docker compose run --rm workshop bash src/run_pyclonevi.sh

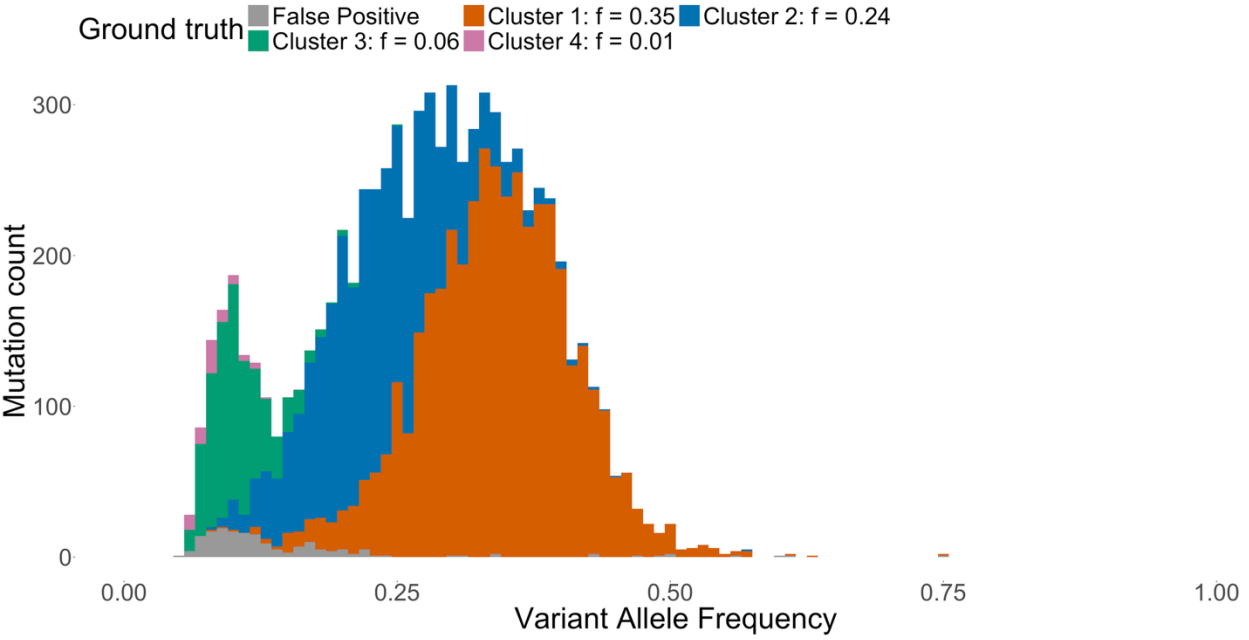
2. Examine the code in **src/run_pyclonevi.sh**
3. Examine the results from PyClone-VI in **results/pyclonevi/S8-noXY_sfs.pdf** and **results/pyclonevi/clone.tsv**

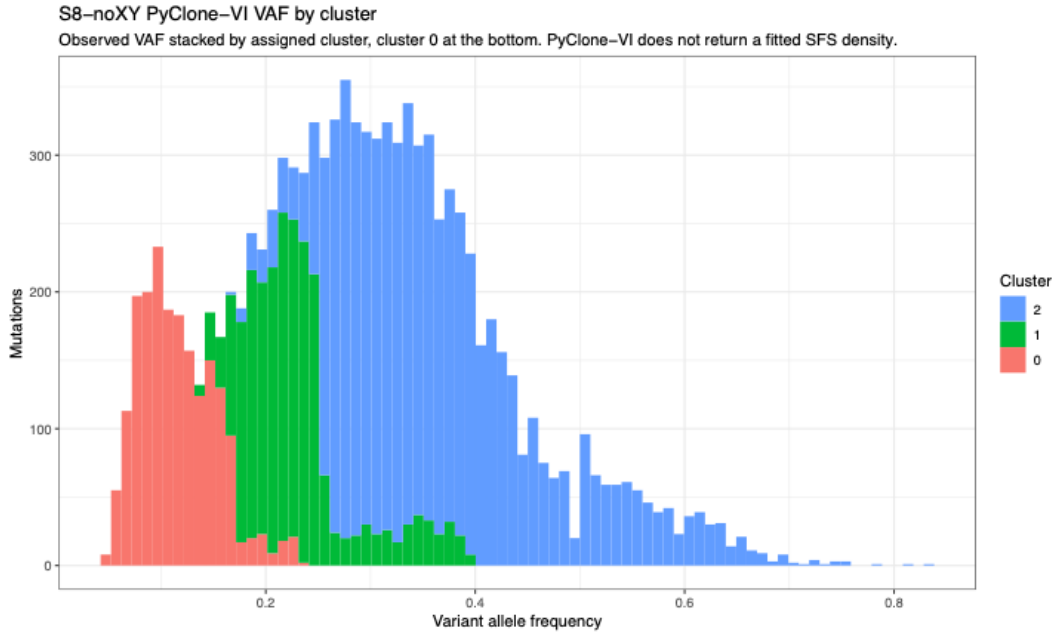
Interactive session 2

Ground truth:

- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: $0.7, 0.48, 0.12, 0.02$
- Subclonal mutation counts: $5981, 3401, 1034, 104$

n_clones	clone	ccf
3	0	0.20752324
3	1	0.68376686
3	2	0.46277341





What do we think of the result?

Interactive session 2

	Incorporate CNVs	Multi-samples	Efficient
PhyloWGS			
PyClone-VI	Multiplicity = 1		

SciClone

- Only analyzes mutations with the same CN background (usually diploid)
- Clusters the VAFs directly (equivalent to CCFs in this case)
- Assumes that each cluster $\boldsymbol{k}$ makes up a portion $\boldsymbol{\pi_k^m}$ of sample $\boldsymbol{m}$, and follows Beta distribution with two parameters $\boldsymbol{u_k}$ and $\boldsymbol{v_k}$, which determine its VAF mean and variance
- Infers cluster count $\boldsymbol{K}$ and $(\boldsymbol{\pi_k^m}, \boldsymbol{u_k}, \boldsymbol{v_k})$ with variational Bayes

Interactive session 3

SciClone requires three input files.

data/sciClone_mutation_table.tsv lists a mutation in each row, with following columns:

- **chr**: chromosome
- **pos**: position in chromosome
- **ref_reads**: reference read count
- **var_reads**: variant read count
- **vaf**: VAF ($= 100 \times$ **var_reads**$/($**var_reads** $+$ **ref_reads**$)$)

Interactive session 3

SciClone requires three input files.

data/sciClone_cn.tsv lists a CN segment in each row, with following columns:

- **chr**: chromosome
- **start**: start position in chromosome
- **stop**: end position in chromosome
- **segment_mean**: total CN in the segment

Interactive session 3

SciClone requires three input files.

data/sciClone_exclude_loh.bed lists a CN segment in each row to exclude in the analysis (e.g., where minor CN $= 0$), with following columns:

- **column 1**: chromosome
- **column 2**: start position in chromosome
- **column 3**: end position in chromosome

Interactive session 3

1. Run SciClone with

docker compose run --rm workshop Rscript src/run_sciclone.R

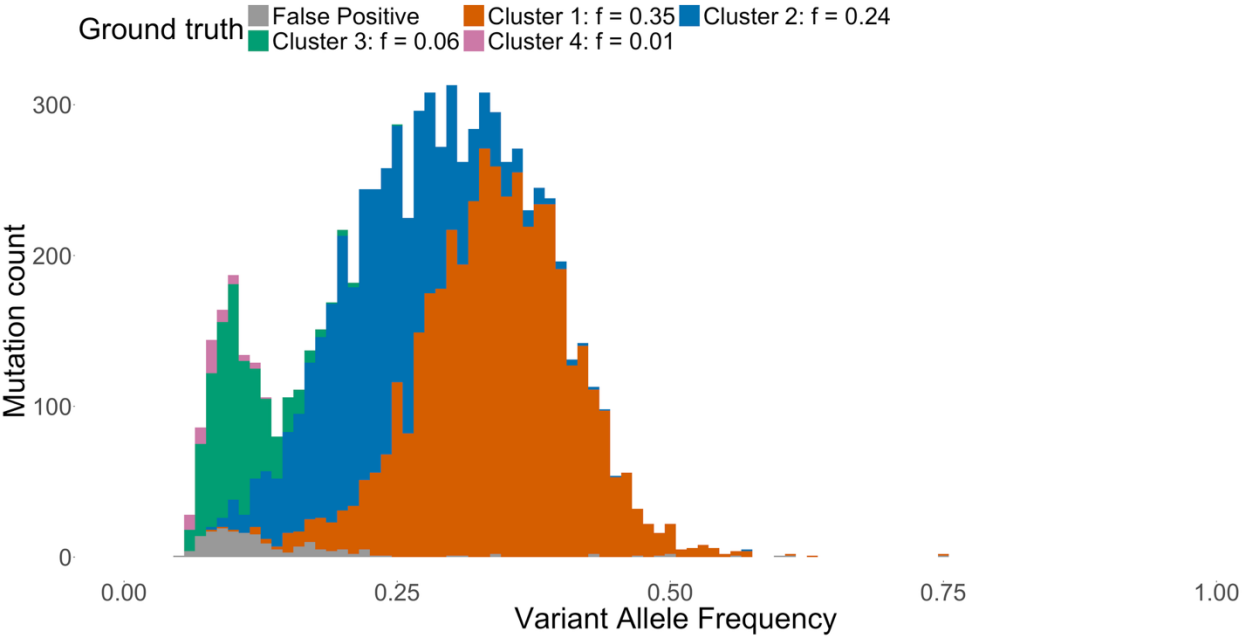
2. Examine the code in **src/run_sciclone.R**
3. Examine the results from SciClone in **results/sciclone/S8-noXY_clusters_1d.pdf**, **results/sciclone/S8-noXY_sfs.pdf** and **results/sciclone/clone.tsv**

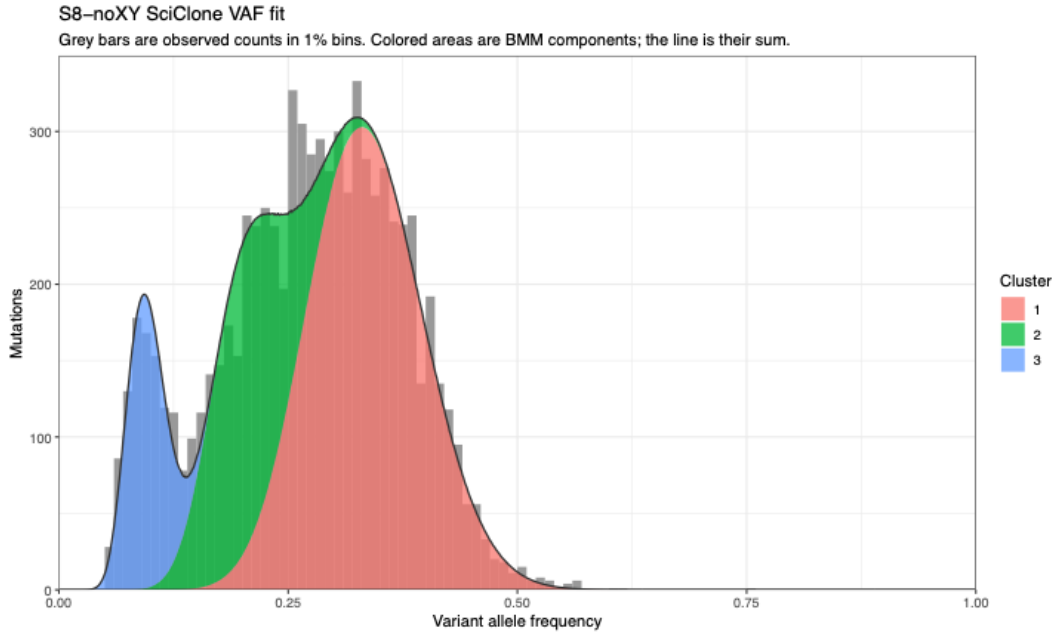
Interactive session 3

Ground truth:

- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: $0.7, 0.48, 0.12, 0.02$
- Subclonal mutation counts: $5981, 3401, 1034, 104$

purity	n_clones	clone	ccf
0.67243214	3	1	0.67243214
0.67243214	3	2	0.42360324
0.67243214	3	3	0.19431377





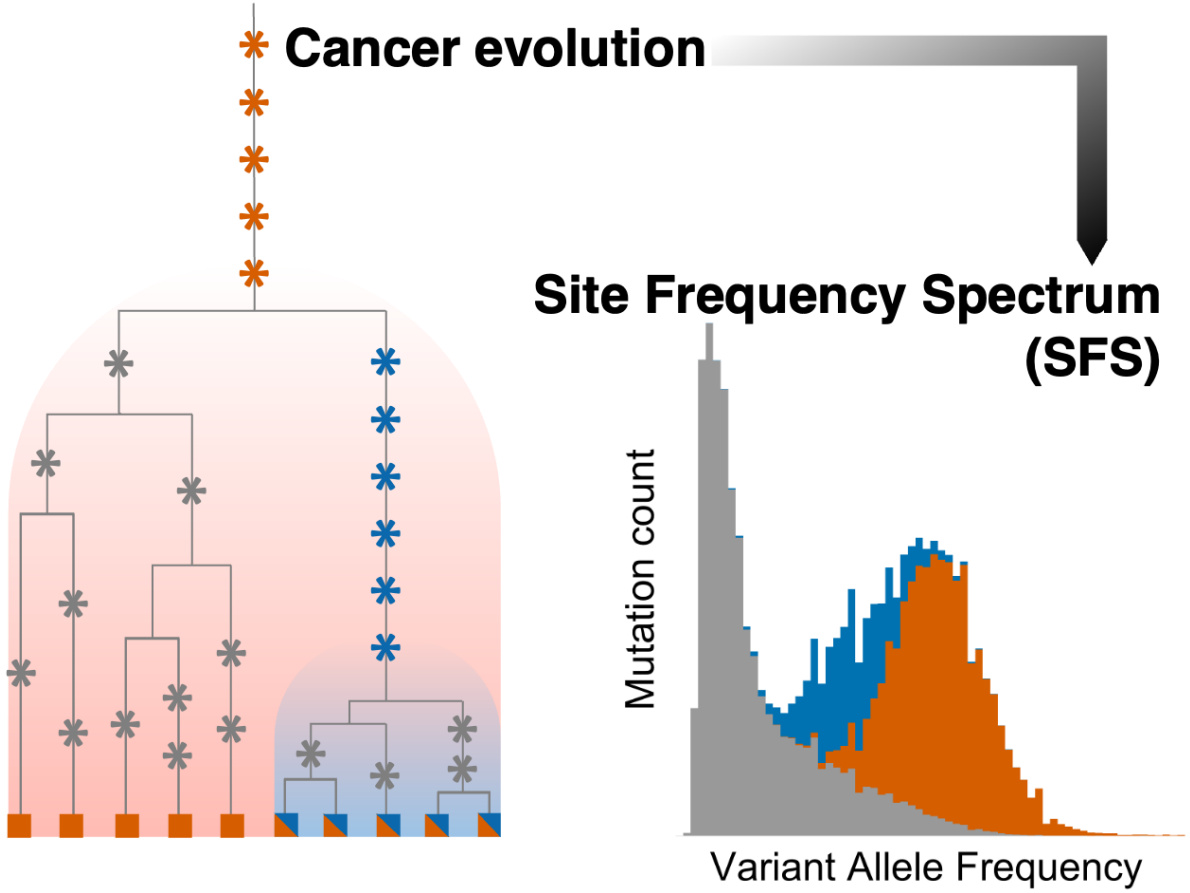
What do we think of the result?

Interactive session 3

	Incorporate CNVs	Multi-samples	Efficient
PhyloWGS			
PyClone-VI	Multiplicity = 1		
SciClone			

Neutral tail in the SFS

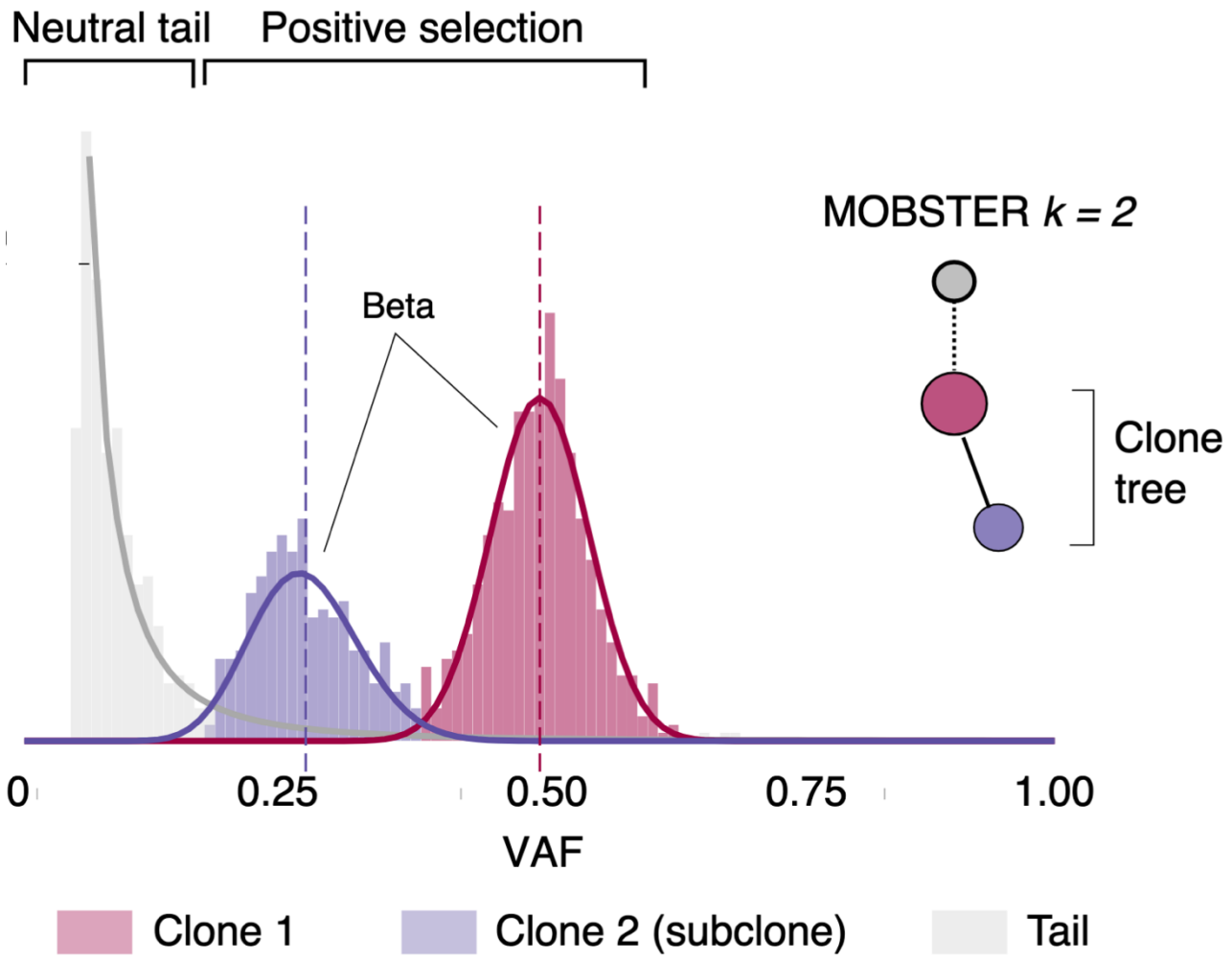
- All methods thus far only consider mutation clusters from each subclone's MRCA
- Population genetics theory predicts a neutral tail in the SFS, in addition to mutation clusters
- The tail contains mutations from recent mitoses
- From population genetics: expected SFS tail follows $\text{mutation count} \propto \text{VAF}^{-\alpha}$
  - $\alpha \approx 1$ in stable-sized tumors
  - $\alpha \approx 2$ in exponentially expanding tumors



- Not incorporating the tail risks <u>overestimating ITH</u> and <u>losing tumor growth information</u>

MOBSTER

- Only analyzes mutations in diploid regions
- Assumes that the tail makes up a portion π_0 of mutations, and ~ Pareto distribution $\propto x_*^{\alpha} / x^{\alpha+1}$, with lower bound x_* (fixed from data) and power α (to be inferred)
- Assumes that each cluster k makes up a portion π_k of mutations, and ~ Beta with two parameters u_k and v_k
- Infers cluster count K and tail and cluster mutations with EM algorithm



Interactive session 4

MOBSTER requires one input file: **data/MOBSTER_mutation_table.csv**, which lists a mutation in each row, with following columns:

- **mutation_id**: mutation ID
- **vaf**: VAF

Interactive session 4

1. Run MOBSTER with

   docker compose run --rm workshop Rscript src/run_mobster.R

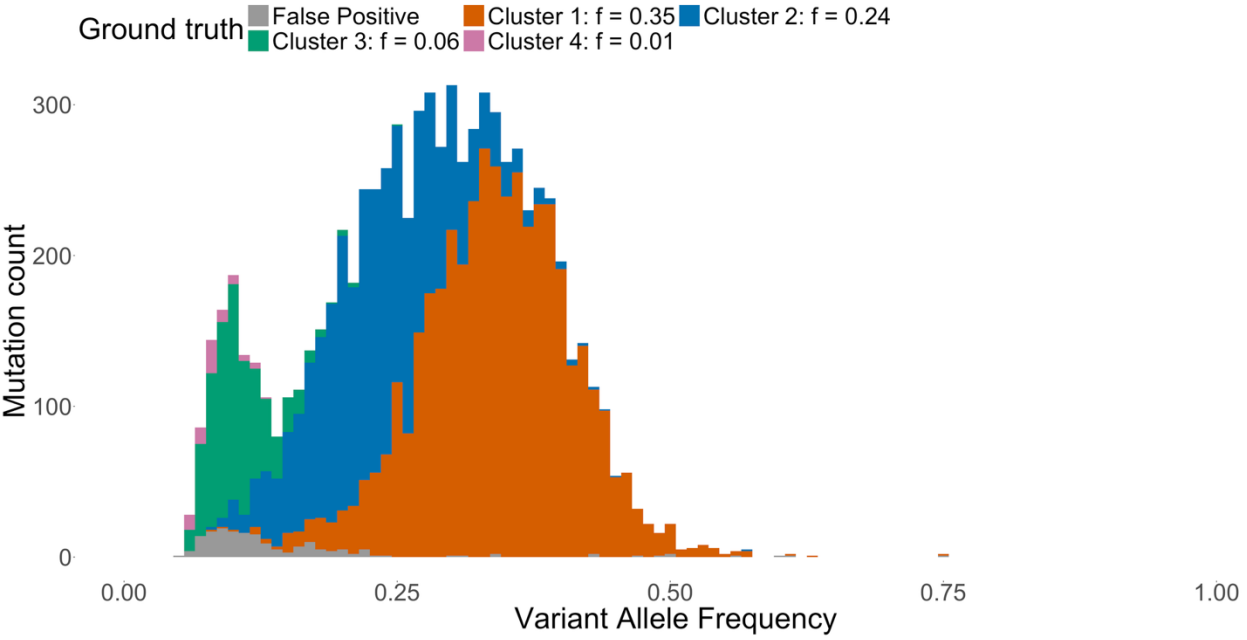
2. Examine the code in **src/run_mobster.R**
3. Examine the results from MOBSTER in **results/mobster/S8-noXY_mobster_fit.pdf** and **results/mobster/clone.tsv**

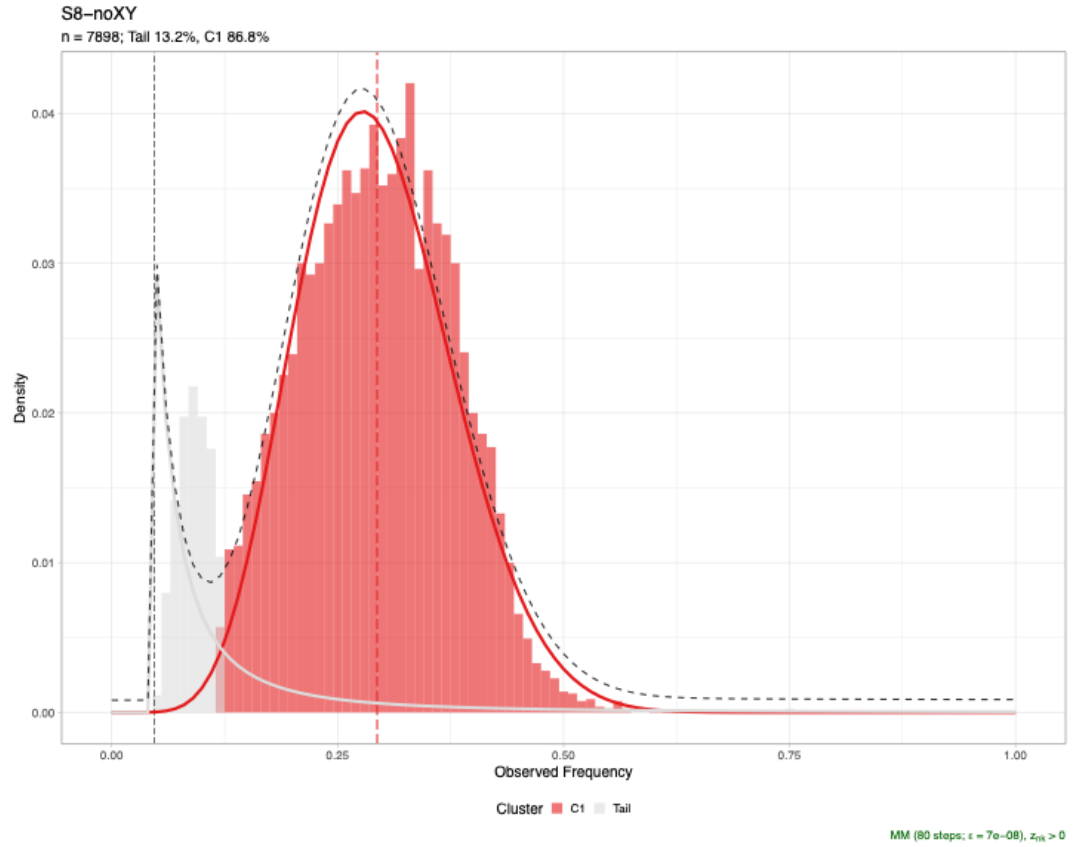
Interactive session 4

Ground truth:

- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: 0.7, 0.48, 0.12, 0.02
- Subclonal mutation counts: 5981, 3401, 1034, 104

purity	n_clones	clone	ccf
0.58888026	1	C1	0.58888026





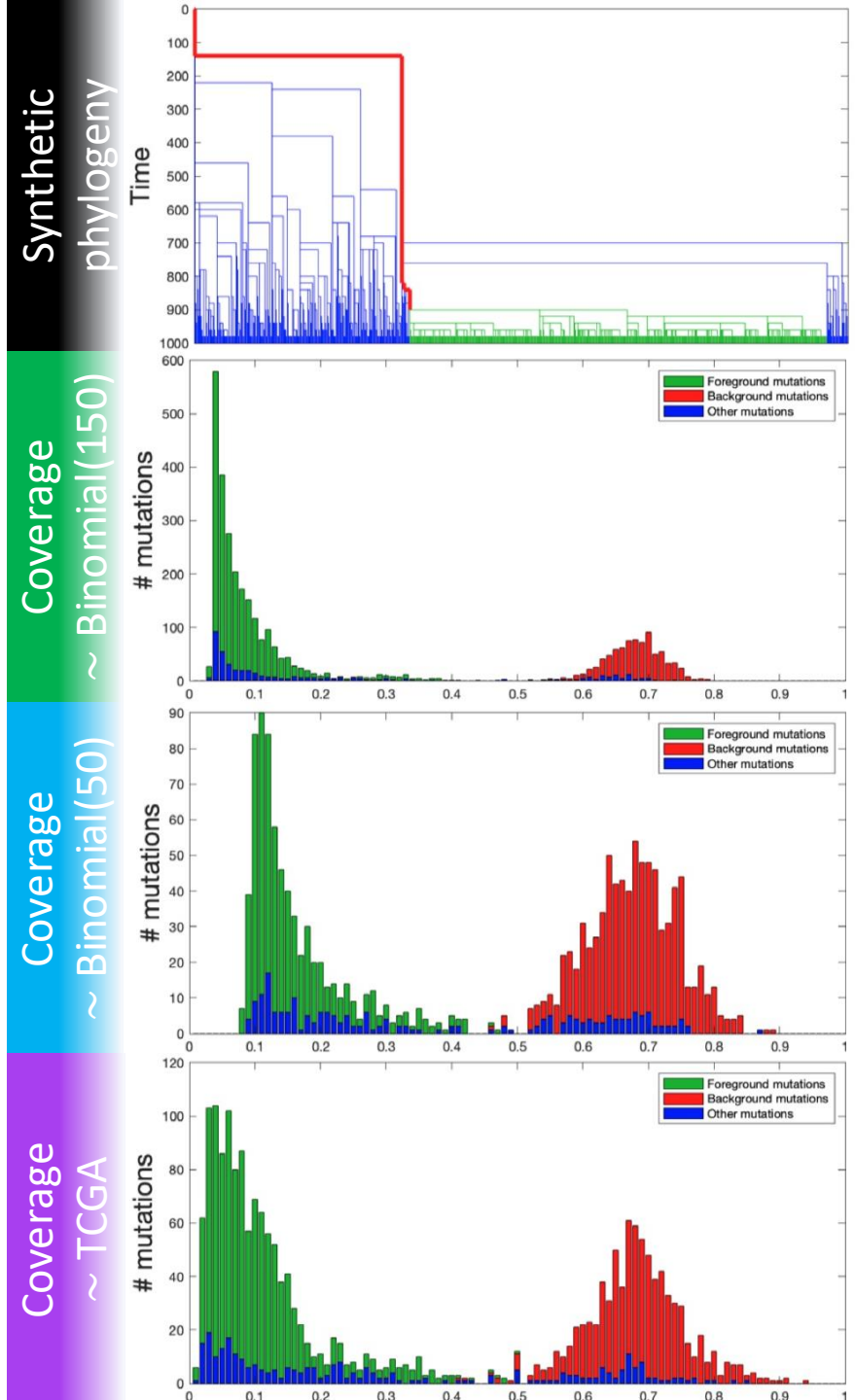
What do we think of the result?

Interactive session 4

	Incorporate CNVs	Multi-samples	Efficient	Tail mutations
PhyloWGS				
PyClone-VI	Multiplicity = 1			
SciClone				
MOBSTER				

Impacts of coverage and filtering

- Many algorithms assume general distribution families (e.g., Beta) for the shape of mutation clusters
- However, the shapes of mutation clusters and especially the tail depends significantly on the coverage distribution
- Furthermore, data filtering (e.g., MuTect2) is biased against rare mutations, which make up the tail
- We therefore need to incorporate the impacts of sequencing coverage and data filtering on the VAF distribution



DECODE

- Sample contains $\boldsymbol{n}$ copies among $\boldsymbol{H}$ subclones. Expected # mutations found in $\boldsymbol{k}$ copies:

$$\mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) = \boldsymbol{\omega_0} \cdot g_{tail}(k|\boldsymbol{\alpha}) + \sum_{h=1}^{H} \boldsymbol{\omega_h} \cdot \boldsymbol{g_{peak}(k|p_h)}$$

where $g_{tail}(k|\boldsymbol{\alpha}) \propto k^{-\alpha}$ is the neutral tail compartment

$\boldsymbol{g_{peak}(k|p_h)} = \binom{n}{k} p_h^k (1-p_h)^{n-k}$ is the mutation cluster of subclone $\boldsymbol{h}$

- The neutral tail is characterized by

$\boldsymbol{\omega_0}$ = number of mutations in the tail $\quad$ $\boldsymbol{\alpha}$ = neutral tail power

- Each cluster $\boldsymbol{h}$ is characterized by

$\boldsymbol{\omega_h}$ = number of mutations in the cluster $\quad$ $\boldsymbol{p_h}$ = mean VAF

Dinh KN, Jaksik R, Kimmel M, Lambert A, Tavaré S. Statistical Science. 2020 Feb 1;35(1):129-44.

DECODE

- Sample contains $\boldsymbol{n}$ copies among $\boldsymbol{H}$ subclones. Expected # mutations found in $\boldsymbol{k}$ copies:

$$\mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) = \boldsymbol{\omega_0} \cdot g_{tail}(k|\boldsymbol{\alpha}) + \sum_{h=1}^{H} \boldsymbol{\omega_h} \cdot \boldsymbol{g_{peak}(k|p_h)}$$

- Define coverage as function $\boldsymbol{\varphi}$, where $\boldsymbol{\varphi(r)} =$ probability that a site is sequenced by $\boldsymbol{r}$ reads
- If a mutation exists in $\boldsymbol{k}$ cells and covered by $\boldsymbol{r}$ total reads: its exact VAF is $\boldsymbol{k/n}$.
- The variant read count then follows $\boldsymbol{z|r} \sim \text{Binomial}\left(\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right)$.

Dinh KN, Jaksik R, Kimmel M, Lambert A, Tavaré S. Statistical Science. 2020 Feb 1;35(1):129-44.

DECODE

- Sample contains $\boldsymbol{n}$ copies among $\boldsymbol{H}$ subclones. Expected # mutations found in $\boldsymbol{k}$ copies:

$$\mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) = \boldsymbol{\omega_0} \cdot g_{tail}(k|\boldsymbol{\alpha}) + \sum_{h=1}^{H} \boldsymbol{\omega_h} \cdot \boldsymbol{g_{peak}(k|p_h)}$$

- The expected count of mutations with observed VAF $\boldsymbol{f} = \frac{\boldsymbol{z}}{\boldsymbol{r}} \in (f_1, f_2] \subset (0,1]$ is then

$$\mathbb{E}\left(\boldsymbol{S}\left((\boldsymbol{f_1}, \boldsymbol{f_2}]\right)\middle|\boldsymbol{\theta_H}\right) = \sum_{k=1}^{n} \mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) \sum_r \boldsymbol{\varphi(r)} \cdot \boldsymbol{B}\left((\boldsymbol{f_1 r}, \boldsymbol{f_2 r}]\middle|\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right)$$

where $\boldsymbol{B}\left((\boldsymbol{f_1 r}, \boldsymbol{f_2 r}]\middle|\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right) = \mathbb{P}\left(\boldsymbol{z} \sim \text{Binomial}\left(\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right) \in (\boldsymbol{f_1 r}, \boldsymbol{f_2 r}]\right)$

Dinh KN, Jaksik R, Kimmel M, Lambert A, Tavaré S. Statistical Science. 2020 Feb 1;35(1):129-44.

DECODE

- Sample contains $\boldsymbol{n}$ copies among $\boldsymbol{H}$ subclones. Expected # mutations found in $\boldsymbol{k}$ copies:

$$\mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) = \boldsymbol{\omega_0} \cdot g_{tail}(k|\boldsymbol{\alpha}) + \sum_{h=1}^{H} \boldsymbol{\omega_h} \cdot \boldsymbol{g_{peak}(k|p_h)}$$

- The expected count of mutations with observed VAF $\boldsymbol{f} = \frac{\boldsymbol{z}}{\boldsymbol{r}} \in (f_1, f_2] \subset (0,1]$ is then

$$\mathbb{E}\left(\boldsymbol{S}\left((\boldsymbol{f_1}, \boldsymbol{f_2}]\right)\middle|\boldsymbol{\theta_H}\right) = \sum_{k=1}^{n} \mathbb{E}(\boldsymbol{S^n(k)}|\boldsymbol{\theta_H}) \sum_r \boldsymbol{\varphi(r)} \cdot \boldsymbol{B}\left((\boldsymbol{f_1 r}, \boldsymbol{f_2 r}]\middle|\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right)$$

- Impact of filtering:

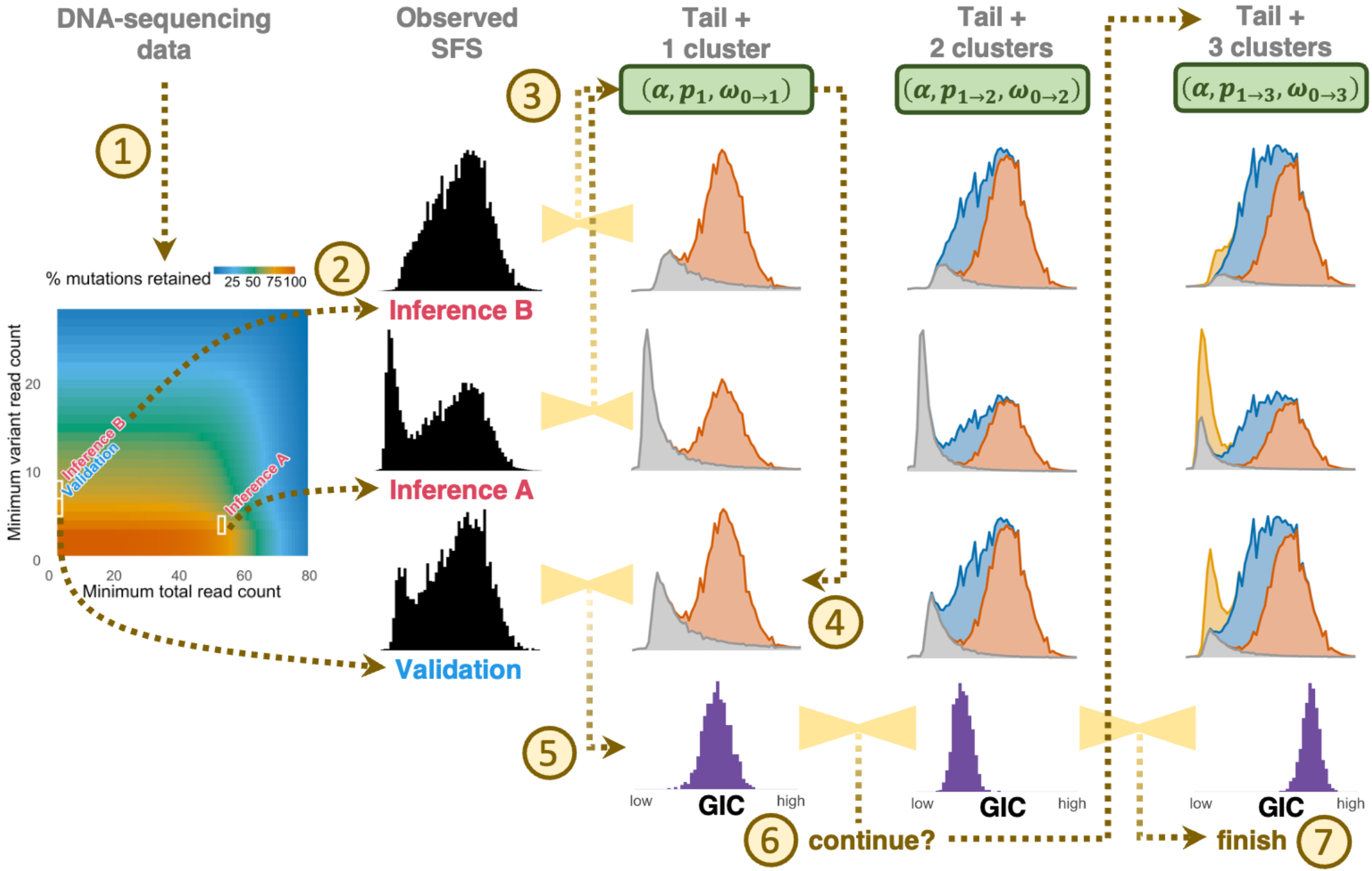
$\boldsymbol{M}$ = minimum total read count $\boldsymbol{L}$ = minimum variant read count

- Filtering with $\boldsymbol{r} \geq \boldsymbol{M}$ and $\boldsymbol{z} \geq \boldsymbol{L}$: condition $\boldsymbol{\varphi(r)}$ so $\boldsymbol{\varphi(r)} = 0$ for $r < M$; and replace $\boldsymbol{B}$

with $\boldsymbol{B}\left((\boldsymbol{f_1 r}, \boldsymbol{f_2 r}]\middle|\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}, \boldsymbol{L}\right) = \mathbb{P}\left(\boldsymbol{z} \sim \text{Binomial}\left(\boldsymbol{r}, \frac{\boldsymbol{k}}{\boldsymbol{n}}\right) \in (\boldsymbol{f_1 r}, \boldsymbol{f_2 r}] \cap [\boldsymbol{L}, \infty)\right)$

Dinh KN, Jaksik R, Kimmel M, Lambert A, Tavaré S. Statistical Science. 2020 Feb 1;35(1):129-44.

DECODE



Chen Y, Jaksik R, Terranova P, El Baghdadi S, Koval A, Kurpas MK, Tavare S, Kimmel M, Dinh KN. bioRxiv. 2026:2026-06.

Interactive session 5

DECODE requires one input file: **data/DECODE_mutation_table.tsv**, which lists a mutation in each row, with following columns:

- **mutation_id**: mutation ID
- **Ref_count**: reference read count
- **Alt_count**: variant read count

Interactive session 5

1. Run DECODE with

   docker compose run --rm workshop Rscript src/run_decode.R

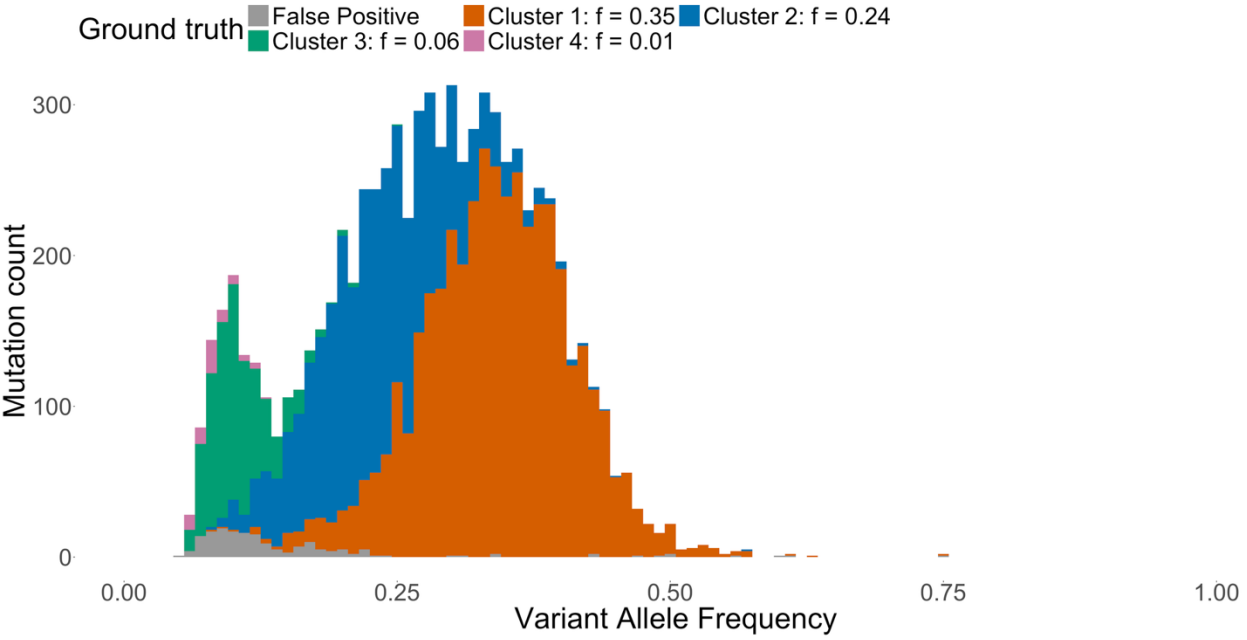
2. Examine the code in **src/run_decode.R**
3. Examine the results from DECODE in **results/decode/S8-noXY_DECODE_SFS.pdf**, **results/decode/ S8-noXY_DECODE_model_selection.pdf** and **results/decode/clone.tsv**

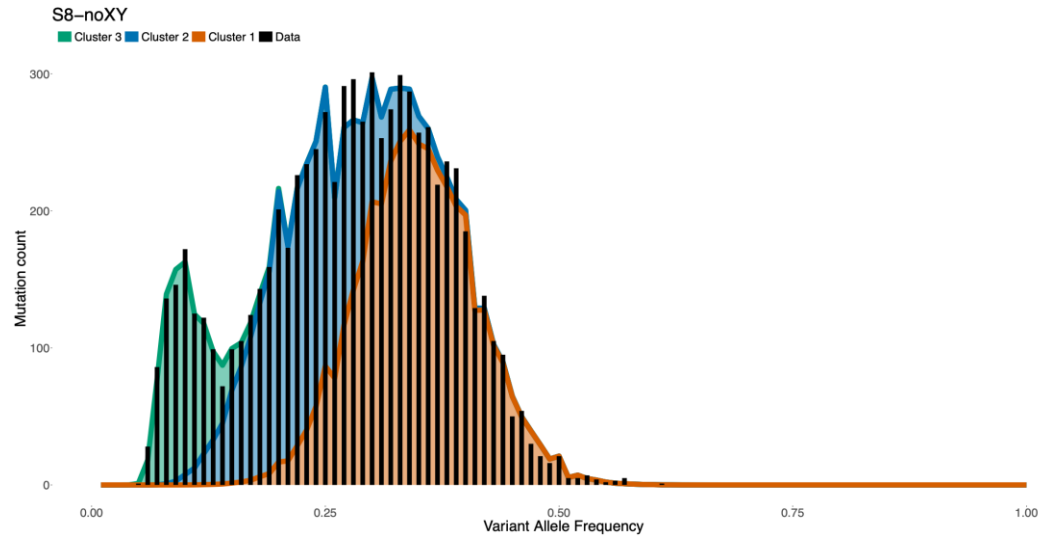
Interactive session 5

Ground truth:

- Sample purity: 0.7
- Clone count: 4
- Subclonal CCFs: $0.7, 0.48, 0.12, 0.02$
- Subclonal mutation counts: $5981, 3401, 1034, 104$

purity	n_clones	clone	ccf
0.68736809	3	1	0.68736809
0.68736809	3	2	0.45420674
0.68736809	3	3	0.14570129





What do we think of the result?

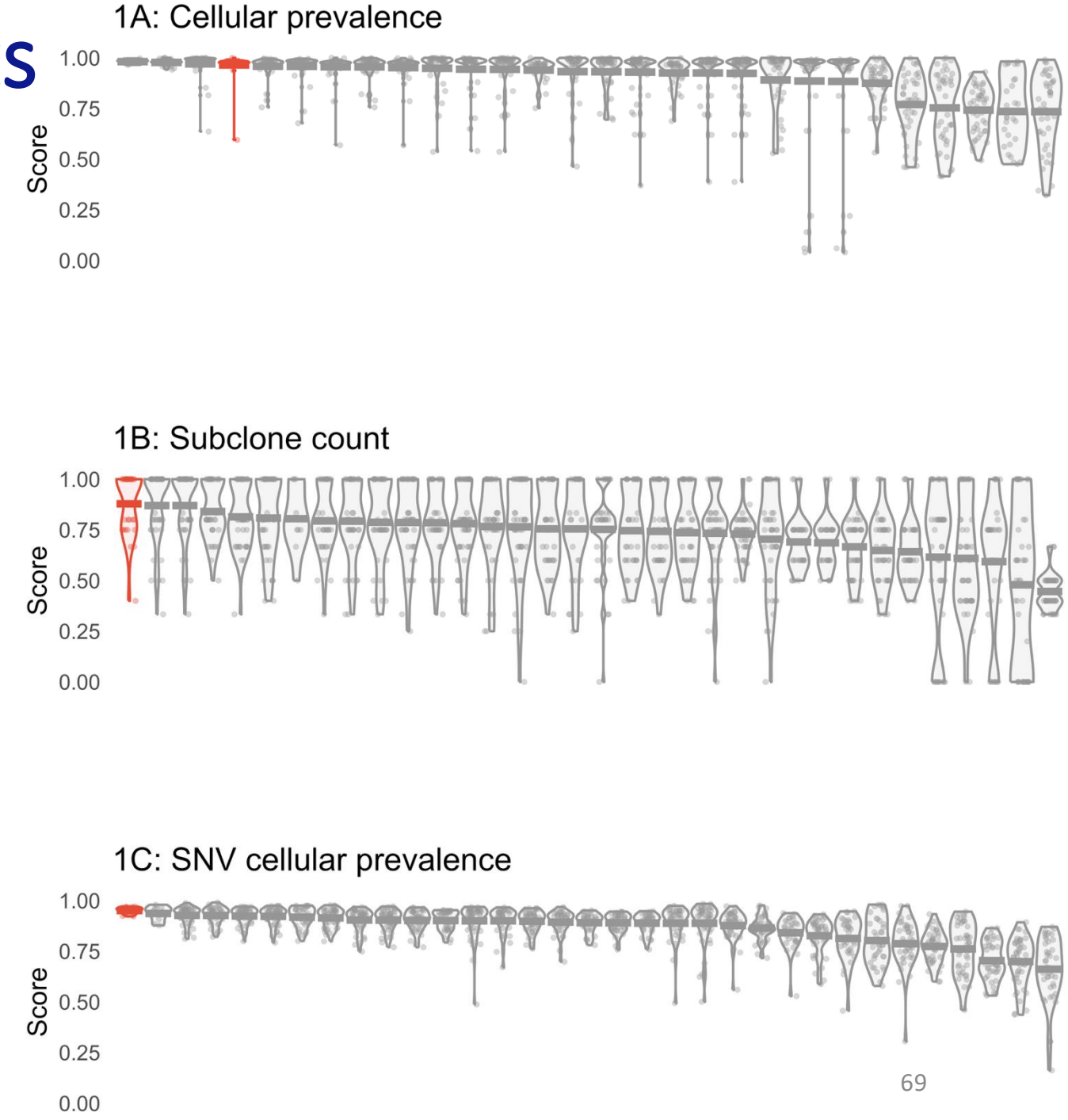
Interactive session 5

	Incorporate CNVs	Multi-samples	Efficient	Tail mutations	Coverage & filtering effects
PhyloWGS					
PyClone-VI	Multiplicity = 1				
SciClone					
MOBSTER					
DECODE					

ICGC/TCGA DREAM results

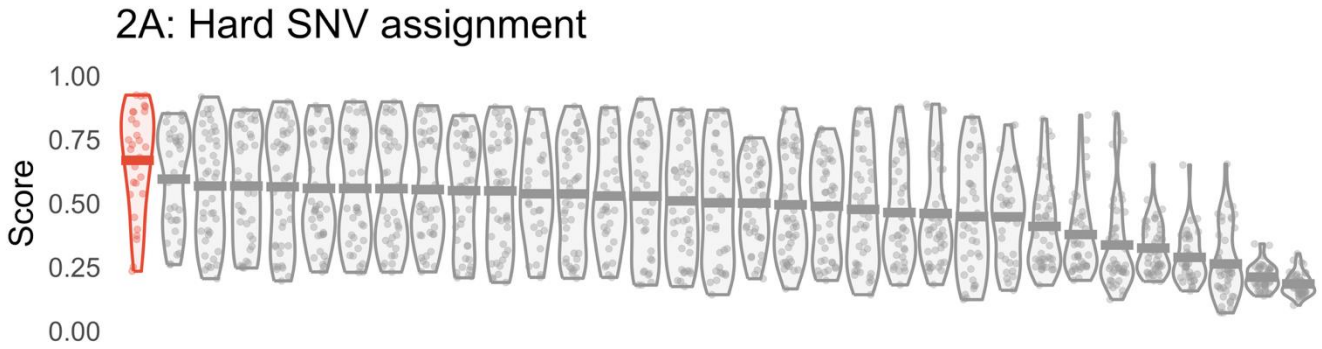
DECODE's results from the ICGC/TCGA DREAM Challenge (51 realistically simulated tumors, against 31 previous methods):

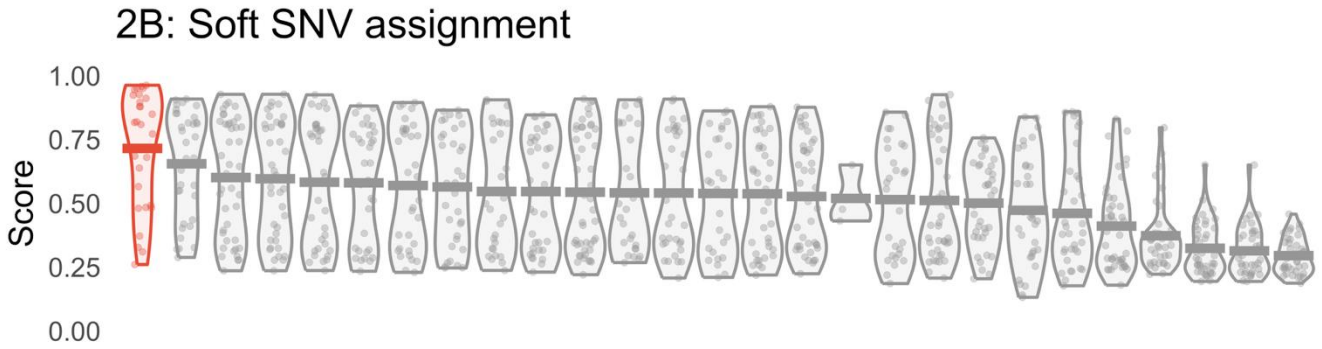
- 4th in inferring sample purity (4% < best method)
- 1st in inferring subclone count (2% > previous best method)
- 1st in inferring subclonal VAFs & mutation counts (5% > previous best method)



ICGC/TCGA DREAM results

- 1st in assigning mutations to each subclone, either deterministically (18% > previous best method) or probabilistically (17% > previous best method)



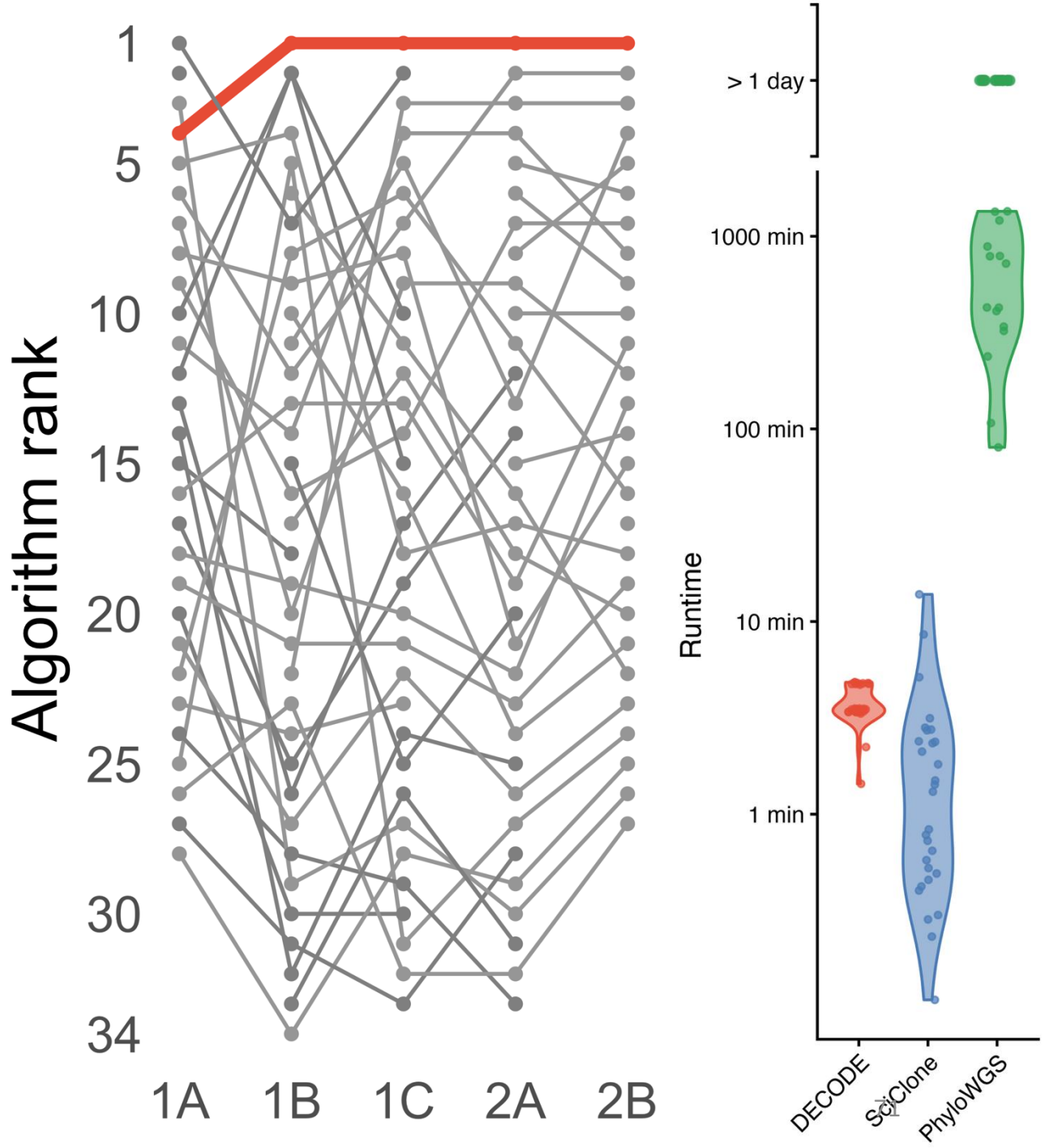


ICGC/TCGA DREAM results

- Most methods excel on only a few objectives

- In contrast, DECODE performs strongly across all DREAM tasks

- DECODE takes ≈ 4 minutes on average per sample, similar to SciClone, MOBSTER, PyClone and much faster than PhyloWGS



Applications in Acute Myeloid Leukemia (AML)

- AML is among the main types of leukemia.

- Data: 10 patients, each with DNA samples at diagnosis (Dx) and relapse (Rx).

  All achieved complete remission after treatment between Dx and Rx.

- Methods: we compare SciClone (can decompose joint SFS; only considers clusters),

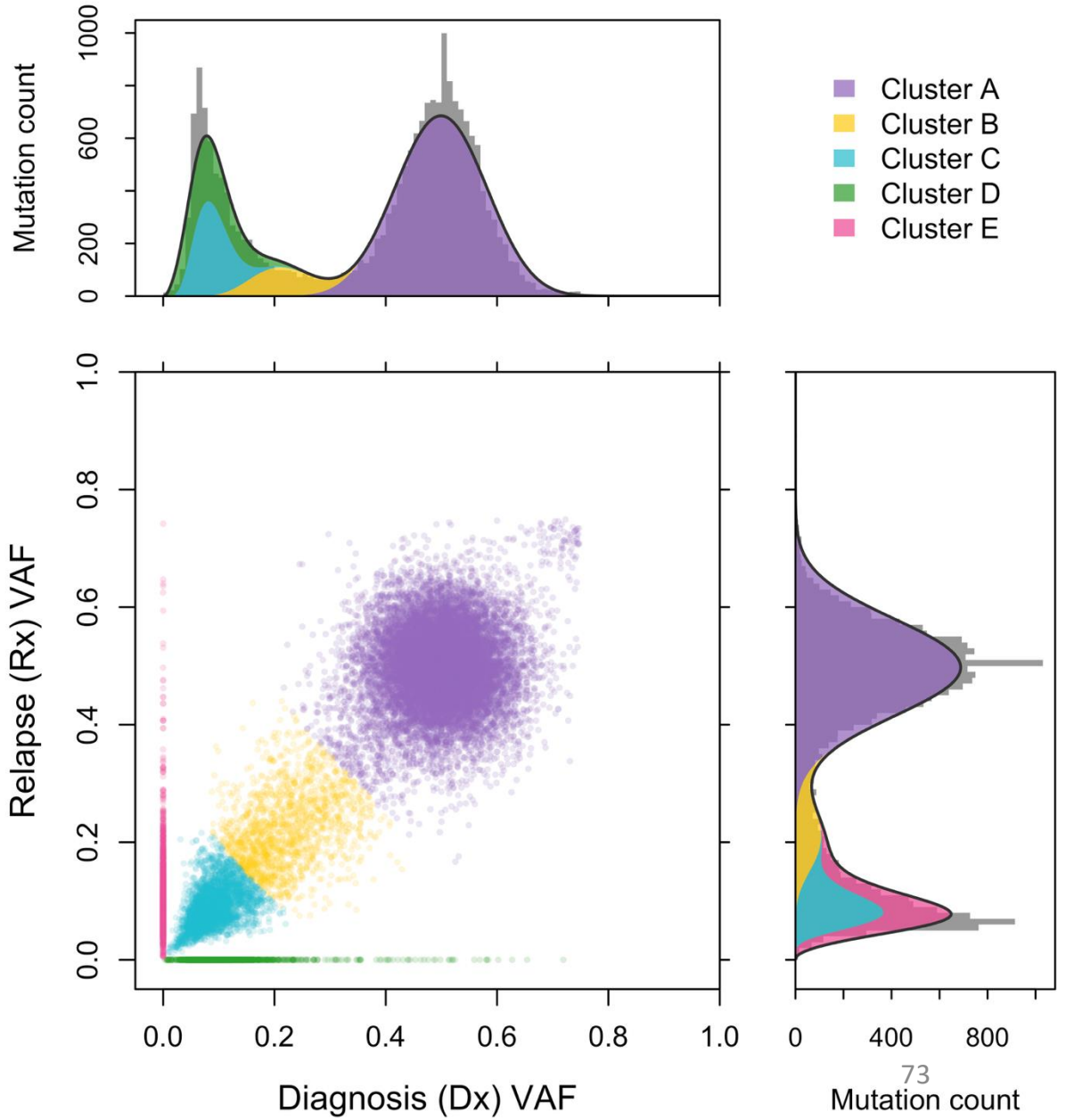
  DECODE and MOBSTER (only methods that model the SFS tail; only decompose single SFS)

Applications in Acute Myeloid Leukemia (AML)

- Multi-sample deconvolution with SciClone infers 4-7 subclones per patient.

Example in Patient 1:

- Subclones A, B and C are shared at both time points.
- Subclone D is present only at Dx.
- Subclone E is present only at Rx.

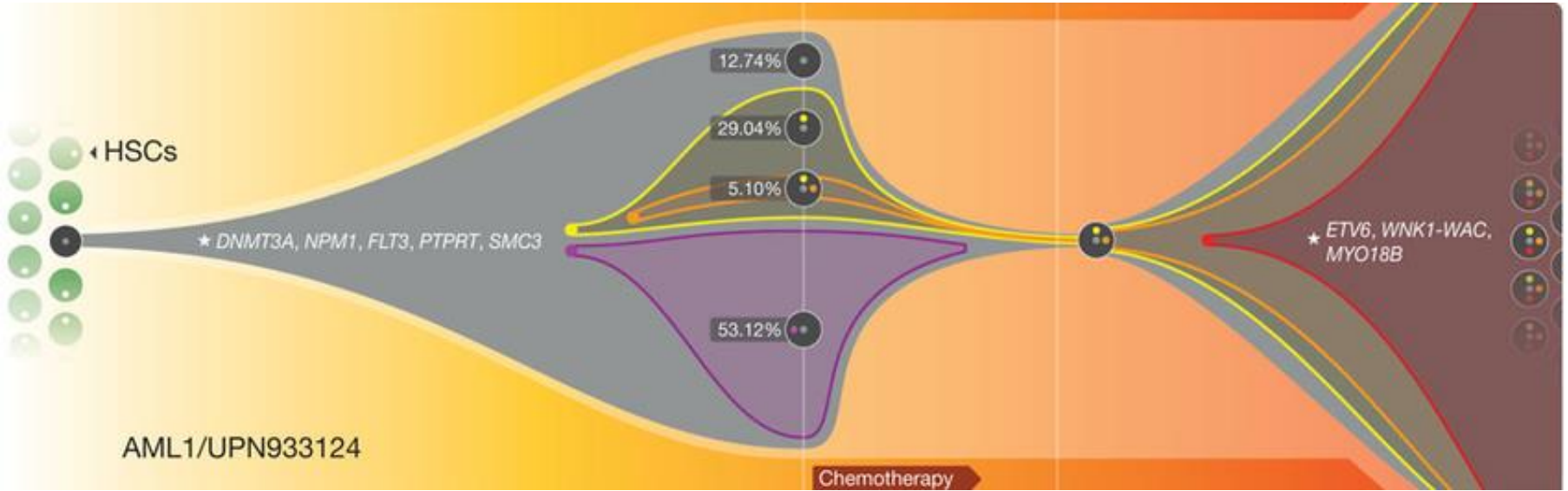


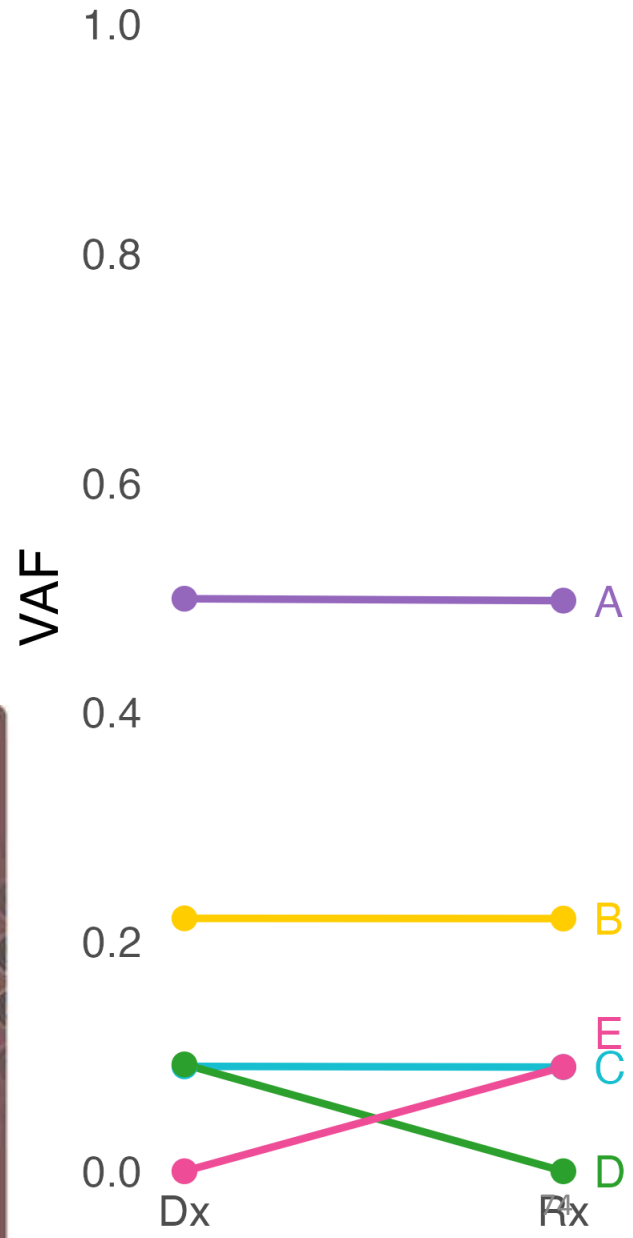
Applications in Acute Myeloid Leukemia (AML)

- SciClone's mean VAFs of the shared subclones are constant between Dx and Rx

- The implied clonal selection equilibrium is unrealistic

- What if some of these subclones actually constitute the tail?



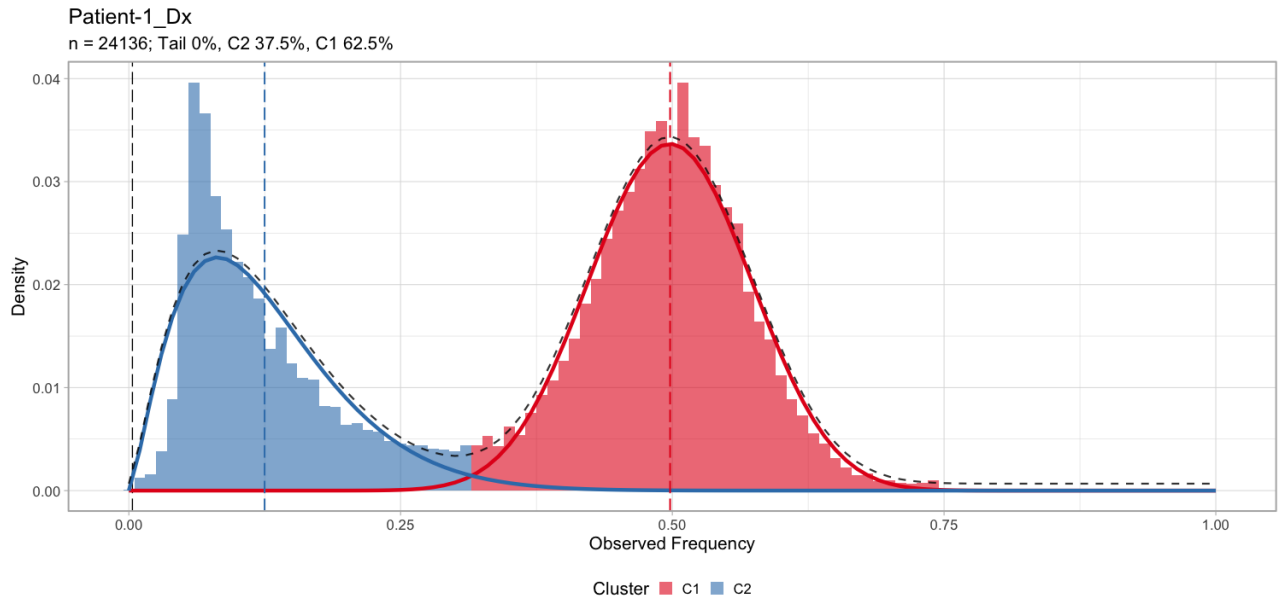


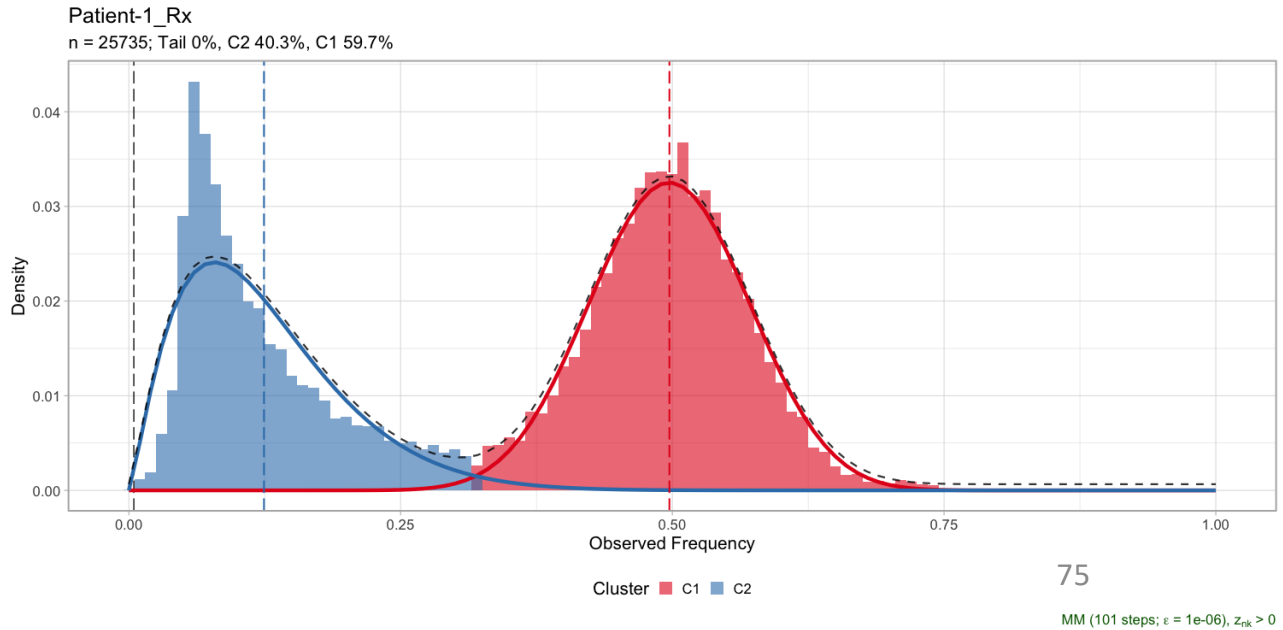
Applications in Acute Myeloid Leukemia (AML)

- MOBSTER distinguishes 2-3 subclones per sample, but detects the neutral tail in only one sample (coverage $137\times$)

- Other $19/20$ samples have significantly lower coverage ($45-70\times$) compared to MOBSTER's tail detection sensitivity ($100\times$)

- This is likely due to the lack of coverage impact in MOBSTER's formulation

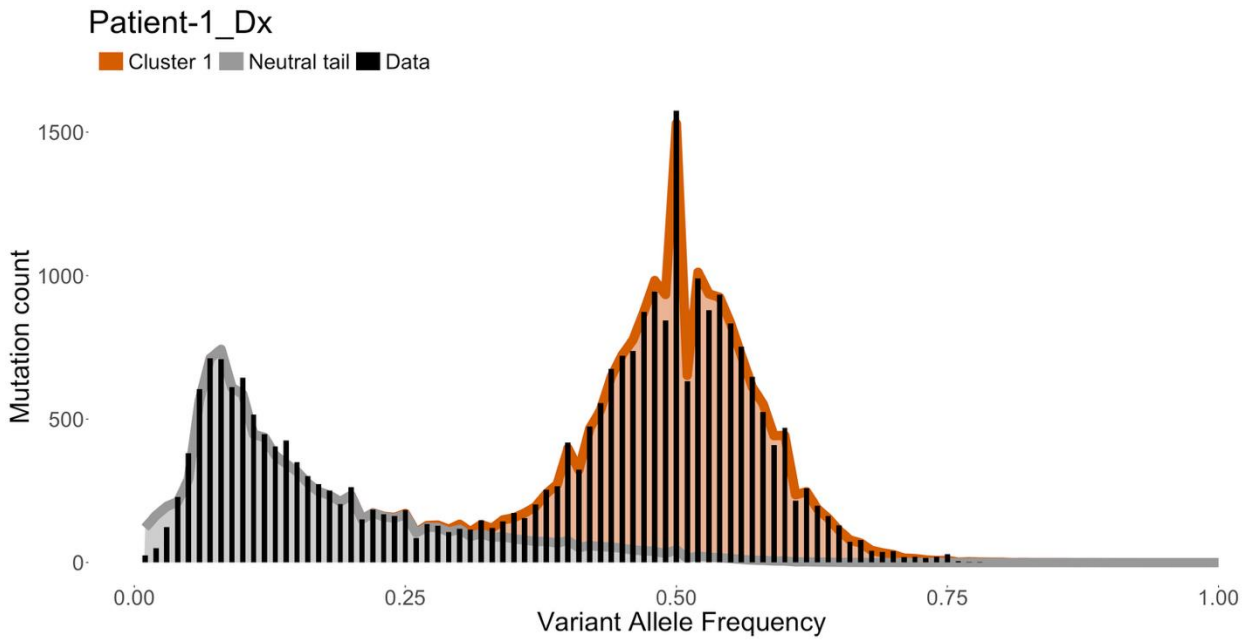


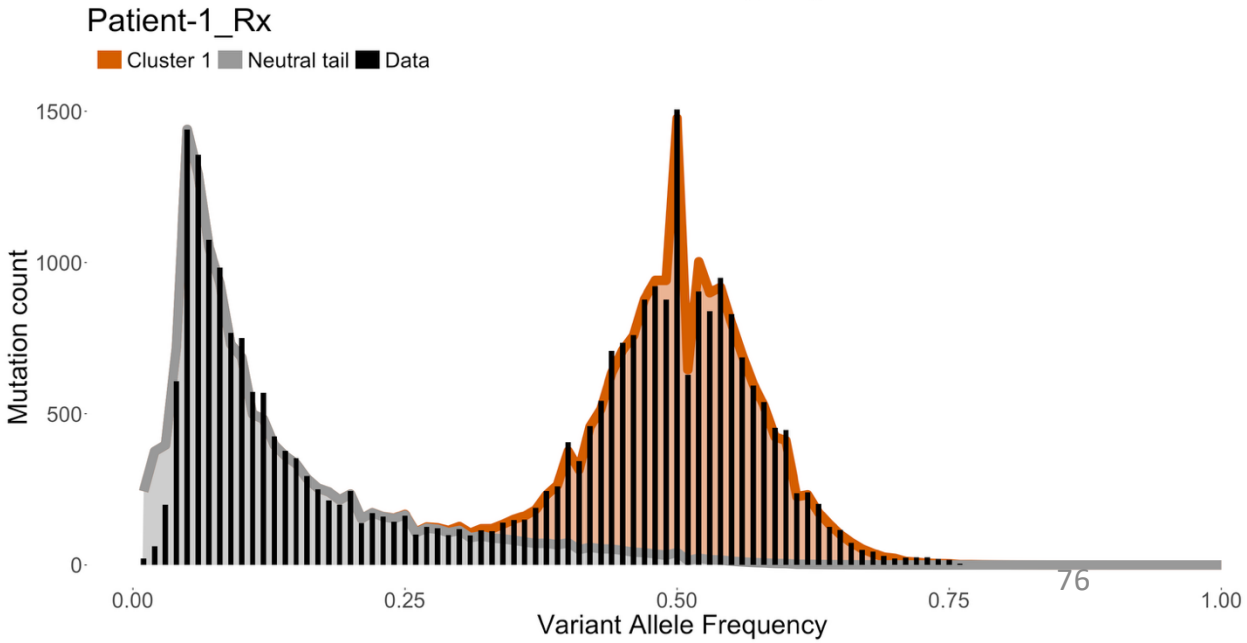


Applications in Acute Myeloid Leukemia (AML)

- DECODE distinguishes 1-3 subclones per sample, and detects the neutral tail in all 20 samples

- What evolution model is consistent with DECODE's results?

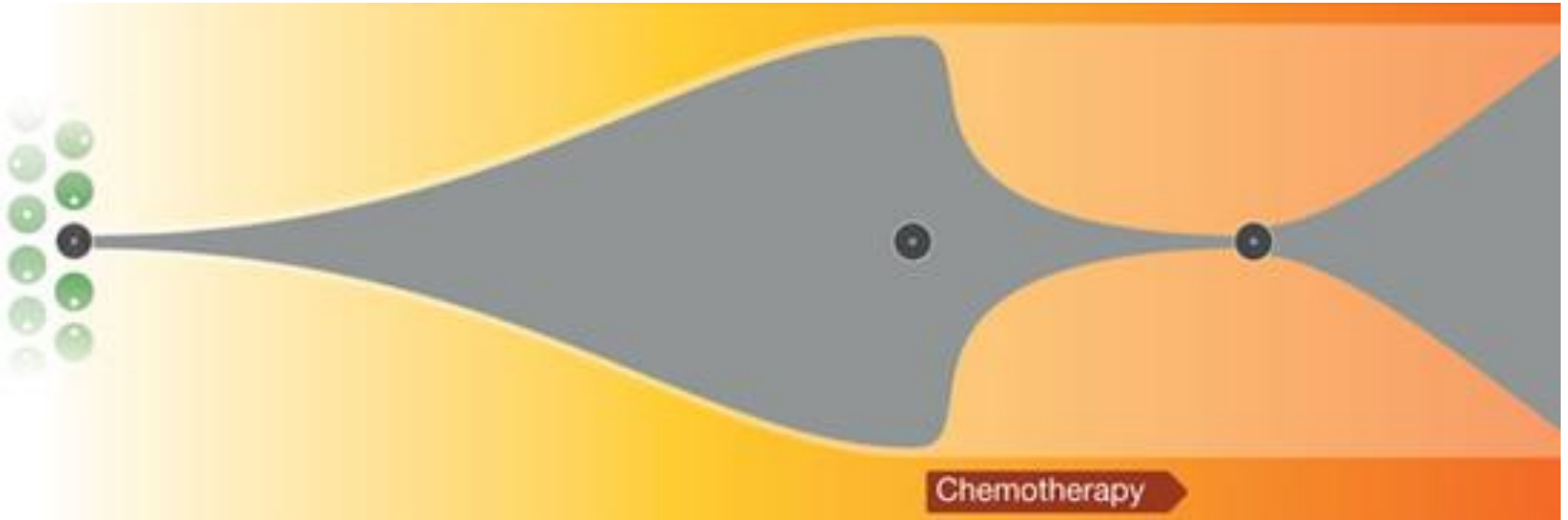


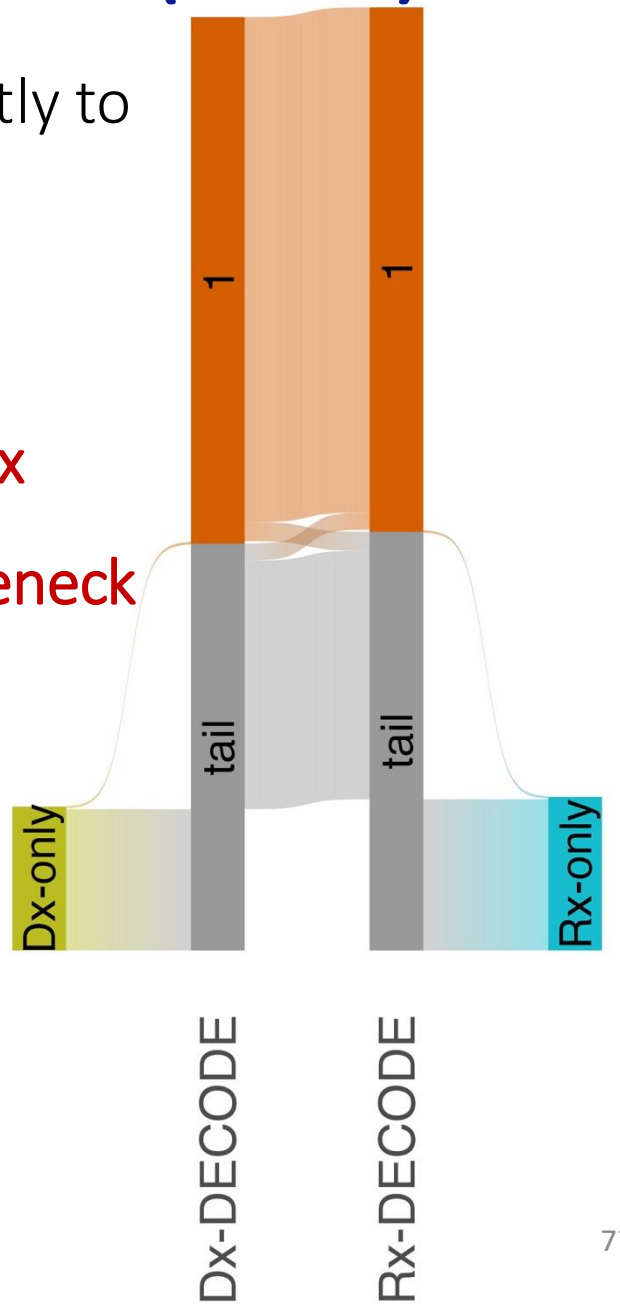


Applications in Acute Myeloid Leukemia (AML)

- Mutations present at both time points are assigned consistently to the same components by DECODE

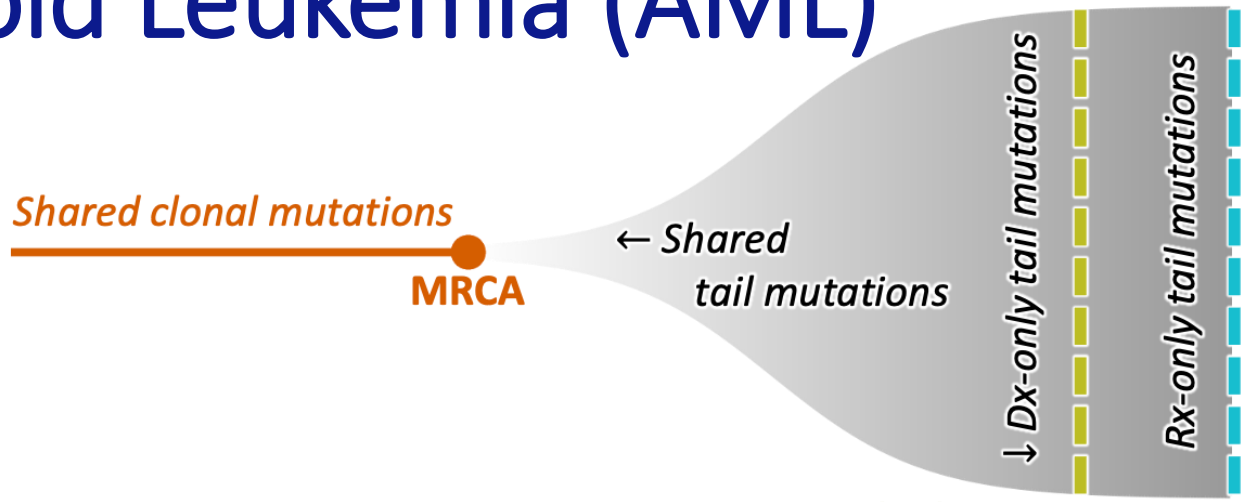
- We therefore hypothesize that Dx consists of one clone, and Rx reflects its continued expansion following chemotherapy bottleneck

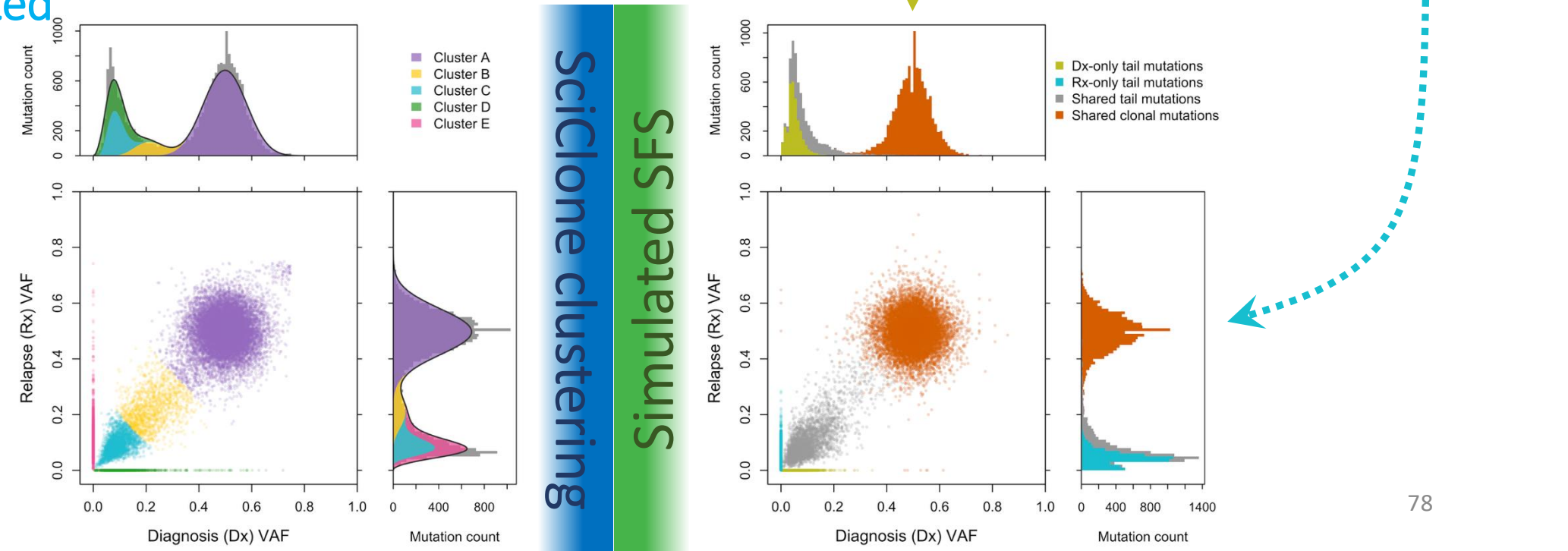




Applications in Acute Myeloid Leukemia (AML)

- SFS simulated under this model are indeed consistent with the empirical joint SFS pattern and in agreement with the original work, which found that the Dx and Rx cells are highly related

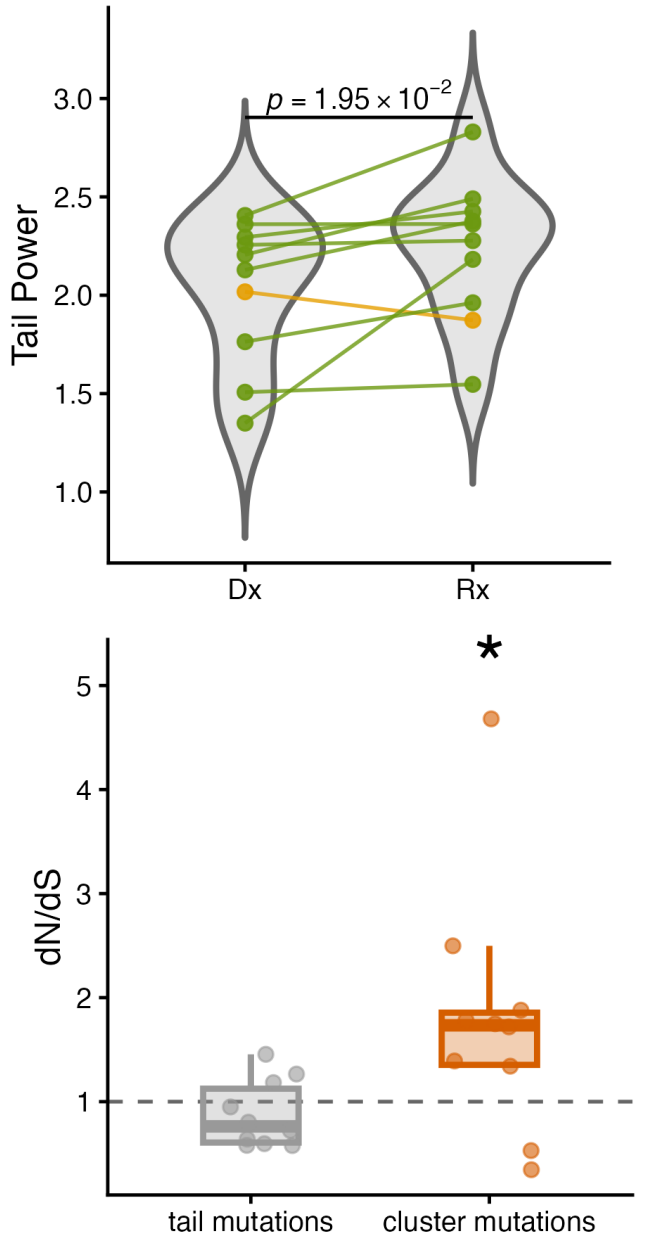




Applications in Acute Myeloid Leukemia (AML)

- Even without clonality, DECODE predicts that Rx is more aggressive than Dx (higher tail power), consistent with known biology

- dN/dS analysis further confirms that tail mutations are neutral and cluster mutations are positively selected



Conclusions

- We have covered basic terminologies in DNA-sequencing analysis and different available software for mutation clustering.
- Thank you for your participation in this workshop!
- Please feel free to contact us if you have questions or need help with DECODE.

